

Q.1 (a) Crawling vs. Indexing Phases of a Web Search Engine [3 Marks]

In the architecture of a web search engine (Ref. [1], Goodrich et al., Ch. 12.5), **crawling** and **indexing** form the data acquisition and organization pipeline that precedes query evaluation:

Dimension	Crawling Phase (Web Crawler / Spider)	Indexing Phase (Indexer)
Primary Objective	Discovery & Retrieval: Systematically discover, download, and fetch web pages across the World Wide Web.	Parsing & Organization: Process raw document contents into an optimized data structure for rapid retrieval.
Operational Mechanism	Starts from a set of seed URLs , uses a frontier queue (FIFO / priority queue), resolves DNS, issues HTTP requests, extracts hyperlinks from HTML, and respects politeness rules (robots.txt).	Performs lexical analysis : HTML stripping, tokenization, stop-word elimination, stemming/lemmatization, and builds an inverted index (or compressed trie/lexicon).
Bottleneck & Output	Network I/O, bandwidth, latency, and duplicate detection. Produces a raw document repository / page cache .	Disk/RAM I/O and CPU. Produces a search index (vocabulary lexicon mapped to postings lists containing document IDs, term frequencies, and word offsets).

Q.2 (b) Amortized Analysis vs. Average-Case Analysis [3 Marks]

(Ref. [2], CLRS 4th ed., Ch. 16)

1. Fundamental Contrast:
- **Amortized Analysis:** Guarantees the **average performance of each operation in the worst-case sequence of operations**. It establishes an upper bound $T(n)$ on the total cost of any sequence of n operations, giving an amortized cost per operation of $\frac{T(n)}{n}$. This guarantee holds over **all** possible sequences—even adversarial ones.
 - **Average-Case Analysis:** Evaluates the **expected running time** of an operation under an assumed probability distribution over the input domain (e.g., assuming all input permutations are equally likely). If actual inputs deviate from the assumed distribution, the average-case bound ceases to hold.
2. Why Amortized Analysis Does Not Require Probabilistic Assumptions:
- Amortized analysis is **strictly deterministic**.
 - It relies on **structural state invariants** inherent to the data structure (e.g., an expensive operation can occur only after a sufficient number of cheap operations have modified the state or accumulated prepaid credits).
 - Because it bounds the total actual cost of the *worst possible sequence* of operations ($\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$), no random variables, probability distributions, or expected values are required.

Q.3 (c) 4-Bit Binary Counter via the Accounting Method [4 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

Consider a 4-bit binary counter $A[0..3]$ initialized to 0000_2 , supporting the `INCREMENT` operation.

1. Actual Cost Model

Let flipping a single bit (either $0 \rightarrow 1$ or $1 \rightarrow 0$) have an actual cost of:

$c = 1$ unit.

In an `INCREMENT` , the algorithm scans bits from index 0 upward:

- It flips k consecutive 1s into 0s (where $k \geq 0$).
- If $k < 4$, it flips the lowest 0 into a 1.
- Actual cost: $c_i = k + 1$ (or k if all bits overflow to 0).

2. Amortized Cost Assignment

- **Flipping a bit from 0 $\rightarrow$ 1:** Assign an amortized cost of:

$$\hat{c} = 2 \text{ units.}$$

- 1 unit immediately pays for the actual flip (0 $\rightarrow$ 1).
- 1 unit is deposited as **credit** directly on the newly set 1 bit.
- **Flipping a bit from 1 $\rightarrow$ 0:** Assign an amortized cost of:

$$\hat{c} = 0 \text{ units.}$$

- The actual cost of 1 unit is paid for entirely by consuming the 1 unit of credit already residing on that 1 bit.

3. Credit Invariant

Credit Invariant: Every bit currently holding the value 1 carries exactly 1 unit of credit; bits holding 0 carry 0 credit.

- At the start (0000₂), the total credit is 0.
- After any sequence of operations, the total accumulated credit is:

$$\text{Total Credit} = \sum_{i=1}^n (\hat{c}_i - c_i) = \text{number of 1-bits in the counter} \geq 0.$$

- Because the total credit remains non-negative at all times, the total amortized cost is an absolute upper bound on the total actual cost: $\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$.

4. Amortized Cost per INCREMENT

In any INCREMENT step:

- At most one bit is flipped from 0 $\rightarrow$ 1 (charge = 2).
- All other k bits flipped are 1 $\rightarrow$ 0 (charge = 0).

$$\hat{c}_{\text{INCREMENT}} = 2 + k \cdot 0 = 2 \text{ units } (O(1)).$$

For any sequence of n operations starting from 0, the total cost is at most $2n$, yielding an amortized cost of $O(1)$ **per increment**.

Q.4 (d) Class NP & Polynomial-Time Certification for 3-SAT [5 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.3)

1. Definition of NP via Polynomial-Time Certification

A decision problem $X \subseteq \{0, 1\}^*$ belongs to the complexity class **NP** if there exists a polynomial $p(\cdot)$ and a deterministic polynomial-time algorithm $B(\cdot, \cdot)$ (called a **certifier**) such that for every input string s :

$$s \in X \iff \exists \text{ certificate } t \text{ with } |t| \leq p(|s|) \text{ such that } B(s, t) = \text{"yes"}.$$

- **Completeness:** If $s \in X$, there exists a polynomial-size certificate t verifying this fact.
- **Soundness:** If $s \notin X$, no certificate t can trick $B(s, t)$ into outputting "yes".
- **Efficiency:** The certifier $B(s, t)$ terminates in deterministic time $O(|s|^c)$ for some constant c .

2. 3-SAT Formulation

- **Instance s :** A boolean formula Φ in 3-Conjunctive Normal Form (3-CNF) over n boolean variables $X = \{x_1, x_2, \dots, x_n\}$ consisting of m clauses $C_1 \wedge C_2 \wedge \dots \wedge C_m$, where each clause $C_j = (l_{j,1} \vee l_{j,2} \vee l_{j,3})$ contains exactly 3 literals.
- **Question:** Does there exist a truth assignment to X that satisfies Φ ?

3. Certificate t

- The certificate t is a proposed **truth assignment** $\tau : X \rightarrow \{\text{True}, \text{False}\}$, encoded as an array or bit-vector of length n :

$$t = (\tau(x_1), \tau(x_2), \dots, \tau(x_n)).$$

- Size: $|t| = n \leq |s|$, which is strictly polynomial (linear) in instance size.

4. Deterministic Polynomial-Time Certifier Algorithm $B(\Phi, t)$

```

Algorithm Certifier_3SAT(Phi, tau):
1. Verify that tau specifies a valid boolean value for each variable x_1, ..., x_n.
2. For each clause C_j in Phi (j = 1 to m):
    clause_satisfied = False
    For each of the 3 literals l in C_j:
        If l is a variable x_i and tau[x_i] == True:
            clause_satisfied = True
            break
        If l is a negated variable NOT(x_i) and tau[x_i] == False:
            clause_satisfied = True
            break
    If clause_satisfied == False:
        Return "no"    // Formula is unsatisfied under assignment tau
3. Return "yes"      // All m clauses are satisfied

```

5. Complexity Verification

- Evaluating each literal takes $O(1)$ time via array indexing into τ .
- Each clause has 3 literals, taking $O(1)$ operations per clause.
- Evaluating all m clauses requires $O(m)$ steps.
- Total running time: $O(n + m) = O(|s|)$, which is strictly linear and therefore **deterministic polynomial time**. Hence, 3-SAT $\in \mathbf{NP}$.

Q.5 (e) Brute Force String Matching: Adversarial Worst-Case [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.1)

1. Adversarial Construction

For text length $n = 12$ and pattern length $m = 4$:

- Let alphabet $\Sigma = \{'A', 'B'\}$.
- **Text T :** "AAAAAAAAAAAA" ($n = 12$, twelve 'A' s)
- **Pattern P :** "AAAB" ($m = 4$, three 'A' s followed by 'B')

(Alternatively, $T = \text{"AAAAAAAAAAAA"}$ with $P = \text{"AAAA"}$ achieves the identical worst-case comparison count).

2. Worst-Case Mechanism

At every alignment position i , the algorithm matches characters along P until the final character $P[m - 1]$, where it encounters a mismatch:

- $P[0] = 'A' == T[i + 0]$ (match)
- $P[1] = 'A' == T[i + 1]$ (match)

- $P[2] = 'A' == T[i + 2]$ (match)
- $P[3] = 'B' \neq T[i + 3]$ (**mismatch** after 4 comparisons)

The algorithm shifts P by only 1 position ($i \leftarrow i + 1$) without retaining any previous comparison information.

3. Exact Comparison Calculation

- Total number of shift positions:

$$\text{Shifts} = n - m + 1 = 12 - 4 + 1 = 9 \text{ alignment positions } (i = 0, 1, 2, \dots, 8).$$

- Breakup of character comparisons per shift:
- Shift $i = 0$ ($T[0..3]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 1$ ($T[1..4]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 2$ ($T[2..5]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 3$ ($T[3..6]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 4$ ($T[4..7]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 5$ ($T[5..8]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 6$ ($T[6..9]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 7$ ($T[7..10]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)
- Shift $i = 8$ ($T[8..11]$ vs $P[0..3]$): 4 comparisons ($A==A$, $A==A$, $A==A$, $A!=B$)

$$\text{Total Comparisons} = (n - m + 1) \times m = 9 \times 4 = \mathbf{36} \text{ comparisons.}$$

Q.6 (f) Suffix Trie $O(n^2)$ Space vs. Suffix Tree $O(n)$ Space [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5; Ref. [4], Drozdek, Ch. 7)

1. Why a Standard Suffix Trie Requires $O(n^2)$ Space

A standard suffix trie for a string S of length n (terminated by a unique character $\$$):

- Stores all n suffixes: $S[0..n - 1], S[1..n - 1], \dots, S[n - 1..n - 1]$.
- Every edge represents exactly **one character**.
- **Worst-Case String:** Consider a string with all distinct characters, e.g., $S = a_1a_2a_3 \dots a_n\$$.
- Since no two suffixes share a non-empty common prefix, each suffix forms an independent path branching directly from the root.
- Suffix 0 has length $n + 1 \implies n + 1$ nodes.
- Suffix 1 has length $n \implies n$ nodes.
- ...
- Suffix n has length 1 $\implies$ 1 node.
- Sum of nodes across paths:

$$\text{Total Nodes} = 1 + \sum_{i=1}^{n+1} i = 1 + \frac{(n+1)(n+2)}{2} = \Theta(n^2).$$

- Even for strings with repetitions such as $a^k b^k$, the standard trie has $\Theta(n^2)$ nodes.

2. How a Suffix Tree Achieves $O(n)$ Space

A **suffix tree** is a compressed suffix trie that resolves this quadratic growth through two structural mechanisms:

1. Path Compression (Node Compaction):

- Any non-branching chain of internal nodes with out-degree 1 is collapsed into a single edge labeled by the concatenation of their characters.
- Consequently, every internal node in a suffix tree is a **branching node** with an out-degree ≥ 2 .

2. Bound on Number of Nodes and Edges:

- Because each of the n distinct suffixes terminates with a unique end-marker $\$$, there are **exactly n leaves** ($L = n$).
- In any rooted tree where every internal node has at least two children, the number of internal nodes I satisfies:

$$I \leq L - 1 = n - 1.$$

- Total number of nodes $V = L + I \leq n + (n - 1) = 2n - 1 = O(n)$.
- Since it is a tree, the total number of edges is $E = V - 1 \leq 2n - 2 = O(n)$.

Q.7 (g) Compact Representation of Compressed Tries [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5)

1. The Bottleneck of Explicit Substring Labels

In a compressed trie (or suffix tree), path compression creates edges labeled with variable-length substrings.

- If edge labels are stored as explicit strings or character arrays, the sum of lengths of these substrings across all edges can still sum to $\Theta(n^2)$ characters in the worst case (e.g., for a suffix tree where edge labels to leaves have lengths $1, 2, \dots, n$).
- This would negate the $O(n)$ node bound.

2. The Compact Index-Pair Representation

To achieve true linear structural storage:

- The original text $T[0..n - 1]$ is stored once in a contiguous primary character array / buffer of size n , taking $O(n)$ space.
- **No edge stores characters explicitly.** Instead, each edge stores a pair of integer indices:

$$(start, end)$$

representing the slice $T[start \dots end]$ in the underlying array.

```
[Standard Edge]    (Parent) ---- "abracadabra" ----> (Child)  [O(k) char bytes]
[Compact Edge]     (Parent) ---- [start=0, end=10] ---> (Child)  [2 integers = O(1) space]
```

3. Why This Guarantees Linear $O(n)$ Storage

- **Constant Size per Edge:** Storing two integers $(start, end)$ takes a constant number of words ($O(1)$ memory, typically 8 or 16 bytes), regardless of how long the substring is.
- **Bounded Number of Edges:** As proven in part (f), path compression guarantees that every internal node has out-degree ≥ 2 , bounding the total number of edges by $E \leq 2n - 2 = O(n)$.
- **Total Memory Accounting:**

$$\text{Total Storage} = \underbrace{O(n)}_{\text{Underlying Text } T} + \underbrace{O(n) \times O(1)}_{\text{Node Records \& Child Pointers}} + \underbrace{O(n) \times O(1)}_{\text{Edge Index Pairs } (start, end)} = \mathbf{O(n)} \text{ space.}$$

This compact representation decouples edge label length from memory consumption, enabling both linear space bounds and $O(n)$ construction algorithms (e.g., Ukkonen's algorithm).

Q.2 (a) Independent Set, Vertex-Cover, Complement Duality, and Reduction [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.2)

1. Formal Decision Versions

- **Independent Set Problem:**
- **Instance:** An undirected graph $G = (V, E)$ and a positive integer $k \leq |V|$.

- **Question:** Does there exist an independent set $S \subseteq V$ of cardinality $|S| \geq k$?
(A set $S \subseteq V$ is an independent set if no two vertices in S are joined by an edge in E , i.e., $\forall u, v \in S, (u, v) \notin E$.)
 - **Vertex-Cover Problem:**
 - **Instance:** An undirected graph $G = (V, E)$ and a positive integer $k \leq |V|$.
 - **Question:** Does there exist a vertex cover $C \subseteq V$ of cardinality $|C| \leq k$?
(A set $C \subseteq V$ is a vertex cover if every edge in E has at least one endpoint in C , i.e., $\forall (u, v) \in E, u \in C \vee v \in C$.)
-

2. The Complement Duality Theorem

Theorem: In any undirected graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set if and only if $V \setminus S$ is a vertex cover.

Proof:

- **Forward Direction ($\implies$):**
Assume $S \subseteq V$ is an independent set.
Let $e = (u, v)$ be an arbitrary edge in E .
By definition of an independent set, u and v cannot **both** belong to S (otherwise $e \in E$ would have both endpoints in S , violating independence).
Therefore, at least one endpoint must not be in S :

$$u \notin S \implies u \in (V \setminus S) \quad \text{or} \quad v \notin S \implies v \in (V \setminus S).$$

Since every edge $e \in E$ has at least one endpoint in $V \setminus S$, the complement $V \setminus S$ is a valid vertex cover of G .

- **Reverse Direction ($\impliedby$):**
Assume $V \setminus S$ is a vertex cover of G .
We prove that S is an independent set by contradiction.
Suppose S is **not** an independent set. Then there must exist two vertices $u, v \in S$ such that $(u, v) \in E$.
Since $u \in S$ and $v \in S$, neither vertex belongs to the complement:

$$u \notin (V \setminus S) \quad \text{and} \quad v \notin (V \setminus S).$$

This implies that the edge $(u, v) \in E$ has **neither** of its endpoints in $V \setminus S$, which directly contradicts the assumption that $V \setminus S$ is a vertex cover.

Hence, no two vertices in S can be adjacent, and S is an independent set. ■

3. Polynomial-Time Reduction: Independent Set $\leq_P$ Vertex-Cover

To show Independent Set $\leq_P$ Vertex-Cover, we construct a reduction algorithm f :

1. **Mapping:** Given an instance of Independent Set $\langle G = (V, E), k \rangle$:
 - Output the Vertex-Cover instance $\langle G' = (V, E), k' \rangle$, where:

$$G' = G \quad \text{and} \quad k' = |V| - k.$$

2. **Computational Complexity:** Computing $|V| - k$ and copying graph G requires $O(|V| + |E|)$ time, which is polynomial in the input size.
3. **Correctness:**

$$\begin{aligned} \langle G, k \rangle \text{ is a YES-instance for Independent Set} &\iff \exists S \subseteq V \text{ such that } S \text{ is an independent set and } |S| \geq k \\ &\iff V \setminus S \text{ is a vertex cover of } G \text{ (by Complement Duality)} \\ &\iff |V \setminus S| = |V| - |S| \leq |V| - k = k' \\ &\iff \langle G', k' \rangle \text{ is a YES-instance for Vertex-Cover.} \end{aligned}$$

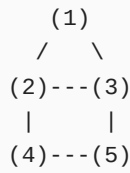
Thus, Independent Set $\leq_P$ Vertex-Cover.

Q.2 (b) Reduction from Vertex Cover to Set Cover on Graph G [8 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.3)

Given the undirected graph $G = (V, E)$ with budget $k = 3$:

- $V = \{1, 2, 3, 4, 5\}$ ($|V| = 5$)
- $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)\}$ ($|E| = 6$)



1. Construction of the Set Cover Instance $(U, \mathcal{S}, k')$

The standard polynomial-time reduction from Vertex Cover to Set Cover maps edges to ground elements and vertices to subsets of incident edges:

1. Universe of Elements (U) :

The universe U is defined as the set of all edges in G :

$$U = E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)\}.$$

(Total elements $|U| = 6$).

2. Family of Subsets $(\mathcal{S})$:

For each vertex $v \in V$, construct a subset $S_v \in \mathcal{S}$ containing all edges incident to v :

$$\mathcal{S} = \{S_1, S_2, S_3, S_4, S_5\}$$

- For vertex 1: incident edges are $(1, 2)$ and $(1, 3)$

$$S_1 = \{(1, 2), (1, 3)\}$$

- For vertex 2: incident edges are $(1, 2)$, $(2, 3)$, and $(2, 4)$

$$S_2 = \{(1, 2), (2, 3), (2, 4)\}$$

- For vertex 3: incident edges are $(1, 3)$, $(2, 3)$, and $(3, 5)$

$$S_3 = \{(1, 3), (2, 3), (3, 5)\}$$

- For vertex 4: incident edges are $(2, 4)$ and $(4, 5)$

$$S_4 = \{(2, 4), (4, 5)\}$$

- For vertex 5: incident edges are $(3, 5)$ and $(4, 5)$

$$S_5 = \{(3, 5), (4, 5)\}$$

3. Target Parameter (k') :

The budget translates directly:

$$k' = k = 3.$$

2. Identification of a Valid Vertex Cover of Size 3 and Corresponding Set Cover

Step 1: Identifying a Vertex Cover of Size $k = 3$

Consider the vertex subset:

$$C = \{2, 3, 4\} \subseteq V \quad (|C| = 3).$$

Let us verify that C covers all edges in E :

- Edge (1, 2): covered by vertex $2 \in C$
- Edge (1, 3): covered by vertex $3 \in C$
- Edge (2, 3): covered by vertices $2, 3 \in C$
- Edge (2, 4): covered by vertices $2, 4 \in C$
- Edge (3, 5): covered by vertex $3 \in C$
- Edge (4, 5): covered by vertex $4 \in C$

Since every edge in E has at least one endpoint in C , $C = \{2, 3, 4\}$ is a **valid vertex cover of size 3**.

(Note: $C' = \{2, 3, 5\}$ and $C'' = \{1, 2, 5\}$ are also valid covers of size 3).

Step 2: Corresponding Subcollection of Subsets in Set Cover

Under the reduction, selecting vertices $\{2, 3, 4\} \subseteq V$ corresponds to selecting the subcollection of subsets:

$$\mathcal{C} = \{S_2, S_3, S_4\} \subseteq \mathcal{S}.$$

- **Size Check:**

$$|\mathcal{C}| = 3 \leq k' = 3.$$

- **Coverage Check:**

$$\begin{aligned} \bigcup_{S_v \in \mathcal{C}} S_v &= S_2 \cup S_3 \cup S_4 \\ &= \{(1, 2), (2, 3), (2, 4)\} \cup \{(1, 3), (2, 3), (3, 5)\} \cup \{(2, 4), (4, 5)\} \\ &= \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)\} \\ &= U. \end{aligned}$$

Because the union of $\{S_2, S_3, S_4\}$ equals the universe U , $\mathcal{C}$ is a **valid set cover of size 3**.

Q.3 (a) Knuth-Morris-Pratt (KMP) Algorithm Design & Analysis [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.3)

1. Design & Core Mechanics

The **Knuth-Morris-Pratt (KMP)** algorithm searches for occurrences of a pattern $P[0 \dots m-1]$ within a text $T[0 \dots n-1]$ in linear deterministic time $O(n+m)$.

- **The Inefficiency of Brute Force:** When a mismatch occurs at $T[i] \neq P[j]$, the brute-force algorithm resets the text pointer to $i - j + 1$ and the pattern pointer to 0, discarding all knowledge about the characters already inspected.
- **The KMP Insight:** If a mismatch occurs at index j of the pattern, the prefix $P[0 \dots j-1]$ has already successfully matched the text segment $T[i-j \dots i-1]$. KMP precomputes an auxiliary table—the **failure function** (or Longest Prefix Suffix / LPS table)—derived solely from the self-symmetry of P . This table determines the maximum valid shift of P that avoids re-examining previously matched text characters.

2. Formal Definition of the Failure Function $f(j)$

For a pattern $P = P[0 \dots m-1]$, the failure function $f(j)$ for $0 \leq j \leq m-1$ is formally defined as:

$$f(j) = \text{the length of the longest proper prefix of } P[0 \dots j] \text{ that is also a suffix of } P[0 \dots j].$$

Mathematically:

$$f(j) = \max(\{0\} \cup \{k \mid 0 < k \leq j \text{ and } P[0 \dots k-1] = P[j-k+1 \dots j]\})$$

(Note: A prefix of $P[0 \dots j]$ is "proper" if its length $k < j + 1$, ensuring $f(0) = 0$ since a single character has no non-empty proper prefix).

Operational Meaning during Matching:

When matching fails at $T[i] \neq P[j]$ (with $j > 0$), the preceding segment $P[0 \dots j-1]$ matched $T[i-j \dots i-1]$. The next possible candidate prefix of P that can align with the suffix of the matched text has length:

$$j_{\text{next}} = f(j-1).$$

3. Formal Proof: Why the Text Pointer i Never Retreats

```
KMP Matching Invariant:
In each iteration of the matching loop:
- If P[j] == T[i]:  i <- i + 1,  j <- j + 1
- If P[j] != T[i]:
    If j > 0:      j <- f[j - 1],  i remains unchanged
    If j == 0:    i <- i + 1
```

Proof of Monotonicity ($\Delta i \geq 0$):

- Direct Code Inspection:** Across all execution branches of the matching phase, i is either incremented by 1 (upon a match or when mismatch occurs at $j = 0$) or kept constant (when falling back via $f[j-1]$). There is **no operation** in the algorithm that decrements i . Thus:

$$i_{t+1} \geq i_t \quad \forall t \geq 0.$$

Correctness of Not Backtracking i (Why It Is Safe):

- Suppose a mismatch occurs at $T[i] \neq P[j]$. This guarantees:

$$T[i-j \dots i-1] = P[0 \dots j-1].$$

- In brute force, one would slide the pattern by a shift $s \in \{1, \dots, j-1\}$, aligning $P[0 \dots j-1-s]$ with $T[i-j+s \dots i-1]$.
- For such a shift s to be valid (i.e., not immediately produce a mismatch prior to $T[i]$), the prefix of the shifted pattern $P[0 \dots j-1-s]$ must match the text slice $T[i-j+s \dots i-1]$.
- But from (1), $T[i-j+s \dots i-1]$ is identical to the suffix of P : $P[s \dots j-1]$.
- Therefore, a valid shift requires that $P[0 \dots j-1-s] = P[s \dots j-1]$, meaning the prefix of P must be a **proper prefix of $P[0 \dots j-1]$ that is also a suffix of $P[0 \dots j-1]$** .
- By definition, the longest such prefix has length $k^* = f(j-1)$, corresponding to the minimal non-trivial shift $s^* = j - f(j-1)$.
- Any shift smaller than s^* cannot match because no longer common prefix-suffix exists. Hence, all smaller shifts can be safely skipped without performing any comparisons.
- By setting $j \leftarrow f(j-1)$ and holding i stationary, the algorithm aligns the longest valid prefix with the text up to $i-1$ without re-comparing $T[i-j \dots i-1]$. The next necessary comparison is directly at $T[i]$ against $P[f(j-1)]$. ■

Q.3 (b) Failure Function Construction and Execution Trace [8 Marks]

1. Construction of Failure Function Table for $P = \text{"ABABC"}$

Pattern P of length $m = 5$:

- $P[0] = \text{'A'}$
- $P[1] = \text{'B'}$
- $P[2] = \text{'A'}$

- $P[3] = \text{'B'}$
- $P[4] = \text{'C'}$

j	Substring $P[0 \dots j]$	Proper Prefixes	Proper Suffixes	Longest Common Prefix-Suffix	$f(j)$
0	"A"	$\emptyset$	$\emptyset$	None	0
1	"AB"	{ "A" }	{ "B" }	None	0
2	"ABA"	{ "A", "AB" }	{ "A", "BA" }	"A"	1
3	"ABAB"	{ "A", "AB", "ABA" }	{ "B", "AB", "BAB" }	"AB"	2
4	"ABABC"	{ "A", "AB", "ABA", "ABAB" }	{ "C", "BC", "ABC", "BABC" }	None	0

$$\mathbf{f} = [0, 0, 1, 2, 0]$$

2. Trace of `KMPMatch` on $T = \text{"ABABDABABCABAB"}$ and $P = \text{"ABABC"}$

- Text length: $n = 14$
- Pattern length: $m = 5$

Text Indices:														
Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Char:	A	B	A	B	D	A	B	A	B	C	A	B	A	B

Detailed Comparison Table:

Step	i	$T[i]$	j	$P[j]$	Comparison Result	Action / Pointer Updates
1	0	'A'	0	'A'	Match	$i \leftarrow 1, j \leftarrow 1$
2	1	'B'	1	'B'	Match	$i \leftarrow 2, j \leftarrow 2$
3	2	'A'	2	'A'	Match	$i \leftarrow 3, j \leftarrow 3$
4	3	'B'	3	'B'	Match	$i \leftarrow 4, j \leftarrow 4$
5	4	'D'	4	'C'	Mismatch	$j > 0 \implies$ Fallback: $j \leftarrow f[4 - 1] = f[3] = \mathbf{2}$; (i stays 4)
6	4	'D'	2	'A'	Mismatch	$j > 0 \implies$ Fallback: $j \leftarrow f[2 - 1] = f[1] = \mathbf{0}$; (i stays 4)
7	4	'D'	0	'A'	Mismatch	$j = 0 \implies$ Advance text: $i \leftarrow 4 + 1 = \mathbf{5}, j = 0$
8	5	'A'	0	'A'	Match	$i \leftarrow 6, j \leftarrow 1$
9	6	'B'	1	'B'	Match	$i \leftarrow 7, j \leftarrow 2$
10	7	'A'	2	'A'	Match	$i \leftarrow 8, j \leftarrow 3$
11	8	'B'	3	'B'	Match	$i \leftarrow 9, j \leftarrow 4$
12	9	'C'	4	'C'	Match	Full match detected ($j = m - 1 = 4$)

Final Outcome:

- **Termination Condition:** $j = m - 1 = 4$.
- **Match Start Index Calculation:**

$$\text{Start Index} = i - m + 1 = 9 - 5 + 1 = \mathbf{5}.$$

- **Final Return Value:** `KMPMatch` terminates and returns **index 5**.

Q.4 (a) Aggregate Analysis of Stack with MULTIPOP [7 Marks]

1. Operation Definitions and Naive Analysis

Consider a stack S supporting three operations:

1. $\text{PUSH}(S, x)$: Pushes element x onto stack S .

$$\text{Actual cost } c_{\text{PUSH}} = O(1) \quad (1 \text{ unit}).$$

2. $\text{POP}(S)$: Pops the top element from stack S .

$$\text{Actual cost } c_{\text{POP}} = O(1) \quad (1 \text{ unit}).$$

3. $\text{MULTIPOP}(S, k)$: Removes the top $\min(|S|, k)$ elements from S .

```
MULTIPOP(S, k):  
    while not STACK-EMPTY(S) and k > 0:  
        POP(S)  
        k = k - 1
```

$$\text{Actual cost } c_{\text{MULTIPOP}} = \min(|S|, k).$$

- **Why Naive Analysis Fails:** In the worst case, a single MULTIPOP can pop up to $|S| = O(n)$ elements, taking $O(n)$ time. Multiplying this worst-case single-operation cost by n operations yields an upper bound of $O(n \times n) = O(n^2)$. This bound is overly pessimistic because a stack cannot contain $O(n)$ elements without first executing $O(n)$ PUSH operations.

2. Aggregate Analysis Over a Sequence of n Operations

Aggregate analysis considers the **entire sequence of n operations** as a collective whole from an initially empty stack $S_0 = \emptyset$.

1. The Push-Pop Duality Invariant:

- An element can be removed from the stack (via either a standalone POP or an internal pop inside MULTIPOP) **only if it was previously pushed onto the stack**.
- Each PUSH adds exactly **one** element to the stack.
- Once an element is popped from the stack, it cannot be popped again without an explicit subsequent PUSH .

2. Bounding the Total Number of Pops:

- In any sequence of n operations, let the number of PUSH operations be n_{push} .
- Since the total number of operations in the sequence is n :

$$n_{\text{push}} \leq n.$$

- The maximum number of elements ever inserted into the stack across the entire sequence is at most $n_{\text{push}} \leq n$.
- Consequently, the total number of elements that can ever be popped across all POP and all MULTIPOP operations combined is bounded by:

$$\text{Total Pops} \leq n_{\text{push}} \leq n.$$

3. Total Sequence Cost $T(n)$:

- Total cost of all PUSH operations: at most $n \times 1 = n$ units.
- Total cost of all POP operations (including all pops within all MULTIPOP calls): at most $n \times 1 = n$ units.

$$\text{Total Actual Cost } T(n) = \sum_{i=1}^n c_i \leq n_{\text{push}} + \text{Total Pops} \leq n + n = 2n = O(n).$$

4. Amortized Cost per Operation:

By the definition of aggregate analysis, the amortized cost per operation is the average cost over the worst-case sequence:

$$\text{Amortized Cost} = \frac{T(n)}{n} \leq \frac{2n}{n} = 2 = \mathbf{O(1)}.$$

Thus, any sequence of n stack operations starting from an empty stack runs in $O(n)$ worst-case time, yielding an amortized cost of $O(1)$ **per operation**.

Q.4 (b) Accounting Analysis of a Queue Implemented Using Two Stacks [8 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Queue Architecture and Operations

A FIFO queue Q is implemented using two internal stacks:

- Stack1: Accepts newly enqueued elements.
- Stack2: Delivers dequeued elements in FIFO order.

```
ENQUEUE(Q, x):  
    PUSH(Stack1, x)  
  
DEQUEUE(Q):  
    if Stack2.empty():  
        while not Stack1.empty():  
            y = POP(Stack1)  
            PUSH(Stack2, y)  
    if Stack2.empty():  
        error "Underflow"  
    return POP(Stack2)
```

Cost Model:

Let pushing an element onto any stack cost 1 unit, and popping an element from any stack cost 1 unit.

2. Lifecycle of an Element and Amortized Cost Assignment

Every element enqueued into Q undergoes up to **four** distinct stack operations during its complete queue lifecycle:

1. Pushed onto Stack1 (actual cost = 1)
2. Popped from Stack1 during transfer (actual cost = 1)
3. Pushed onto Stack2 during transfer (actual cost = 1)
4. Popped from Stack2 upon final dequeue (actual cost = 1)

$$\text{Total Lifetime Actual Cost per Element} = 1 + 1 + 1 + 1 = 4 \text{ units.}$$

We assign the following amortized costs:

- **For ENQUEUE(x):** Assign an amortized cost of:

$$\hat{c}_{\text{ENQUEUE}} = \mathbf{3 \text{ units}} \quad (O(1)).$$

- 1 unit pays for the immediate actual operation (PUSH onto Stack1).
- 1 unit is saved as credit with x to pay for its future POP from Stack1.
- 1 unit is saved as credit with x to pay for its future PUSH onto Stack2.

(Net credit deposited per enqueued element = $3 - 1 = 2$ units).

- **For DEQUEUE():** Assign an amortized cost of:

$$\hat{c}_{\text{DEQUEUE}} = 1 \text{ unit } (O(1)).$$

- This 1 unit directly pays for the final actual operation (POP from Stack2).

(Both operations have an assigned amortized cost of $O(1)$).

3. Proof: Stored Credit Never Becomes Negative

State Credit Invariant:

Invariant: Every element currently residing in Stack1 carries exactly 2 units of prepaid credit.

Let the total credit in the data structure after operation t be C_t .

By the invariant:

$$C_t = 2 \cdot |\text{Stack1}_t|.$$

We prove that $C_t \geq 0$ for all $t \geq 0$ by induction on the operation sequence length t :

- **Base Case ($t = 0$):**

Initially, both stacks are empty: $|\text{Stack1}_0| = 0$.

$$C_0 = 2 \cdot 0 = 0 \geq 0.$$

The invariant holds.

- **Inductive Step ($t \rightarrow t + 1$):** Assume $C_t = 2 \cdot |\text{Stack1}_t| \geq 0$. Consider the $(t + 1)$ -th operation:

1. Case 1: Operation is ENQUEUE(x)

- Actual cost: $c_{t+1} = 1$ (PUSH onto Stack1).
- Amortized cost: $\hat{c}_{t+1} = 3$.
- Credit change: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 3 - 1 = +2$.
- New credit:

$$C_{t+1} = C_t + 2 = 2|\text{Stack1}_t| + 2 = 2(|\text{Stack1}_t| + 1) = 2|\text{Stack1}_{t+1}| \geq 0.$$

The invariant is preserved.

2. Case 2: Operation is DEQUEUE() and Stack2 is non-empty

- Actual cost: $c_{t+1} = 1$ (POP from Stack2).
- Amortized cost: $\hat{c}_{t+1} = 1$.
- Credit change: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 1 - 1 = 0$.
- Since Stack1 was untouched: $|\text{Stack1}_{t+1}| = |\text{Stack1}_t|$.

$$C_{t+1} = C_t = 2|\text{Stack1}_{t+1}| \geq 0.$$

The invariant is preserved.

3. Case 3: Operation is DEQUEUE() and Stack2 is empty

- Let $k = |\text{Stack1}_t|$ be the number of elements in Stack1 ($k \geq 1$ for a valid dequeue).
- The transfer requires:
 - k pops from Stack1 (cost = k)
 - k pushes onto Stack2 (cost = k)
 - 1 final pop from Stack2 (cost = 1)
- Total actual cost:

$$c_{t+1} = k + k + 1 = 2k + 1.$$

- Amortized cost charged: $\hat{c}_{t+1} = 1$.
- Credit withdrawn from the bank:

$$\text{Deficit} = c_{t+1} - \hat{c}_{t+1} = (2k + 1) - 1 = 2k.$$

- By the inductive hypothesis, the k elements in Stack1 possessed exactly $2k$ units of credit ($C_t = 2k$).
- This $2k$ credit pays for the k pops and k pushes completely:

$$C_{t+1} = C_t - 2k = 2k - 2k = 0.$$

- Since Stack1 is now empty ($|\text{Stack1}_{t+1}| = 0$):

$$C_{t+1} = 2 \cdot 0 = 2|\text{Stack1}_{t+1}| = 0 \geq 0.$$

The invariant is preserved.

Conclusion:

Because $|\text{Stack1}_t| \geq 0$ at all times, the stored credit satisfies:

$$C_n = \sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \geq 0 \quad \forall n \geq 0.$$

The accumulated credit **never becomes negative**. Consequently, the total amortized cost upper bounds the total actual cost ($\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$), proving that both ENQUEUE and DEQUEUE run in $O(1)$ **amortized time**.

Q.5 (a) Standard Trie Data Structure [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5)

1. Description of the Standard Trie

A **standard trie** (also called a prefix tree) is an ordered tree-based retrieval data structure used to store and search a dynamic set of strings over an alphabet Σ .

- **Root Node:** Represents the empty string ϵ .
- **Edge Representation:** Every edge is labeled with exactly **one character** $c \in \Sigma$. No two edges emanating from the same node may carry the same character.
- **Prefix Sharing:** The path from the root to any node v spells out a prefix of one or more keys in the dictionary. All keys sharing a common prefix share the identical initial ancestral path in the tree.
- **Complexity:** Searching, inserting, or deleting a string of length m takes $O(m \cdot |\Sigma|)$ time (or $O(m)$ using direct array indexing), which is independent of the total number of keys in the trie.

2. Node Structure

In a standard C++ object-oriented implementation, each node contains an array (or map) of pointers to child nodes and a terminal status flag:

```
const int ALPHABET_SIZE = 26; // for lowercase English letters 'a' through 'z'

struct TrieNode {
    TrieNode* children[ALPHABET_SIZE]; // pointers to child nodes
    bool isEndOfWord;                  // true if a valid dictionary word terminates here

    TrieNode() {
        isEndOfWord = false;
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            children[i] = nullptr;
        }
    }
};
```

```
}  
};
```

3. Role of the `isEndOfWord` Boolean Flag

- **Prefix Disambiguation:** In any dictionary, one valid word can be a strict prefix of another (for example, "be" and "bear", or "in" and "inn").
- **Terminal Indication:** The nodes along the path from the root to "bear" (representing "", "b", "be", "bea", "bear") all exist structurally. Without the `isEndOfWord` flag, a search query cannot distinguish whether "be" was explicitly inserted into the dictionary or merely exists as a structural stepping stone to "bear".
- **State Values:**
 - `isEndOfWord == true`: Denotes that the string formed by the path from the root to this node is a complete, recognized word in $\mathcal{W}$.
 - `isEndOfWord == false`: Denotes that the node represents an internal prefix that is not itself a recognized standalone word in $\mathcal{W}$.

4. Pseudocode for `TrieInsert` and `TrieSearch`

```
Algorithm TrieInsert(root, key):  
Input: Root node of the trie, and string key of length m  
Output: Inserts key into the trie  
  
1. curr = root  
2. for each character c in key:  
3.     index = charToIndex(c)           // e.g., c - 'a' for lowercase letters  
4.     if curr.children[index] == NULL:  
5.         curr.children[index] = new TrieNode()  
6.     curr = curr.children[index]  
7. curr.isEndOfWord = True             // Mark the end of the word
```

```
Algorithm TrieSearch(root, key):  
Input: Root node of the trie, and string key of length m  
Output: True if key exists in the trie, False otherwise  
  
1. curr = root  
2. for each character c in key:  
3.     index = charToIndex(c)  
4.     if curr.children[index] == NULL:  
5.         return False                 // Branch does not exist; key not present  
6.     curr = curr.children[index]  
7. return curr.isEndOfWord             // True only if marked as a complete word
```

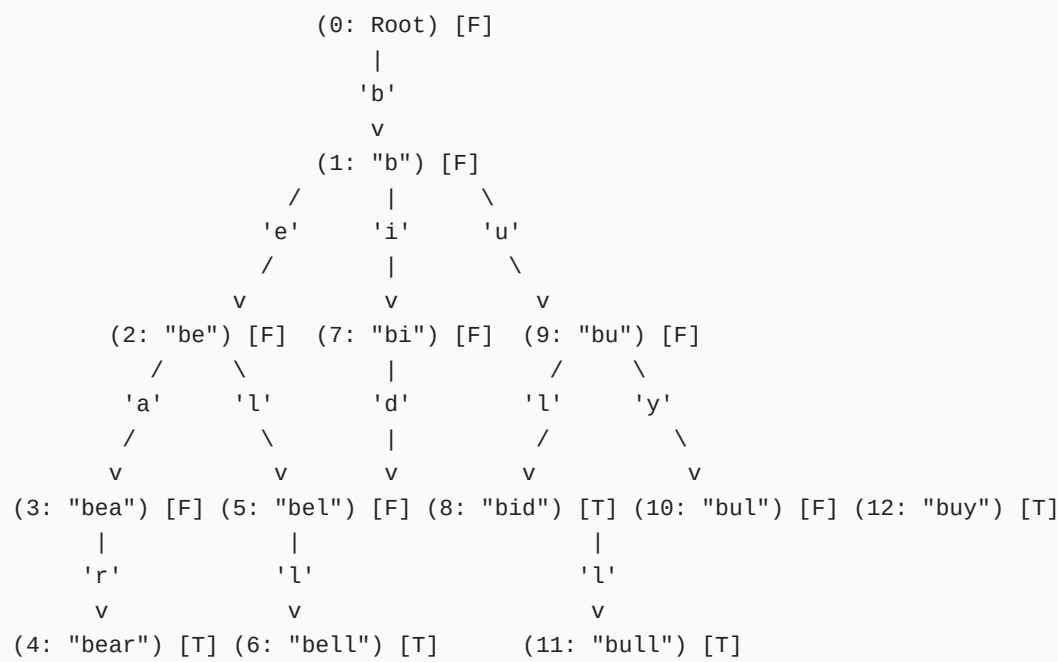
Q.5 (b) Standard Trie vs. Compressed Trie for $\mathcal{W} = \{\text{"bear", "bell", "bid", "bull", "buy"}\}$ [8 Marks]

1. Standard Trie Construction and Diagram

All words share the initial character 'b' :

- "bear" $\rightarrow$ path: $\epsilon \xrightarrow{'b'} b \xrightarrow{'e'} be \xrightarrow{'a'} bea \xrightarrow{'r'} bear$ (Terminal)
- "bell" $\rightarrow$ path: $\dots \xrightarrow{'e'} be \xrightarrow{'l'} bel \xrightarrow{'l'} bell$ (Terminal)

- "bid" → path: ... $\xrightarrow{'b'}$ b $\xrightarrow{'i'}$ bi $\xrightarrow{'d'}$ bid (Terminal)
- "bull" → path: ... $\xrightarrow{'b'}$ b $\xrightarrow{'u'}$ bu $\xrightarrow{'l'}$ bul $\xrightarrow{'l'}$ bull (Terminal)
- "buy" → path: ... $\xrightarrow{'u'}$ bu $\xrightarrow{'y'}$ buy (Terminal)



Legend: [F] = isEndOfWord: False, [T] = isEndOfWord: True

Node Inventory of Standard Trie:

1. (0) Root: "" [F]
2. (1) "b" [F]
3. (2) "be" [F]
4. (3) "bea" [F]
5. (4) "bear" [T] (Leaf)
6. (5) "bel" [F]
7. (6) "bell" [T] (Leaf)
8. (7) "bi" [F]
9. (8) "bid" [T] (Leaf)
10. (9) "bu" [F]
11. (10) "bul" [F]
12. (11) "bull" [T] (Leaf)
13. (12) "buy" [T] (Leaf)

Total Nodes in Standard Trie = **13**.

2. Conversion to a Compressed Trie

A **compressed trie** eliminates **redundant internal nodes**. An internal node *v* is redundant if:

1. It has **out-degree 1** (exactly one child), AND
2. It does not terminate a valid dictionary word (`isEndOfWord == False`).

Chains of redundant nodes are collapsed into a single edge labeled by the concatenation of their individual characters.

Analysis of Redundant Nodes:

- **Chain to "bear"** : Node (3: "bea") has out-degree 1 and is not an end-of-word. It is compressed:
 "be" $\xrightarrow{'a'}$ "bea" $\xrightarrow{'r'}$ "bear" $\implies$ "be" $\xrightarrow{\text{"ar"}}$ "bear".

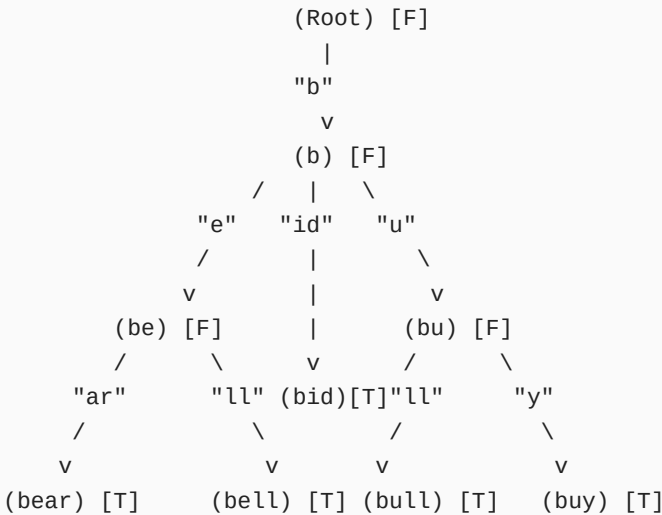
(Node "bea" eliminated).

- **Chain to "bell" :** Node (5: "be") has out-degree 1 and is not an end-of-word. It is compressed:
 $"be" \xrightarrow{'l'} "bel" \xrightarrow{'l'} "bell" \implies "be" \xrightarrow{"ll"} "bell"$.
(Node "bel" eliminated).
- **Chain to "bid" :** Node (7: "bi") has out-degree 1 and is not an end-of-word. It is compressed:
 $"b" \xrightarrow{'i'} "bi" \xrightarrow{'d'} "bid" \implies "b" \xrightarrow{"id"} "bid"$.
(Node "bi" eliminated).
- **Chain to "bull" :** Node (10: "bu") has out-degree 1 and is not an end-of-word. It is compressed:
 $"bu" \xrightarrow{'l'} "bul" \xrightarrow{'l'} "bull" \implies "bu" \xrightarrow{"ll"} "bull"$.
(Node "bul" eliminated).

Branching Nodes Retained:

- Node "b" : Has 3 children ("be" , "bid" , "bu"), so its out-degree is 3. **Retained.**
- Node "be" : Has 2 children ("bear" , "bell"), out-degree is 2. **Retained.**
- Node "bu" : Has 2 children ("bull" , "buy"), out-degree is 2. **Retained.**
- Leaves: "bear" , "bell" , "bid" , "bull" , "buy" . **All 5 retained.**

3. Compressed Trie Diagram



(Note: If the non-branching single edge from the empty root $\epsilon \xrightarrow{"b"} b$ is collapsed so that the root itself is the top branching node, the root directly branches via "be" , "bid" , and "bu").

4. Summary of Node Count Reduction

- **Nodes Eliminated (4 nodes):**

1. Node "bea"
2. Node "be"l"
3. Node "bi"
4. Node "bu"l"

(If the root-to- b link is also contracted, Node "b" is merged into Root as well).

- **Final Node Count:**
- **Standard Trie:** 13 nodes (1 root + 7 internal nodes + 5 leaf nodes)
- **Compressed Trie:** 9 nodes (1 root + 3 internal branching nodes + 5 leaf nodes; or 8 nodes if root is contracted with "b").
- **Absolute Reduction:** 4 nodes eliminated (a 30.77% reduction in total node count).

Q.6 (a) TF-IDF Term Weighting Scheme and Cosine Similarity Ranking [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. Concept of TF-IDF

In web search engines and Information Retrieval (IR), the **TF-IDF** (Term Frequency–Inverse Document Frequency) weighting scheme evaluates how important a word t is to a document d within a broader corpus $\mathcal{D}$. It balances two complementary linguistic intuitions:

1. **Local Salience (TF)**: If a term occurs frequently within a specific document, it is central to that document's topic.
2. **Global Specificity (IDF)**: If a term occurs across nearly every document in the collection (e.g., stop words like "the", "is", "which"), it has low discriminative power and should be penalized.

2. Mathematical Formulations

A. Term Frequency (TF)

Measures the frequency of term t in document d :

- **Standard / Relative Term Frequency**:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

where $f_{t,d}$ is the raw count (number of occurrences) of term t in document d , and the denominator is the total word count of d .

- **Sub-linear / Log-scaled Term Frequency (to dampen high frequencies)**:

$$\text{TF}_{\log}(t, d) = \begin{cases} 1 + \log_{10}(f_{t,d}) & \text{if } f_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

B. Inverse Document Frequency (IDF)

Measures how rare or informative term t is across the entire corpus $\mathcal{D}$ of $N = |\mathcal{D}|$ documents:

$$\text{IDF}(t, \mathcal{D}) = \log_{10} \left(\frac{N}{\text{DF}(t)} \right)$$

where:

- $N = |\mathcal{D}|$ is the total number of documents in the collection.
- $\text{DF}(t) = |\{d \in \mathcal{D} \mid t \in d\}|$ is the **Document Frequency** (the number of documents containing term t).
(If a term appears in all documents, $\text{DF}(t) = N \implies \text{IDF} = \log_{10}(1) = 0$, completely neutralizing common words).

C. Composite Weight:

$$\text{TF-IDF}(t, d, \mathcal{D}) = \text{TF}(t, d) \times \text{IDF}(t, \mathcal{D}).$$

3. Vector Space Model & Cosine Similarity Ranking

In the **Vector Space Model**, the entire lexicon of unique terms across the corpus defines a V -dimensional space, where $V = |\mathcal{V}|$.

- Each document d is mapped to a vector:

$$\vec{d} = \left(w_{t_1,d}, w_{t_2,d}, \dots, w_{t_V,d} \right)^T, \quad \text{where } w_{t,d} = \text{TF-IDF}(t, d)$$

- A multi-term user query q is similarly treated as a short document and mapped to a query vector:

$$\vec{q} = \left(w_{t_1,q}, w_{t_2,q}, \dots, w_{t_v,q} \right)^T$$

Cosine Similarity Formula:

The relevance of document d to query q is quantified by the **cosine of the angle** θ between their respective vectors in this space:

$$\text{CosineSim}(q, d) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \frac{\sum_{t \in q \cap d} w_{t,q} \cdot w_{t,d}}{\sqrt{\sum_{t \in q} w_{t,q}^2} \cdot \sqrt{\sum_{t \in d} w_{t,d}^2}}$$

Ranking Discussion:

- Dot Product (Numerator):** Rewards documents that share heavily weighted terms with the query. If a document shares a rare query term (high IDF), the product $w_{t,q} \cdot w_{t,d}$ increases significantly.
- Length Normalization (Denominator):** Normalizing by the Euclidean norms $\|\vec{q}\|$ and $\|\vec{d}\|$ is crucial:
 - It eliminates **document length bias**, ensuring that long, verbose documents do not dominate the ranking simply because they contain higher raw word counts.
- Score Interpretation:** Because all weights are non-negative ($w \geq 0$), $\text{CosineSim} \in [0, 1]$:
 - $\text{CosineSim} = 1$: Document vector is collinear with the query vector (identical term distribution).
 - $\text{CosineSim} = 0$: Orthogonal vectors (the document shares zero terms with the query).
- Final Ranking:** The search engine computes $\text{CosineSim}(q, d)$ for all candidate documents containing at least one query term and returns the documents sorted in **descending order** of their similarity scores.

Q.6 (b) Inverted Index Construction & Boolean Query Processing [8 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. Document Collection

- Document 1 (D_1):** "data science python"
Tokens & 1-based positions: (data, 1), (science, 2), (python, 3)
- Document 2 (D_2):** "python machine learning"
Tokens & 1-based positions: (python, 1), (machine, 2), (learning, 3)
- Document 3 (D_3):** "data science machine learning python"
Tokens & 1-based positions: (data, 1), (science, 2), (machine, 3), (learning, 4), (python, 5)

Total Documents: $N = 3$.

2. Inverted Index with Full Occurrence (Posting) Lists

An inverted index maps each distinct term to its **document frequency (df)** and a sorted **postings list** containing (Document_ID, Term_Frequency, [Positions]):

Lexicon Term	Document Frequency (df)	Occurrence / Postings List (DocID: TermFreq, [Offsets])
data	2	$(D_1 : 1, [1]) \longrightarrow (D_3 : 1, [1])$
learning	2	$(D_2 : 1, [3]) \longrightarrow (D_3 : 1, [4])$
machine	2	$(D_2 : 1, [2]) \longrightarrow (D_3 : 1, [3])$
python	3	$(D_1 : 1, [3]) \longrightarrow (D_2 : 1, [1]) \longrightarrow (D_3 : 1, [5])$

Lexicon Term	Document Frequency (<i>df</i>)	Occurrence / Postings List (DocID: TermFreq, [Offsets])
science	2	$\left(D_1 : 1, [2]\right) \longrightarrow \left(D_3 : 1, [2]\right)$

3. Execution Trace of the Boolean Query: ("data" AND "python") AND NOT "machine"

Query Evaluation Parse Tree:

$$\text{Result} = \left(\text{Postings}(\text{"data"}) \cap \text{Postings}(\text{"python"})\right) \setminus \text{Postings}(\text{"machine"})$$

Step 1: Postings Retrieval

- $L_{\text{data}} = [D_1, D_3]$
- $L_{\text{python}} = [D_1, D_2, D_3]$
- $L_{\text{machine}} = [D_2, D_3]$

Step 2: Intersection $L_{\text{int}} = L_{\text{data}} \cap L_{\text{python}}$

Using the standard **two-pointer linear merge algorithm** on sorted postings lists:

- Initialize pointers: p_1 on $L_{\text{data}}[0] = D_1$, p_2 on $L_{\text{python}}[0] = D_1$.

Step	Pointer p_1 (L_{data})	Pointer p_2 (L_{python})	Comparison	Action taken	Intermediate Result
1	D_1	D_1	$D_1 == D_1$	Match: Append D_1 ; advance both pointers	$[D_1]$
2	D_3	D_2	$D_3 > D_2$	$D_2 < D_3$: Advance p_2	$[D_1]$
3	D_3	D_3	$D_3 == D_3$	Match: Append D_3 ; advance both pointers	$[D_1, D_3]$

- Both lists/pointers are exhausted.

$$L_{\text{int}} = \text{Postings}(\text{"data"} \wedge \text{"python"}) = [D_1, D_3].$$

Step 3: Difference (AND NOT) $L_{\text{final}} = L_{\text{int}} \setminus L_{\text{machine}}$

Subtract $L_{\text{machine}} = [D_2, D_3]$ from $L_{\text{int}} = [D_1, D_3]$ using the sorted list difference algorithm:

- Initialize pointers: q_1 on $L_{\text{int}}[0] = D_1$, q_2 on $L_{\text{machine}}[0] = D_2$.

Step	Pointer q_1 (L_{int})	Pointer q_2 (L_{machine})	Comparison	Action taken	Final Result List
1	D_1	D_2	$D_1 < D_2$	D_1 does not exist in L_{machine} ; Keep D_1 ; advance q_1	$[D_1]$
2	D_3	D_2	$D_3 > D_2$	Advance q_2	$[D_1]$
3	D_3	D_3	$D_3 == D_3$	Match found in negation list; Discard D_3 ; advance both q_1, q_2	$[D_1]$

- Pointer q_1 reaches the end of L_{int} .

Final Output:

Resulting Document Set = $\{\mathbf{D}_1\}$.

The search engine returns **Document 1** (D_1) as the sole matching document satisfying the Boolean query.

Q.1 (a) Brute Force String Matching Outer Loop Termination Bound [3 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.1)

In the Brute Force string matching algorithm, let the text T have length n (indices $0 \dots n - 1$) and the pattern P have length m (indices $0 \dots m - 1$).

1. Mathematical Range of Alignment

At any outer loop index i , the algorithm attempts to match the pattern characters $P[0 \dots m - 1]$ against the text substring $T[i \dots i + m - 1]$.

- The **last character** inspected in T for a shift starting at index i is located at:

$$\text{Index} = i + m - 1.$$

- For this index to remain within the valid bounds of the text array ($0 \leq \text{Index} \leq n - 1$):

$$i + m - 1 \leq n - 1 \implies \mathbf{i \leq n - m}.$$

2. Why Iterating up to $n - 1$ is Forbidden

- Length Insufficiency:** For any shift $i > n - m$, the remaining suffix of the text $T[i \dots n - 1]$ has length:

$$\text{Length} = n - i < m.$$

A pattern of length m cannot possibly match a text window containing strictly fewer than m characters.

- Memory Safety & Array Bounds:** If the inner loop attempted to check all m characters when $i > n - m$, it would access text indices $i + j \geq n$, triggering an out-of-bounds memory access error or undefined behavior.

Thus, terminating strictly at $i = n - m$ guarantees exactness, prevents redundant execution, and ensures array safety.

Q.1 (b) Credit Invariant Equation in the Accounting Method [3 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Credit Invariant Equation

In the accounting method, each operation i in a sequence of n operations has an actual cost c_i and is charged an amortized cost $\hat{c}_i$. The net credit accumulated in the data structure after n operations is:

$$\text{Credit}_n = \sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i.$$

The **credit invariant** requires that total credit must remain non-negative after every operation:

$$\text{Credit}_n \geq 0 \quad \forall n \geq 0 \quad \iff \quad \sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i \quad \forall n \geq 0.$$

2. Consequence if Credit Drops Below Zero

If at any intermediate step k , $\text{Credit}_k < 0$, then:

$$\sum_{i=1}^k \hat{c}_i < \sum_{i=1}^k c_i.$$

- **Breakdown of Upper-Bound Guarantee:** The amortized cost under-accounts for the actual work performed. If the sequence terminates at step k , the total amortized cost is strictly less than the total actual cost.
- **Proof Invalidation:** It becomes mathematically invalid to claim that the amortized cost per operation ($\frac{1}{k} \sum \hat{c}_i$) serves as an upper bound on the average actual cost per operation, defeating the foundational purpose of amortized analysis.

Q.1 (c) Gate Gadget and CNF Clauses for an OR Gate in Circuit-SAT to 3-SAT [4 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.4)

1. Logic Gate Specification

Consider an OR gate having inputs u, w and output v :

```

      u  -----\
                )--- v
      w  -----/

Logical relation:  v <=> (u OR w)

```

The gate operates correctly if and only if the boolean assignment satisfies the equivalence:

$$v \iff (u \vee w).$$

2. Derivation of CNF Clauses

We decompose the equivalence into two one-way implications:

$$1. v \implies (u \vee w):$$

$$\neg v \vee (u \vee w) \equiv (\mathbf{u} \vee \mathbf{w} \vee \neg \mathbf{v})$$

$$2. (u \vee w) \implies v:$$

$$\neg(u \vee w) \vee v \equiv (\neg u \wedge \neg w) \vee v \equiv (\neg \mathbf{u} \vee \mathbf{v}) \wedge (\neg \mathbf{w} \vee \mathbf{v})$$

Combining both gives the equivalent Conjunctive Normal Form:

$$\Phi_{\text{gate}} = (u \vee w \vee \neg v) \wedge (\neg u \vee v) \wedge (\neg w \vee v).$$

3. Standard 3-Literal Clause Normalization (3-SAT Format)

In 3-SAT, every clause must contain **exactly 3 literals**.

- The first clause $(u \vee w \vee \neg v)$ already has 3 literals.
- To expand the 2-literal clauses $(\neg u \vee v)$ and $(\neg w \vee v)$ into 3-literal clauses, introduce auxiliary variables z_1, z_2 :

$$(\neg u \vee v) \equiv (\neg u \vee v \vee z_1) \wedge (\neg u \vee v \vee \neg z_1)$$

$$(\neg w \vee v) \equiv (\neg w \vee v \vee z_2) \wedge (\neg w \vee v \vee \neg z_2)$$

Final 3-SAT Clause Set:

$$\Phi_{3\text{-SAT}} = (\mathbf{u} \vee \mathbf{w} \vee \neg \mathbf{v}) \wedge (\neg \mathbf{u} \vee \mathbf{v} \vee \mathbf{z}_1) \wedge (\neg \mathbf{u} \vee \mathbf{v} \vee \neg \mathbf{z}_1) \wedge (\neg \mathbf{w} \vee \mathbf{v} \vee \mathbf{z}_2) \wedge (\neg \mathbf{w} \vee \mathbf{v} \vee \neg \mathbf{z}_2).$$

Q.1 (d) Necessity of Appending Unique Terminal Character \$ in Suffix Tries [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5; Ref. [4], Drozdek, Ch. 7)

1. Structural Necessity

In a suffix trie, every suffix of a string S must map to a distinct **leaf node**.

- Without a unique terminator, a shorter suffix of S can be a **proper prefix of a longer suffix**.
- If a suffix is a proper prefix of another suffix, its path terminates at an **internal branching node** rather than a leaf. This introduces structural ambiguity:
- One cannot determine whether an internal node marks a complete suffix or merely an internal prefix of a longer suffix without additional status flags.
- In compressed suffix trees, non-leaf suffixes complicate path-compression invariants (every suffix must terminate at an explicit leaf).

Appending a special end-marker symbol \$ (where $\$ \notin \Sigma$) guarantees that **no suffix can be a proper prefix of any other suffix**, because \$ appears exclusively as the final character.

2. Concrete Illustration on $S = \text{"BANANA"}$

The string $S = \text{"BANANA"}$ has length $n = 6$. Its 6 suffixes are:

- $S_0 = \text{"BANANA"}$
- $S_1 = \text{"ANANA"}$
- $S_2 = \text{"NANA"}$
- $S_3 = \text{"ANA"}$
- $S_4 = \text{"NA"}$
- $S_5 = \text{"A"}$

Without Terminal \$:

- Look at suffixes $S_5 = \text{"A"}$, $S_3 = \text{"ANA"}$, and $S_1 = \text{"ANANA"}$:

"A" is a proper prefix of "ANA" which is a proper prefix of "ANANA".

- In the trie, the path for suffix "A" ends at an internal node on the way to "ANA".
- The path for suffix "ANA" ends at an internal node on the way to "ANANA".
- Look at suffixes $S_4 = \text{"NA"}$ and $S_2 = \text{"NANA"}$:

"NA" is a proper prefix of "NANA".

- Suffix "NA" ends at an internal node on the way to "NANA".
- Result:** Only "BANANA", "ANANA", and "NANA" reach leaves; "A", "ANA", and "NA" are hidden within internal paths.

With Terminal \$ ($S' = \text{"BANANA\$"}$):

The suffixes become:

"BANANA\$", "ANANA\$", "NANA\$", "ANA\$", "NA\$", "A\$", "\$"

- Because \$ appears only at the end, "A\$" is **not** a prefix of "ANA\$", and "NA\$" is **not** a prefix of "NANA\$".
- Every one of the 7 suffixes terminates in a unique, unambiguous **leaf node**.

Q.1 (e) Independent Set Decision Formulation & Polynomial-Time Certifier [5 Marks]

1. Formal Decision Formulation

- **Instance:** An undirected graph $G = (V, E)$ and a positive integer $k \leq |V|$.
- **Question:** Does there exist an independent set $S \subseteq V$ of cardinality $|S| \geq k$?
(A set $S \subseteq V$ is an independent set if no two vertices in S are joined by an edge in E , i.e., $\forall u, v \in S, (u, v) \notin E$.)

2. Deterministic Polynomial-Time Verification Algorithm

- **Certificate:** A candidate vertex subset $S \subseteq V$.

```
Algorithm Verify_Independent_Set(G=(V, E), k, S):
Input: Graph G = (V, E), integer threshold k, candidate certificate S subseteq V
Output: True if S is a valid independent set of size >= k; False otherwise

1. // Step 1: Validate Cardinality
   if |S| < k:
       return False

2. // Step 2: Validate Subset Membership
   if S is not a subset of V:
       return False

3. // Step 3: Fast Membership Lookup Setup
   Initialize a boolean array in_S of size |V| to False
   for each vertex u in S:
       in_S[u] = True

4. // Step 4: Check for Edge Conflicts
   for each edge (u, v) in E:
       if in_S[u] == True and in_S[v] == True:
           return False // Found an edge between two vertices in S; violation!

5. return True // All checks passed
```

Complexity Analysis:

- **Step 1 & 2:** Cardinality and membership check take $O(|S|) \leq O(|V|)$ time.
- **Step 3:** Initializing and populating `in_S` takes $O(|V|)$ time.
- **Step 4:** Iterating through all edges takes $O(|E|)$ time, with each lookup taking $O(1)$ time.
- **Total Time:** $O(|V| + |E|)$, which is strictly linear and therefore **deterministic polynomial time**.

Q.1 (f) Amortized Degradation to $\Theta(k)$ with DECREMENT in a k -bit Counter [5 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Standard Counter Baseline

In an increment-only k -bit counter, flipping $0 \rightarrow 1$ deposits 1 credit to pay for the future $1 \rightarrow 0$ flip, yielding $O(1)$ amortized cost.

2. Adversarial Construction with DECREMENT

Let the counter have k bits $A[0 \dots k - 1]$.

- An adversary initializes the counter or drives it to the critical power-of-two boundary:

Value $2^{k-1} - 1 = 0111 \dots 1_2$ (lowest $k - 1$ bits are 1).

Consider an alternating sequence of n operations:

INCREMENT, DECREMENT, INCREMENT, DECREMENT, ...

State A: 0 1 1 1 ... 1 (Value $2^{k-1} - 1$)

|

| INCREMENT: Bit $0..(k-2)$ flip 1 -> 0; Bit $(k-1)$ flips 0 -> 1 (Cost = k)

v

State B: 1 0 0 0 ... 0 (Value 2^{k-1})

|

| DECREMENT: Bit $(k-1)$ flips 1 -> 0; Bit $0..(k-2)$ flip 0 -> 1 (Cost = k)

v

State A: 0 1 1 1 ... 1 (Value $2^{k-1} - 1$)

3. Exact Cost Accounting per Operation

1. **INCREMENT on State A** ($0111 \dots 1_2 \rightarrow 1000 \dots 0_2$):

- Flips $k - 1$ ones to zeros.
- Flips 1 zero to one.
- **Actual Cost:** $(k - 1) + 1 = k$ bit flips.

2. **DECREMENT on State B** ($1000 \dots 0_2 \rightarrow 0111 \dots 1_2$):

- Flips the leading 1 to zero.
- Flips $k - 1$ zeros to ones.
- **Actual Cost:** $1 + (k - 1) = k$ bit flips.

4. Total Sequence Cost and Amortized Cost

For any sequence of n alternating operations:

Total Actual Cost $T(n) = \sum_{i=1}^n c_i = n \times k = \Theta(nk)$.

The amortized cost per operation becomes:

Amortized Cost $= \frac{T(n)}{n} = \frac{\Theta(nk)}{n} = \Theta(k)$.

Why the Accounting Method Fails:

Each DECREMENT flips $k - 1$ zeros into ones without accumulating any advance credit from prior cheap operations. The adversary repeatedly exploits the cascading carry/borrow chain, eliminating the possibility of an $O(1)$ amortized bound.

Q.1 (g) Document-Level vs. Positional Posting Lists [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. Comparison of Inverted File Formats

Dimension	Document-Level Posting List	Positional Posting List
Data Stored	Records only the document identifier (and optionally raw term frequency):	

term -> [docID_1, docID_2, ...] | Records document identifiers along with exact token offsets/positions within the text:

term -> [(docID_1, tf_1, [pos_1, pos_2]), ...] |

| **Storage Overhead** | **Compact:** Proportional to the number of distinct (term, document) pairs: $O(\sum_d |\mathcal{V}_d|)$. | **Large:**

Proportional to the total number of word occurrences across the entire corpus: $O(\sum_d \text{length}(d))$. |

| **Query Capability** | Can resolve Boolean AND, OR, NOT queries at document granularity. | Can resolve phrase queries, proximity searches, and contextual snippets. |

2. Query Types That Strictly Necessitate Positional Information

1. Exact Phrase Queries:

- *Example:* "machine learning", "data structures and algorithms".
- *Necessity:* A document-level index only confirms that "machine" and "learning" appear somewhere in the same document. A positional index is required to verify that:

$$\text{pos}(\text{"learning"}) = \text{pos}(\text{"machine"}) + 1.$$

2. Proximity Searches (NEAR / Within-k-Words Queries):

- *Example:* cancer NEAR/5 treatment.
- *Necessity:* The user requires that the two words appear within a distance of at most 5 words of each other:

$$|\text{pos}(\text{cancer}) - \text{pos}(\text{treatment})| \leq 5.$$

This constraint cannot be evaluated without intra-document positional offsets.

3. Contextual Snippet Generation & Keyword Highlighting:

- *Necessity:* To display search result summaries with the query terms highlighted in bold within their surrounding sentences, the search engine must locate the exact positions of matching tokens.

Q.2 (a) Polynomial-Time Reduction, Consequences, and the Direction Fallacy [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1)

1. Definition of Polynomial-Time Reduction ($Y \leq_P X$)

Let Y and X be two decision problems.

We say that Y is **polynomial-time reducible to** X (denoted $Y \leq_P X$) if arbitrary instances of problem Y can be solved using a polynomial number of standard computational operations, plus a polynomial number of calls to an **oracle** (subroutine) that solves problem X in a single step ($O(1)$ time).

In the specific case of a **Karp (many-one) reduction**, $Y \leq_P X$ means there exists a deterministic polynomial-time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for every instance s :

$$s \in Y \iff f(s) \in X.$$

Intuitive Meaning: " $Y \leq_P X$ " formally establishes that X is **at least as hard as** Y (or equivalently, Y is no harder than X).

2. The Two Fundamental Consequences of $Y \leq_P X$

A. Tractability (Algorithm Design / Positive Consequence)

If $Y \leq_P X$ and X can be solved in polynomial time, then Y can be solved in polynomial time.

- **Implication:** $X \in \mathbf{P} \implies Y \in \mathbf{P}$.
- **Usage:** If we already possess an efficient, polynomial-time algorithm for X , we can immediately construct a polynomial-time algorithm for Y by converting any instance of Y into an instance of X and running the algorithm for X .

B. Intractability (Hardness Proofs / Negative Consequence)

If $Y \leq_P X$ and Y cannot be solved in polynomial time, then X cannot be solved in polynomial time.

- **Contrapositive Implication:** $Y \notin P \implies X \notin P$ (and if Y is **NP-hard**, then X is also **NP-hard**).
- **Usage:** Used to prove that a newly encountered problem X is computationally intractable. By reducing a *known* hard problem Y (e.g., 3-SAT) to X , we prove that an efficient algorithm for X would resolve Y , which is believed to be impossible under $P \neq NP$.

3. The Reduction Direction Fallacy: Why $Q \leq_P 3\text{-SAT}$ Does Not Prove Q is NP-Complete

To prove a problem Q is **NP-complete**, one must satisfy two conditions:

1. $Q \in NP$ (membership in NP).
2. Q is **NP-hard** (every problem in NP reduces to Q). To establish hardness, one must reduce a *known* NP-complete problem (such as 3-SAT) to Q :

$3\text{-SAT} \leq_P Q \implies Q \text{ is at least as hard as 3-SAT (NP-hard).}$

The Fallacy of Proving $Q \leq_P 3\text{-SAT}$:

- Proving $Q \leq_P 3\text{-SAT}$ establishes that Q is at most as hard as 3-SAT.
- By the Cook-Levin theorem, 3-SAT is NP-complete. Therefore, **every** problem in the class **NP** polynomial-time reduces to 3-SAT.
- Consequently, even simple, trivially solvable problems in **P** (such as 2-SAT, Shortest Path, Minimum Spanning Tree, or Sorting) satisfy:

$\text{Sorting} \leq_P 3\text{-SAT}, \quad 2\text{-SAT} \leq_P 3\text{-SAT}.$

- Reducing Q to 3-SAT merely proves an **upper bound on the complexity of Q** (i.e., that $Q \in NP$). It provides **zero evidence** of a lower bound on hardness and completely fails to prove that Q is NP-hard or NP-complete.

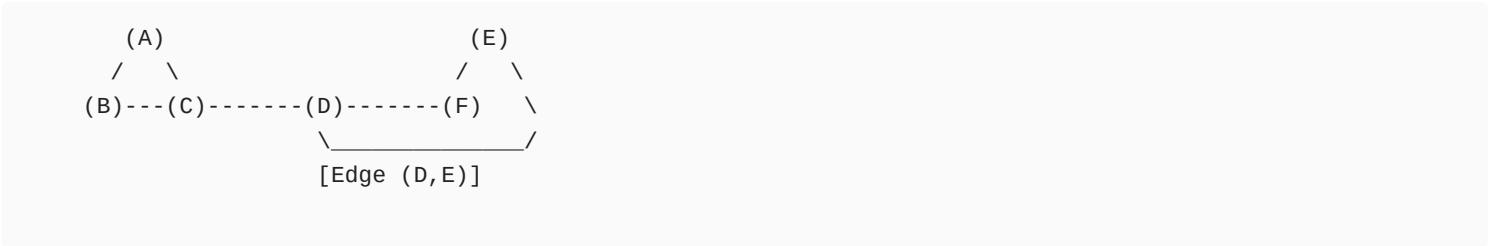
Q.2 (b) Maximum Independent Set and Minimum Vertex Cover on Graph G [8 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.2)

1. Graph Specification

Let $G = (V, E)$ with $|V| = 6$ vertices and $|E| = 7$ edges:

- $V = \{A, B, C, D, E, F\}$
- $E = \{(A, B), (A, C), (B, C), (C, D), (D, E), (D, F), (E, F)\}$



- **Structural Decomposition:** The graph consists of two triangles (3-cliques K_3) joined by a bridge edge (C, D) :
- Clique 1: $\{A, B, C\}$ (edges $(A, B), (A, C), (B, C)$)
- Clique 2: $\{D, E, F\}$ (edges $(D, E), (D, F), (E, F)$)
- Connecting edge: (C, D)

2. Derivation of the Maximum Independent Set (S)

An **independent set** is a subset $S \subseteq V$ containing no adjacent vertices.

- **Clique Upper Bound:**
- In any clique K_p , no two vertices can be chosen together in an independent set. Thus, any independent set can contain **at most 1 vertex** from Clique 1 $\{A, B, C\}$, and **at most 1 vertex** from Clique 2 $\{D, E, F\}$.
- Therefore, the maximum possible size of any independent set in G is:

$$|S| \leq 1 + 1 = 2.$$

- **Candidate Independent Sets of Size 2:**

To select a valid independent set of size 2, pick one vertex from $\{A, B, C\}$ and one from $\{D, E, F\}$ such that the chosen vertices do not share an edge. The only cross-clique edge is (C, D) .

- Pairs including A : $\{A, D\}, \{A, E\}, \{A, F\}$ (All independent)
- Pairs including B : $\{B, D\}, \{B, E\}, \{B, F\}$ (All independent)
- Pairs including C : $\{C, E\}, \{C, F\}$ (Independent, as C is adjacent only to D)

Maximum Independent Set: $S = \{A, E\}$ (or $\{A, D\}, \{B, E\}$, etc.)

$$\alpha(G) = |S| = 2.$$

3. Derivation of the Minimum Vertex Cover via Complement Duality

By the **Complement Duality Theorem** (Ref. [3], Theorem 8.1):

In any graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set $\iff V \setminus S$ is a vertex cover.

Furthermore, S is a **maximum** independent set if and only if $V \setminus S$ is a **minimum** vertex cover:

$$\beta(G) = |V| - \alpha(G) = 6 - 2 = 4.$$

Taking the maximum independent set $S = \{A, E\}$:

$$C = V \setminus S = \{A, B, C, D, E, F\} \setminus \{A, E\} = \{B, C, D, F\}.$$

Verification of Coverage on all 7 Edges for $C = \{B, C, D, F\}$:

1. (A, B) : covered by $B \in C$
2. (A, C) : covered by $C \in C$
3. (B, C) : covered by $B, C \in C$
4. (C, D) : covered by $C, D \in C$
5. (D, E) : covered by $D \in C$
6. (D, F) : covered by $D, F \in C$
7. (E, F) : covered by $F \in C$

Every edge has at least one endpoint in C . Thus, $C = \{B, C, D, F\}$ is a **valid minimum vertex cover of size 4**.

4. Verification of the Cardinality Invariant

$$\begin{aligned} |S| + |V \setminus S| &= 2 + 4 \\ &= 6 \\ &= |V|. \end{aligned}$$

The identity $|S| + |V \setminus S| = |V|$ holds.

Q.3 (a) Internal Mechanics and Insertion Cases of a Compressed Trie [7 Marks]

1. Internal Architecture & Principles

A **compressed trie** (also called a Patricia trie or radix tree) is an space-optimized multi-way search tree where:

- **Edge Labels are Non-Empty Substrings:** Instead of single characters, edges are labeled with variable-length substrings of the alphabet Σ .
- **Deterministic Branching:** No two edges originating from the same node may begin with the identical character.
- **Internal Node Degree Invariant:** Every internal branching node has an out-degree of at least 2 (with the possible exception of the root if all stored keys share an identical prefix).
- **Terminal Status (`isEndOfWord`):** An explicit boolean flag is maintained at each node because a valid word can terminate at an internal branching node (e.g., `"test"` is a prefix of `"tester"`).

2. The Four Insertion Cases

When inserting a string W into a compressed trie starting at current node u :
The algorithm searches for an outgoing edge $e = (u, v)$ whose label L shares a common initial character with W . By deterministic branching, at most **one** such edge can exist.

Let:

- L be the string label of the candidate edge (u, v) .
- W be the remaining suffix of the string being inserted.
- $CP = \text{LCP}(L, W)$ be the **longest common prefix** between L and W , with length $k = |CP|$.

Comparison Spectrum:

|----- L -----|

|----- CP -----|--- remainder L --|

|----- CP -----|--- remainder W --|

|----- W -----|

Case 1: No Edge Match ($k = 0$)

- **Condition:** Node u has no outgoing edge whose label starts with the first character of W .
- **Operation:**

1. Allocate a new leaf node v_{new} with `isEndOfWord` = `True` .
2. Create a directed edge (u, v_{new}) with edge label equal to the entire remaining string W .

Case 2: Edge Label is a Proper Prefix of String ($k = |L| < |W|$)

- **Condition:** The entire edge label L matches the beginning of W , but W has additional characters remaining.
- **Operation:**

1. Consume prefix L from W : update $W \leftarrow W[|L| \dots]$.
2. Recursively (or iteratively) continue the insertion process starting from child node v with the shortened string W .

Case 3: String is a Proper Prefix of Edge Label ($k = |W| < |L|$)

- **Condition:** The string to be inserted terminates strictly inside the edge label L .
- **Operation (Edge-Splitting at Word Boundary):**

1. Remove edge (u, v) .
2. Create an intermediate split node v_{split} marked as terminal: `isEndOfWord = True`.
3. Create an edge (u, v_{split}) with label W .
4. Create an edge (v_{split}, v) with label $L[|W| \dots]$ (the remaining suffix of L), preserving the original child node v and its entire subtree.

Case 4: Mismatch Inside the Edge Label ($0 < k < |L|$ and $k < |W|$)

- **Condition:** The edge label L and string W share a non-empty common prefix CP of length k , but diverge at index k ($L[k] \neq W[k]$). Neither is a prefix of the other.
- **Operation (Full Edge-Splitting & Branching):**

1. Remove original edge (u, v) .
2. Insert an intermediate branching node v_{split} with `isEndOfWord = False` (unless CP was an existing word).
3. Connect u to v_{split} via an edge (u, v_{split}) labeled with the common prefix $CP = L[0 \dots k - 1]$.
4. Reattach the original child v to v_{split} via edge (v_{split}, v) labeled with the remainder of the original edge:

$$\text{Label}_1 = L[k \dots |L| - 1].$$

5. Allocate a new leaf node v_{new} with `isEndOfWord = True`.
6. Attach v_{new} to v_{split} via edge $(v_{\text{split}}, v_{\text{new}})$ labeled with the remainder of the inserted word:

$$\text{Label}_2 = W[k \dots |W| - 1].$$

Before Split:

(u) ----- L -----> (v)

After Split:

```
(u) --- CP ---> (v_split) [F] --- L[k...] ---> (v)
                        \
                        \--- W[k...] ---> (v_new) [T]
```

Q.3 (b) Sequential Insertion Trace for {"test", "tester", "testing", "team", "toast"} [8 Marks]

We trace the step-by-step evolution of an initially empty Compressed Trie:

Step 1: Insert "test"

- **Trigger:** Trie is empty (Case 1).
- **Action:** Create a single edge from Root labeled "test" to a new terminal node.

```
(Root) [F]
  |
  "test"
  v
(1) [T: "test"]
```

Step 2: Insert "tester"

- **Trigger:** Edge label "test" is a proper prefix of "tester" (Case 2: $|CP| = 4 = |L|$).
- **Action:** Traverse down to Node (1) . The remaining suffix is "tester"[4..] = "er" . Node (1) has no edge starting with 'e' (Case 1 at Node (1)). Add edge labeled "er" to a new terminal node (2) .



Step 3: Insert "testing"

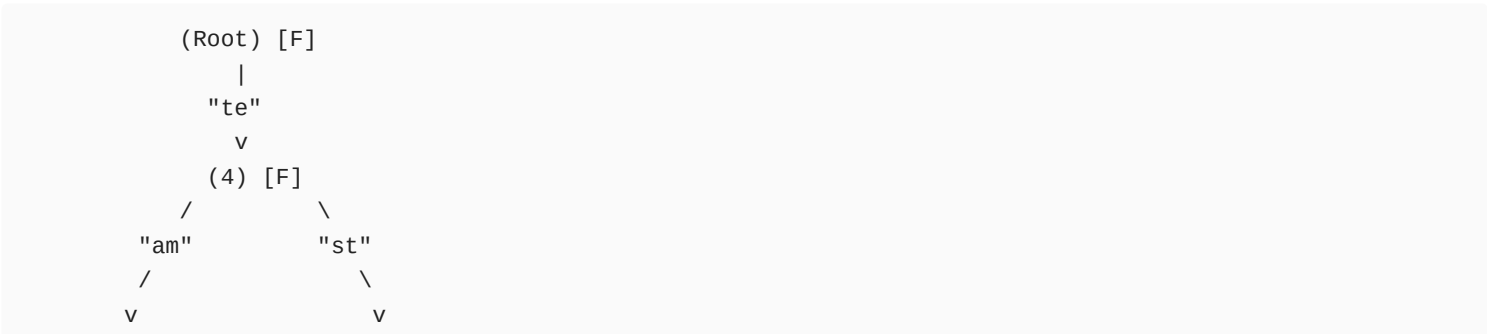
- **Trigger:** Traverse edge "test" to Node (1) . Remaining suffix is "testing"[4..] = "ing" .
- **Action:** At Node (1) , the only existing outgoing edge is "er" (starts with 'e'). No edge starts with 'i' (Case 1 at Node (1)). Add a second outgoing edge labeled "ing" to a new terminal node (3) .

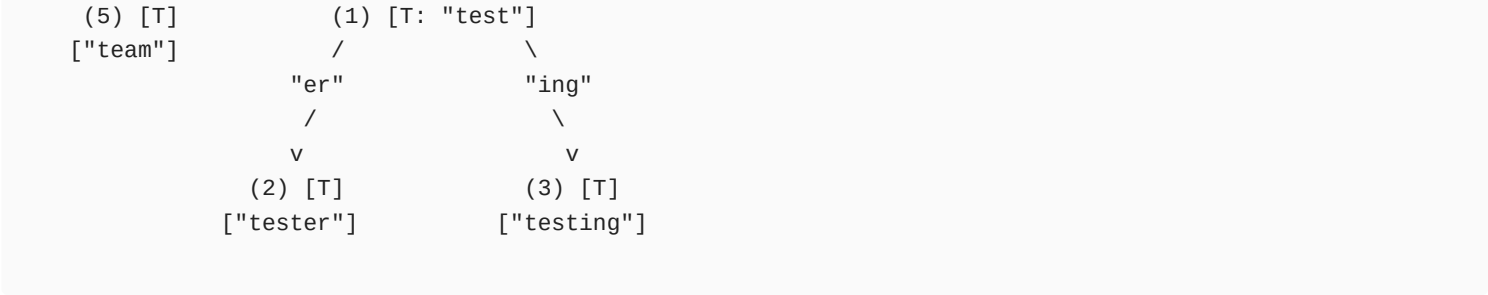


Step 4: Insert "team"

- **Trigger:** Edge from Root is "test" . Comparing $L = \text{"test"}$ with $W = \text{"team"}$:
- Common prefix $CP = \text{"te"}$ ($k = 2$).
- Divergence at index 2: $L[2] = \text{'s'} \neq W[2] = \text{'a'}$.
- **Action: Case 4 (Edge-Splitting):**

1. Split edge "test" at "te" , creating an internal non-terminal node (4) .
2. Edge from Root to (4) is labeled "te" .
3. Reattach Node (1) to (4) with edge label "st" (since "te" + "st" = "test"). Node (1) retains its terminal status and its children (2) and (3) .
4. Create new terminal node (5) for "team" and attach it to (4) with edge label "am" .

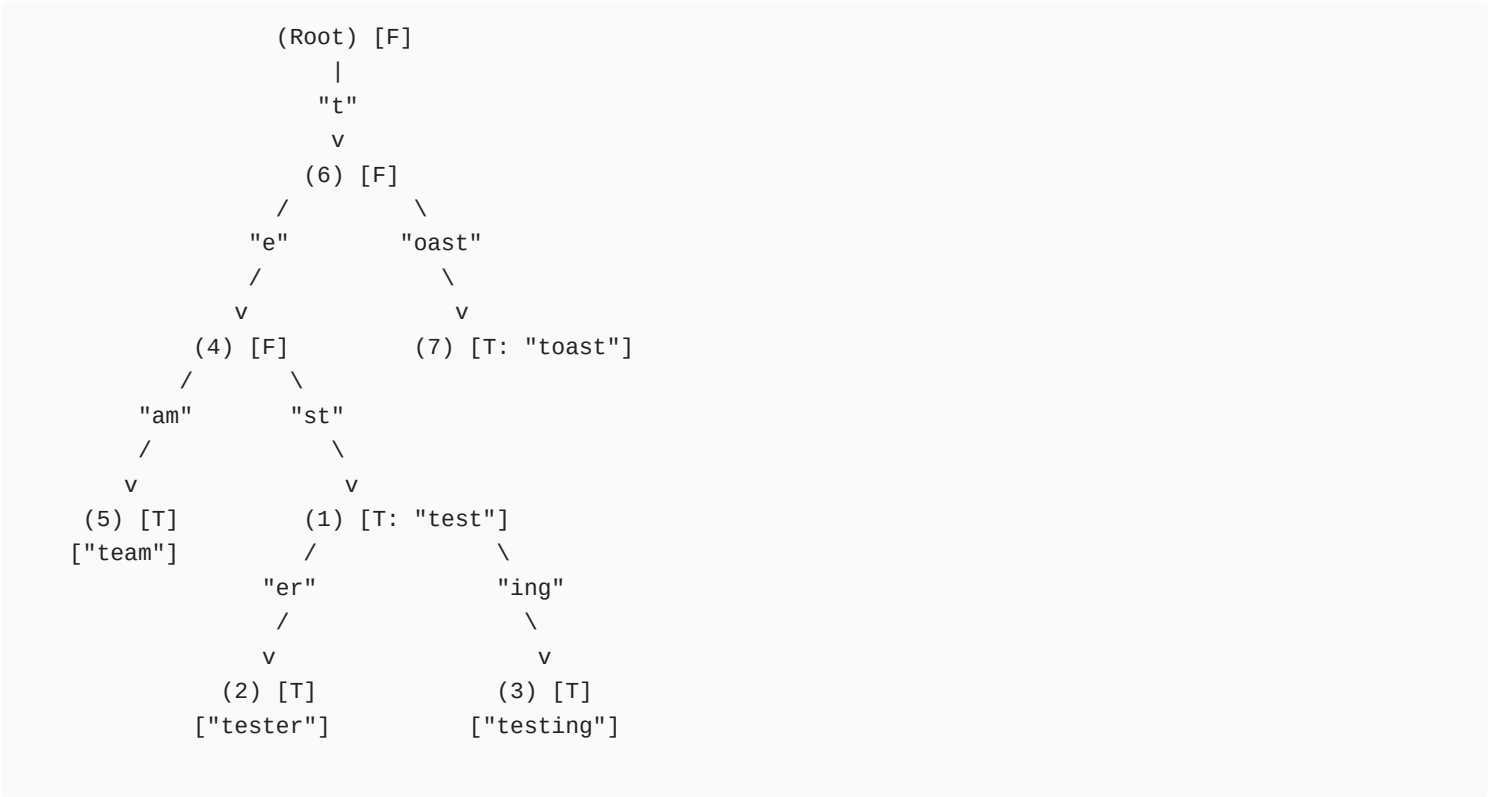




Step 5: Insert "toast"

- **Trigger:** Edge from Root is "te" . Comparing $L = "te"$ with $W = "toast"$:
 - Common prefix $CP = "t"$ ($k = 1$).
 - Divergence at index 1: $L[1] = 'e' \neq W[1] = 'o'$.
 - **Action: Case 4 (Edge-Splitting):**
1. Split edge "te" at "t" , creating an internal non-terminal node (6) .
 2. Edge from Root to (6) is labeled "t" .
 3. Reattach Node (4) to (6) with edge label "e" (since "t" + "e" = "te"). Node (4) retains its entire existing subtree.
 4. Create new terminal node (7) for "toast" and attach it to (6) with edge label "oast" .

Final State of the Compressed Trie



Final Node & Edge Inventory:

- **Root:** Non-terminal [F]
- **Internal Branching Nodes:**
- Node (6) : Prefix "t" , [F] , out-degree 2 (branches via "e" and "oast")
- Node (4) : Prefix "te" , [F] , out-degree 2 (branches via "am" and "st")
- Node (1) : Prefix "test" , [T] (both a terminal word and a branching node), out-degree 2 (branches via "er" and "ing")
- **Terminal Leaf Nodes:**
- Node (5) : Key "team" ([T])

- Node (2) : Key "tester" ([T])
- Node (3) : Key "testing" ([T])
- Node (7) : Key "toast" ([T])
- **Total Nodes:** 8 nodes (1 root + 3 internal + 4 pure leaves). Exactly 5 terminal nodes corresponding to the 5 inserted words.

Q.4 (a) The Cook-Levin Theorem, Circuit-SAT as the Foundation, and Circuit Construction [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.4)

1. Statement of the Cook-Levin Theorem

Theorem (Cook 1971, Levin 1973): Circuit-SAT (and 3-SAT) is NP-complete.

More formally:

1. Circuit-SAT $\in$ NP.
2. For **every** decision problem $X \in$ NP, there exists a polynomial-time reduction to Circuit-SAT:

$$X \leq_P \text{Circuit-SAT}.$$

Hence, Circuit-SAT is NP-hard, and therefore **NP-complete**.

2. Why Circuit-SAT Serves as the Foundational Problem for NP-Completeness

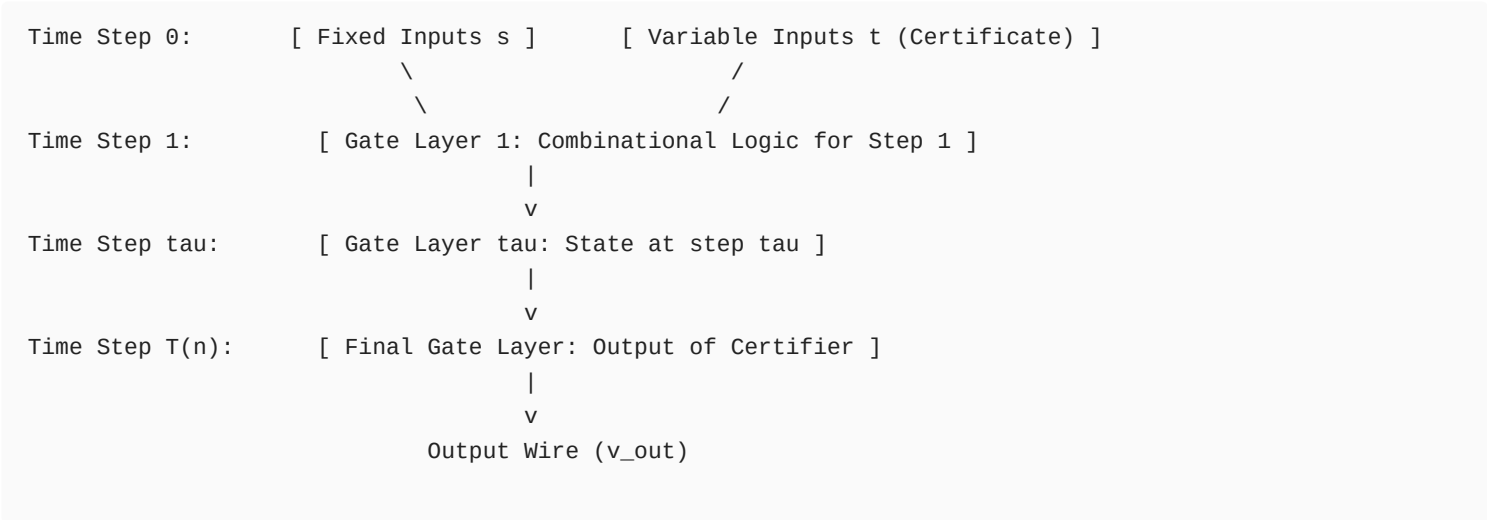
- **Breaking the Circularity of Reductions:** Normally, to prove a problem B is NP-complete, one reduces a *known* NP-complete problem A to B ($A \leq_P B$). For the very first NP-complete problem, however, no existing NP-complete problem was available to reduce from.
- **Direct Encoding of Computation:** One had to prove directly from first principles that *every arbitrary problem* in the infinite class NP reduces to this single target problem.
- **Physical Realization:** Boolean combinational circuits are the mathematical abstraction of digital hardware (ALUs, gates, registers). Because any algorithm running on a physical computer (or Turing machine) is physically realized as a sequence of boolean logic operations, **Circuit-SAT** serves as the universal, natural bridge between the abstract definition of polynomial-time certification and combinatorial logic.

3. Mapping a Polynomial-Time Verification Computation into a Combinational Boolean Circuit

Let X be an arbitrary problem in NP.

By definition, there exists a deterministic certifier algorithm $B(s, t)$ that takes an instance s (length $n = |s|$) and a certificate t (length $|t| \leq p(n)$), running in deterministic time $T(n) \leq q(n)$ for some polynomials p and q .

We unroll the execution of $B(s, t)$ into a layered, directed acyclic graph (DAG) of Boolean gates:



1. Circuit Inputs:

- **Hardwired Constant Inputs:** n inputs are hardwired to fixed constants corresponding to the bits of the known instance $s = s_1 s_2 \dots s_n$.
- **Variable Inputs:** $p(n)$ inputs are left as free, unassigned boolean variables representing the candidate certificate bits $t = t_1 t_2 \dots t_{p(n)}$.

2. Unrolling Time into Space (Gate Layers):

- The state of the computer at any discrete time step $\tau \in \{1, 2, \dots, T(n)\}$ is fully described by a polynomial number of bits (the memory/tape cells, registers, program counter, and status flags).
- The transition of each state bit from step $\tau - 1$ to step τ is determined by fixed, local hardware logic.
- For every time step τ and every state bit j , we instantiate a small constant number of standard gates (AND, OR, NOT) that compute the new bit value from the previous bit values at $\tau - 1$.
- Since there are $T(n)$ time steps and $O(T(n))$ state bits per step, the total number of gates in the unrolled circuit is:

$$\text{Size of Circuit } K = O(T(n)^2) = O(q(n)^2),$$

which is strictly **polynomial** in the input length n .

3. Output Wire:

- The final output bit produced by $B(s, t)$ at time step $T(n)$ (where 1 = "yes" and 0 = "no") is assigned to a designated output wire v_{out} .

4. Equivalence:

- The constructed circuit K has a satisfying assignment to the free input variables t such that $v_{\text{out}} = 1$ **if and only if** there exists a certificate t such that $B(s, t) = \text{"yes"} \iff s \in X$.
- Generating the circuit K from s is deterministic and runs in polynomial time $O(q(n)^2)$. Thus, $X \leq_P \text{Circuit-SAT}$.

Q.4 (b) Auxiliary Variable Padding and Clause Normalization in Circuit-SAT to 3-SAT [8 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.4)

In the reduction from Circuit-SAT to 3-SAT:

1. Every gate v generates CNF clauses of size at most 3.
2. The circuit output requirement generates a 1-literal clause: (v_{out}) .
3. Internal gates (like NOT gates or output-enforcing implications) yield 2-literal or 1-literal clauses.

To strictly conform to **3-SAT**, every clause must contain **exactly 3 literals**.

1. The Auxiliary Variable Padding Technique

Rather than introducing unique dummy variables per clause, the reduction introduces **four global helper variables**:

$$z_1, z_2, z_3, z_4.$$

- The reduction appends a fixed set of clauses over these four variables that forces $z_1 = 0$ (False) and $z_2 = 0$ (False) under **any** satisfying assignment.
- Once z_1 and z_2 are permanently pinned to 0 (False):
- Any 2-literal clause $(l_1 \vee l_2)$ can be padded with z_1 without altering its logical truth value:

$$(l_1 \vee l_2 \vee z_1) \equiv (l_1 \vee l_2 \vee 0) \equiv (l_1 \vee l_2).$$

- Any 1-literal clause (l_1) can be padded with both z_1 and z_2 :

$$(l_1 \vee z_1 \vee z_2) \equiv (l_1 \vee 0 \vee 0) \equiv (l_1).$$

2. The 8 Zero-Enforcement Clauses

To force $z_1 = 0$ and $z_2 = 0$, we construct the complete truth-table combinations over (z_3, z_4) joined with $\overline{z_1}$, and similarly for $\overline{z_2}$.

A. Four Clauses Forcing $z_1 = 0$:

- $(\overline{z_1} \vee z_3 \vee z_4)$
- $(\overline{z_1} \vee z_3 \vee \overline{z_4})$
- $(\overline{z_1} \vee \overline{z_3} \vee z_4)$
- $(\overline{z_1} \vee \overline{z_3} \vee \overline{z_4})$

Why this forces $z_1 = 0$:

If $z_1 = 1$, then $\overline{z_1} = 0$. The four clauses reduce to:

$$(z_3 \vee z_4) \wedge (z_3 \vee \overline{z_4}) \wedge (\overline{z_3} \vee z_4) \wedge (\overline{z_3} \vee \overline{z_4}),$$

which represents all 4 possible truth assignments of (z_3, z_4) and is identically **unsatisfiable**. Thus, any valid truth assignment **must set** $\overline{z_1} = 1 \implies z_1 = 0$.

B. Four Clauses Forcing $z_2 = 0$:

- $(\overline{z_2} \vee z_3 \vee z_4)$
- $(\overline{z_2} \vee z_3 \vee \overline{z_4})$
- $(\overline{z_2} \vee \overline{z_3} \vee z_4)$
- $(\overline{z_2} \vee \overline{z_3} \vee \overline{z_4})$

Why this forces $z_2 = 0$:

By the identical logic, setting $z_2 = 1$ forces the contradictory subformula over (z_3, z_4) . Hence, any satisfying assignment **must set** $\overline{z_2} = 1 \implies z_2 = 0$.

3. Converting $(x_1 \vee \overline{x_2})$ and (x_3) into Valid 3-Literal Clauses

Using the certified constant values $z_1 = 0$ and $z_2 = 0$:

A. Converting the 2-Literal Clause: $(x_1 \vee \overline{x_2})$

- Pad with the single zero-variable z_1 :

$$\mathbf{C}_{\text{converted}} = (\mathbf{x_1} \vee \overline{\mathbf{x_2}} \vee \mathbf{z_1})$$

- Equivalence Verification:**

$$(x_1 \vee \overline{x_2} \vee z_1) \iff (x_1 \vee \overline{x_2} \vee 0) \iff (x_1 \vee \overline{x_2}).$$

B. Converting the 1-Literal Clause: (x_3)

- Pad with both zero-variables z_1 and z_2 :

$$\mathbf{C}'_{\text{converted}} = (\mathbf{x_3} \vee \mathbf{z_1} \vee \mathbf{z_2})$$

- Equivalence Verification:**

$$(x_3 \vee z_1 \vee z_2) \iff (x_3 \vee 0 \vee 0) \iff (x_3).$$

Both resulting clauses contain **exactly 3 literals**, strictly preserving the satisfiability of the original circuit.

Q.5 (a) Comparative Analysis: Brute Force vs. Knuth-Morris-Pratt (KMP) [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.1, 12.3.3)

Let n denote the length of the text T and m denote the length of the pattern P ($m \leq n$).

Metric / Dimension	Brute Force String Matching	Knuth-Morris-Pratt (KMP) Matching
Preprocessing Time	$O(1)$	

Requires no precomputation; matching begins immediately on raw strings. | $O(m)$

Precomputes the failure function (f /LPS table) by analyzing the self-symmetry of the pattern P . |
| **Worst-Case Matching Time** | $O(n \cdot m)$ (specifically $O((n - m + 1)m)$)

Occurs on repetitive texts/patterns (e.g., $T = a^n, P = a^{m-1}b$). | $O(n)$ (Total time: $O(n + m)$)

Linear deterministic matching bound; independent of alphabet size or character repetition. |
| **Auxiliary Space** | $O(1)$

In-place search; only maintains two scalar index pointers (i and j). | $O(m)$

Requires an integer array of size m to store the precomputed failure function $f[0 \dots m - 1]$. |
| **Comparison Reuse & Text Pointer Behavior** | **Zero Reuse (Repeated Work):**

Upon a mismatch at $T[i] \neq P[j]$, the text pointer backtracks to $i - j + 1$, re-comparing characters that were already validated. |
Maximum Reuse (Zero Backtracking):

Text pointer i never retreats. The failure table reuses the known matched prefix $P[0 \dots j - 1]$ to skip invalid shifts, resuming comparisons directly at $T[i]$ with $P[f[j - 1]]$. |

Detailed Discussion:

- 1. **The Core Algorithmic Divergence:** Brute force treats each alignment as completely independent of prior history. In contrast, KMP treats pattern matching as a deterministic finite-state automaton (DFA) where states represent the length of the longest matched prefix.
- 2. **Online Streaming Advantage:** Because KMP never decrements the text pointer i , it can process text as an **unbuffered continuous stream** (from a network socket or sequential file), whereas Brute Force requires random-access buffering or file seeking to backtrack i .

Q.5 (b) Failure Function Construction & KMP Execution Trace [8 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.3)

1. Construction of LPS Table $f[0 \dots 7]$ for $P = \text{”AAACAAAA”}$

Pattern length: $m = 8$.

For each $j \in \{0, \dots, 7\}$, $f[j]$ is the length of the longest proper prefix of $P[0 \dots j]$ that is also a suffix of $P[0 \dots j]$.

j	Substring $P[0 \dots j]$	Proper Prefixes	Proper Suffixes	Longest Common Prefix-Suffix	$f(j)$
0	"A"	$\emptyset$	$\emptyset$	None	0
1	"AA"	{"A"}	{"A"}	"A"	1
2	"AAA"	{"A", "AA"}	{"A", "AA"}	"AA"	2
3	"AAAC"	{"A", "AA", "AAA"}	{"C", "AC", "AAC"}	None	0
4	"AAACA"	{"A", ..., "AAAC"}	{"A", "CA", "ACA", "AACA"}	"A"	1
5	"AAACAA"	{"A", "AA", ...}	{"A", "AA", "CAA", "ACAA", ...}	"AA"	2
6	"AAACAAA"	{"A", "AA", "AAA", ...}	{"A", "AA", "AAA", "CAAA", ...}	"AAA"	3
7	"AAACAAAA"	{"A", "AA", "AAA", ...}	{"A", "AA", "AAA", "AAAA", "CAAAA", ...}	"AAA"	3

$$f = [0, 1, 2, 0, 1, 2, 3, 3]$$

2. KMP Matching Execution Trace on $T = \text{”AAACAAACAAAA”}$

- Text T ($n = 12$):

Index:	0	1	2	3	4	5	6	7	8	9	10	11
Char:	A	A	A	C	A	A	A	C	A	A	A	A

- Pattern P ($m = 8$): "AAACAAAA"
- Failure Table f : [0, 1, 2, 0, 1, 2, 3, 3]

Step-by-Step Comparison Table:

Step	Text Pointer (i)	$T[i]$	Pattern Pointer (j)	$P[j]$	Comparison	Action / Shift Mechanics
1	0	'A'	0	'A'	Match	$i \leftarrow 1, j \leftarrow 1$
2	1	'A'	1	'A'	Match	$i \leftarrow 2, j \leftarrow 2$
3	2	'A'	2	'A'	Match	$i \leftarrow 3, j \leftarrow 3$
4	3	'C'	3	'C'	Match	$i \leftarrow 4, j \leftarrow 4$
5	4	'A'	4	'A'	Match	$i \leftarrow 5, j \leftarrow 5$
6	5	'A'	5	'A'	Match	$i \leftarrow 6, j \leftarrow 6$
7	6	'A'	6	'A'	Match	$i \leftarrow 7, j \leftarrow 7$
8	7	'C'	7	'A'	Mismatch	$j > 0 \implies$ Fallback: $j \leftarrow f[7 - 1] = f[6] = 3$; (i stays 7)
9	7	'C'	3	'C'	Match	Comparison resumes at $T[7]$! $i \leftarrow 8, j \leftarrow 4$
10	8	'A'	4	'A'	Match	$i \leftarrow 9, j \leftarrow 5$
11	9	'A'	5	'A'	Match	$i \leftarrow 10, j \leftarrow 6$
12	10	'A'	6	'A'	Match	$i \leftarrow 11, j \leftarrow 7$
13	11	'A'	7	'A'	Match	Full match detected ($j = m - 1 = 7$)

3. Match Confirmation & Final Output

- **Termination:** Pattern pointer reaches $j = m - 1 = 7$ at Step 13.
- **Match Starting Index in Text:**

Start Index = $i - m + 1 = 11 - 8 + 1 = 4$.

- **Verification:**

$T[4 \dots 11] = \text{”AAACAAAA”} = P$.

- The algorithm terminates and reports a successful match starting at **index 4**.

Q.6 (a) Aggregate Analysis of a *k*-Bit Binary Counter [7 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.1)

1. Counter Mechanics and Naive Bound

Consider a *k*-bit binary counter $A[0 \dots k - 1]$ initialized to 0, where $A[0]$ is the least significant bit (LSB) and $A[k - 1]$ is the most significant bit (MSB).

- An INCREMENT operation adds 1 to the counter:

```
INCREMENT(A, k):
    i = 0
    while i < k and A[i] == 1:
        A[i] = 0
        i = i + 1
    if i < k:
        A[i] = 1
```

- **Naive Worst-Case Bound:** A single INCREMENT can flip up to *k* bits (e.g., from $011 \dots 1_2$ to $100 \dots 0_2$). Multiplying this single-operation worst case by *n* operations yields an upper bound of $O(nk)$. This bound is loose because not every increment flips all *k* bits.

2. Aggregate Analysis: Exact Bit-Flip Frequency Across *n* Increments

Aggregate analysis counts the total number of flips for each bit position $i \in \{0, 1, \dots, k - 1\}$ individually across the entire sequence of *n* operations starting from $000 \dots 0_2$:

Bit Position	Flip Interval	Flips in n Increments
A[0] (LSB)	Flips on every increment	$n = \text{floor}(n / 2^0)$
A[1]	Flips every 2nd increment	$\text{floor}(n / 2) = \text{floor}(n / 2^1)$
A[2]	Flips every 4th increment	$\text{floor}(n / 4) = \text{floor}(n / 2^2)$
...	...	...
A[i]	Flips every 2^i -th increment	$\text{floor}(n / 2^i)$

For any bit position $i \geq 0$, bit $A[i]$ flips if and only if a carry propagates to position *i*, which occurs exactly once every 2^i increments. Therefore, in a sequence of *n* increments:

Total flips of bit $A[i] = \left\lfloor \frac{n}{2^i} \right\rfloor$.

3. Formal Proof of the $2n$ Bound

Summing the bit flips across all k bit positions:

$$T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor$$

Removing the floor function gives a strict inequality:

$$T(n) \leq \sum_{i=0}^{k-1} \frac{n}{2^i} = n \sum_{i=0}^{k-1} \frac{1}{2^i}$$

Evaluating the finite geometric series $\sum_{i=0}^{k-1} \left(\frac{1}{2}\right)^i = \frac{1-(1/2)^k}{1-1/2} = 2 - \frac{1}{2^{k-1}} < 2$:

$$T(n) < n \sum_{i=0}^{\infty} \frac{1}{2^i} = n \cdot \left(\frac{1}{1-1/2} \right) = 2n.$$

$$\mathbf{T(n) < 2n.}$$

4. Amortized Cost

The amortized cost per `INCREMENT` operation is the total actual cost divided by n :

$$\text{Amortized Cost} = \frac{T(n)}{n} < \frac{2n}{n} = 2 = \mathbf{O(1)}.$$

Thus, any sequence of n `INCREMENT` operations executes at most $2n$ bit flips, achieving an amortized cost of $O(1)$ **per operation**.

Q.6 (b) Accounting Analysis of a Stack with Periodic Backups [8 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. System Model & Cost Structure

- **Data Structure:** A stack of maximum capacity k .
- **Base Operations:**
 - PUSH: Actual cost $c = 1$ unit.
 - POP: Actual cost $c = 1$ unit.
- **Periodic Backup Event:**
 - After every k operations (at operation indices $k, 2k, 3k, \dots$), a complete backup is triggered.
 - Backup cost = k units.
- **Total Actual Cost for the m -th operation (c_m):**

$$c_m = \begin{cases} 1 & \text{if } m \not\equiv 0 \pmod{k} \\ 1 + k & \text{if } m \equiv 0 \pmod{k} \end{cases}$$

2. Amortized Cost Assignment via Accounting Method

To absorb the periodic cost of k units occurring once every k operations, each regular operation must pay for its own execution plus a small surcharge to pre-fund the upcoming backup.

We assign the following amortized costs:

$$\hat{c}_{\text{PUSH}} = \mathbf{2 \text{ units}} \quad (O(1))$$

$$\hat{c}_{\text{POP}} = \mathbf{2 \text{ units}} \quad (O(1))$$

Fund Allocation:

- 1 unit pays immediately for the actual execution of the base operation (PUSH or POP).
 - 1 unit is deposited as credit into a dedicated "**Backup Reserve Bank**".
-

3. Proof: Stored Credit is Always Nonnegative

Let m be the number of operations performed so far.

By the Division Algorithm, express m uniquely as:

$$m = q \cdot k + r$$

where:

- $q = \lfloor m/k \rfloor$ is the total number of periodic backups executed so far ($q \geq 0$).
- $r = m \bmod k$ is the number of operations performed since the last backup ($0 \leq r < k$).

A. Total Amortized Cost Charged ($\sum_{i=1}^m \hat{c}_i$):

Because every operation (whether PUSH or POP) is charged exactly 2 units:

$$\sum_{i=1}^m \hat{c}_i = 2m = 2(qk + r) = \mathbf{2qk + 2r}.$$

B. Total Actual Cost Incurred ($\sum_{i=1}^m c_i$):

- Base cost for m operations: $m \times 1 = qk + r$.
- Cost of q backups executed so far: $q \times k = qk$.

$$\sum_{i=1}^m c_i = (qk + r) + qk = \mathbf{2qk + r}.$$

C. Accumulated Credit Balance (Credit_m):

$$\begin{aligned} \text{Credit}_m &= \sum_{i=1}^m \hat{c}_i - \sum_{i=1}^m c_i \\ &= (2qk + 2r) - (2qk + r) \\ &= \mathbf{r}. \end{aligned}$$

Nonnegativity Verification:

Since $r = m \bmod k$, it follows that:

$$\text{Credit}_m = \mathbf{r \geq 0 \quad \forall m \geq 0}.$$

- **Immediately after a backup ($r = 0$):** $\text{Credit} = 0$. The accumulated credit perfectly covered the k -unit backup cost, leaving no deficit.
- **Between backups ($1 \leq r \leq k - 1$):** Credit accumulates steadily: $1, 2, \dots, k - 1$.
- **At the k -th operation ($r = k$ before backup):** Exactly k units of credit are available to pay for the k -unit backup cost.

Because the credit balance is never negative, the amortized cost strictly upper-bounds the actual cost.

4. Total Actual Time Bound for Any Sequence of n Operations

For any arbitrary sequence of n operations:

$$\sum_{i=1}^n c_i = \sum_{i=1}^n \hat{c}_i - \text{Credit}_n \leq \sum_{i=1}^n \hat{c}_i = 2n = \mathbf{O(n)}.$$

The total actual execution time is strictly bounded by $2n$, proving that any sequence of n operations executes in $O(n)$ **actual time** with an amortized cost of $O(1)$ **per operation**.

Q.1 (a) Role of the `isEndOfWord` Flag in a Standard Trie [3 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.1)

1. Why `isEndOfWord` is Necessary

A standard trie shares prefixes among keys. Consequently, every ancestor node of a stored word represents a valid prefix string.

- Without an explicit boolean status flag (such as `isEndOfWord` or `isTerminal`), the trie can only determine whether a given query string exists as a **prefix** along some path.
- It cannot distinguish between:

- A string that was explicitly inserted as a complete word in the dictionary.
- A string that merely exists as an internal prefix of a longer word.

2. Concrete Example Showing Incorrect Membership Queries

Suppose we insert only the word `"cat"` into an empty trie.

- The trie creates a path of nodes:

$$\text{Root} \xrightarrow{'c'} (c) \xrightarrow{'a'} (ca) \xrightarrow{'t'} (cat)$$

Now, consider a membership query for the word `"ca"`:

- Without `isEndOfWord`:**

The search algorithm starts at the root, traverses edge `'c'` to node `(c)`, and edge `'a'` to node `(ca)`. Since the path exists and the characters match, the algorithm returns **True (False Positive)**, falsely asserting that `"ca"` is in the dictionary.

- With `isEndOfWord`:**

Node `(ca)` has `isEndOfWord = False`, while node `(cat)` has `isEndOfWord = True`. The query for `"ca"` inspects the flag at node `(ca)` and correctly returns **False**.

Q.1 (b) Aggregate Analysis of Variable-Cost Operation Sequence [3 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.1)

1. Cost Model

For the i -th operation in a sequence of n operations:

$$c_i = \begin{cases} i & \text{if } i = 2^j \text{ for some integer } j \geq 0 \\ 1 & \text{otherwise} \end{cases}$$

2. Summing Total Actual Cost $T(n)$

Partition the cost into two components:

- Every operation incurs a baseline cost of 1 unit (totaling n units).
- Operations where i is a power of 2 incur an additional surcharge of $(i - 1)$ units.

$$T(n) = \sum_{i=1}^n c_i = \sum_{\substack{1 \leq i \leq n \\ i \neq 2^j}} 1 + \sum_{\substack{1 \leq i \leq n \\ i=2^j}} i \\ \leq n + \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j$$

3. Bounding the Geometric Series

Let $k = \lfloor \log_2 n \rfloor$. The sum of powers of 2 is:

$$\sum_{j=0}^k 2^j = 2^{k+1} - 1 = 2 \cdot 2^{\lfloor \log_2 n \rfloor} - 1 \leq 2n - 1 < 2n.$$

Substituting back:

$$T(n) < n + 2n = \mathbf{3n}.$$

4. Amortized Cost per Operation

$$\text{Amortized Cost} = \frac{T(n)}{n} < \frac{3n}{n} = 3 = \mathbf{O(1)}.$$

The amortized cost per operation is at most **3 units** ($O(1)$).

Q.1 (c) Set Packing Decision Formulation and Reduction from Independent Set [4 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.3)

1. Set Packing Problem (Decision Version)

- **Instance:** A finite universe U , a family of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ where each $S_i \subseteq U$, and a positive integer $k \leq m$.
- **Question:** Does there exist a subcollection of at least k subsets $\mathcal{S}' \subseteq \mathcal{S}$ ($|\mathcal{S}'| \geq k$) that are **mutually disjoint** (i.e., for every distinct pair $S_i, S_j \in \mathcal{S}'$, $S_i \cap S_j = \emptyset$)?

2. Polynomial-Time Reduction Rule: Independent Set $\leq_P$ Set Packing

Given an arbitrary instance of Independent Set $\langle G = (V, E), k \rangle$:

1. **Universe (U):** Set the universe to be the edge set of graph G :

$$\mathbf{U = E}.$$

2. **Subset Family ($\mathcal{S}$):** For each vertex $v \in V$, construct a corresponding subset $S_v \in \mathcal{S}$ containing all edges incident to v :

$$\mathbf{S_v = \{e \in E \mid e \text{ is incident to } v\} \quad \forall v \in V.}$$

3. **Threshold Parameter:**

$$\mathbf{k' = k}.$$

Correctness:

Two subsets $S_u, S_v \in \mathcal{S}$ are disjoint ($S_u \cap S_v = \emptyset$) $\iff$ vertices u and v share no common incident edge $\iff (u, v) \notin E$.

Thus, a collection of k disjoint subsets exists in $\mathcal{S}$ if and only if the corresponding k vertices form an independent set in G . The reduction takes linear time $O(|V| + |E|)$.

Q.1 (d) Proof that $P \subseteq NP$ [5 Marks]

1. Definitions

- **Class P:** A decision problem X is in **P** if there exists a deterministic algorithm A that, given an instance s , decides whether $s \in X$ in time $O(|s|^c)$ for some constant c .
- **Class NP (via Polynomial-Time Certification):** A decision problem X is in **NP** if there exists a polynomial $p(\cdot)$ and a deterministic certifier algorithm $B(s, t)$ running in polynomial time such that:

$$s \in X \iff \exists \text{ certificate } t \text{ with } |t| \leq p(|s|) \text{ such that } B(s, t) = \text{"yes"}.$$

2. Proof Construction

Let X be an arbitrary decision problem in **P**.

Since $X \in \mathbf{P}$, there exists a deterministic polynomial-time algorithm A such that:

$$A(s) = \begin{cases} \text{"yes"} & \text{if } s \in X \\ \text{"no"} & \text{if } s \notin X \end{cases}$$

with running time bounded by $O(|s|^c)$.

We construct a certifier algorithm $B(s, t)$ for X as follows:

1. **Certificate t :** Set the certificate to be the **empty string** $t = \epsilon$ (the certifier completely ignores t).
Length bound: $|t| = 0 \leq p(|s|)$, which trivially satisfies the polynomial length bound.
2. **Certifier Algorithm $B(s, t)$:**

```
Algorithm B(s, t):  
    return A(s)
```

3. Verification of NP Conditions

- **Completeness:** If $s \in X$, then $A(s) = \text{"yes"}$. Choosing $t = \epsilon$ yields $B(s, \epsilon) = \text{"yes"}$.
- **Soundness:** If $s \notin X$, then $A(s) = \text{"no"}$. For every possible certificate t , $B(s, t) = A(s) = \text{"no"}$. No certificate can force an acceptance.
- **Efficiency:** The running time of $B(s, t)$ is identical to the running time of $A(s)$, which is deterministic $O(|s|^c)$ polynomial time.

Because every problem $X \in \mathbf{P}$ satisfies the definition of **NP**, it follows that $\mathbf{P} \subseteq \mathbf{NP}$. ■

Q.1 (e) Analysis of MULTIPUSH(S, k) on Amortized Bounds [5 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.1, 16.2)

1. Problem Statement

Suppose we add MULTIPUSH(S, k)—which pushes k arbitrary elements onto stack S in $\Theta(k)$ actual time—to a stack supporting PUSH, POP, and MULTIPOP. Does the $O(1)$ amortized cost bound **per operation** still hold?

2. Formal Analysis: Degradation of Per-Operation Amortized Cost

The **O(1)** amortized bound per operation does NOT hold.

Proof via Adversarial Sequence:

1. In standard amortized analysis, the amortized cost per operation is:

$$\text{Amortized Cost per Operation} = \frac{\sum_{i=1}^m c_i}{m},$$

where m is the total number of operations in the sequence.

2. Consider an adversary executing an operation sequence of length $m = 2$:

- **Operation 1:** MULTIPUSH(S, n) — Pushes n elements.

$$\text{Actual cost } c_1 = \Theta(n).$$

- **Operation 2:** POP(S) — Pops 1 element.

$$\text{Actual cost } c_2 = 1.$$

3. Total actual cost:

$$T(2) = c_1 + c_2 = \Theta(n) + 1 = \Theta(n).$$

4. Average cost per operation over this valid 2-operation sequence:

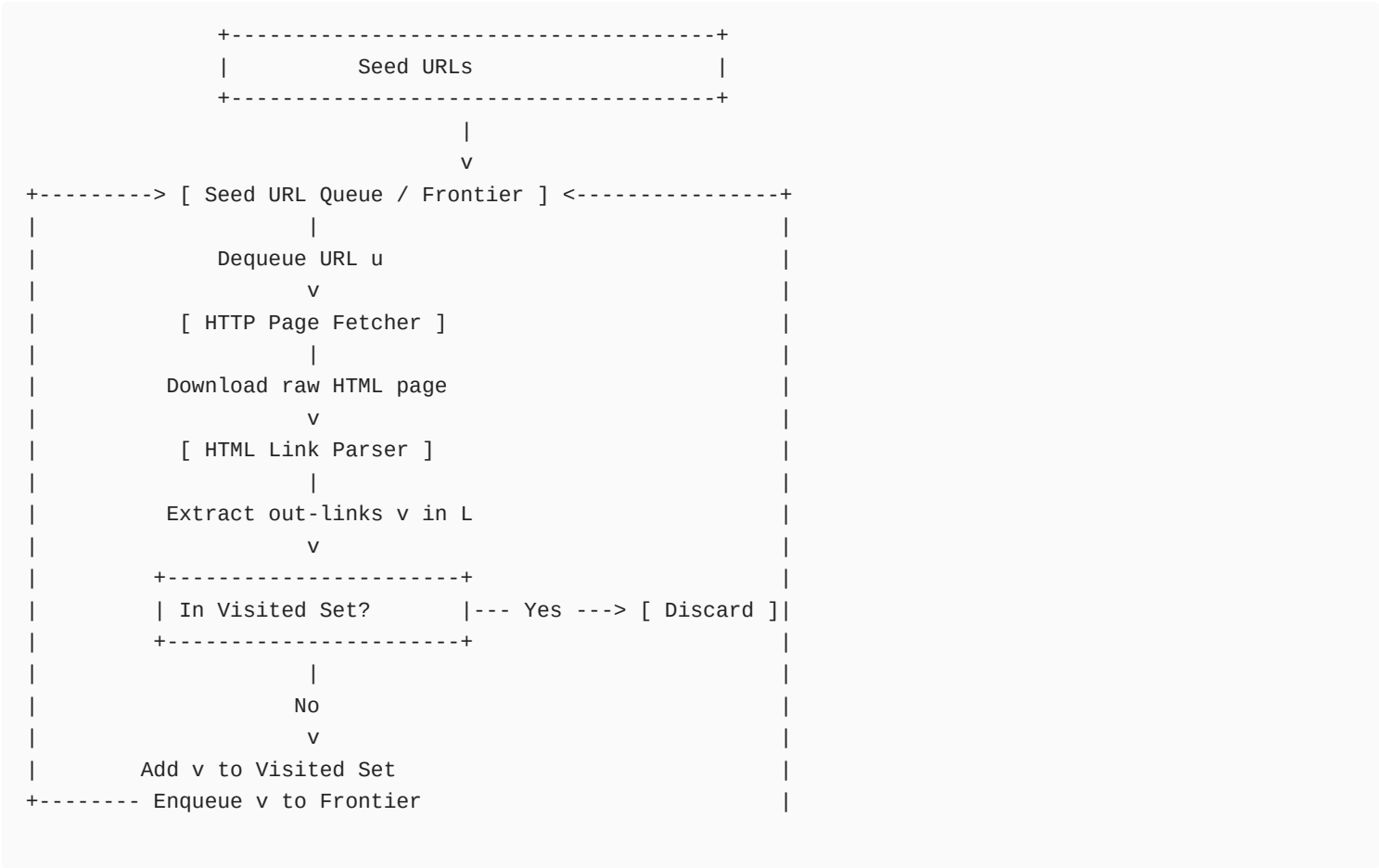
$$\text{Cost per Operation} = \frac{T(2)}{2} = \frac{\Theta(n)}{2} = \Theta(n) \neq O(1).$$

Because k is an unrestricted variable parameter provided at runtime, a single MULTIPUSH operation can perform $\Theta(n)$ work in 1 operation.

(Note: While the amortized cost **per element** remains $O(1)$, the amortized cost **per operation** degrades to $\Theta(k)$ and cannot be bounded by $O(1)$).

Q.1 (f) Algorithmic Workflow of a Web Crawler [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)



Key Component Roles:

1. **Seed URL Queue (URL Frontier):**
 - Acts as the scheduling pipeline (FIFO or Priority Queue) of discovered URLs waiting to be fetched.
 - Enforces **politeness policies** (delay between requests to the same host domain) and prioritization (e.g., crawling high PageRank or frequently updated pages first).
2. **Visited Set (Seen Filter / Deduplication Store):**
 - A high-throughput hash set or Bloom filter storing fingerprints/hashes of all processed URLs.
 - **Prevents infinite crawling loops**, cycles in web topology, and redundant downloading of previously indexed pages.
3. **HTML Link Parser & Normalizer:**
 - Scans raw HTML tokens, identifies hyperlink anchors (`<a href=" . . ">`), and extracts outgoing URLs.
 - **URL Normalization:** Converts relative paths into absolute URLs, strips fragments (`#section`), standardizes port numbers, and unifies lowercase domains.
-

Q.1 (g) Ordinary Trie vs. Suffix Trie [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5)

Comparison Dimension	Ordinary (Standard) Trie	Suffix Trie
Input Representation	Stores a dictionary / collection of k distinct, independent strings $\mathcal{W} = \{S_1, S_2, \dots, S_k\}$.	Stores all n suffixes of a single string S of length n : $\{S[i \dots n - 1] \mid 0 \leq i < n\}$.
Node Count (Worst-Case)	$O(N)$ nodes , where $N = \sum_{i=1}^k S_i $ is the total length of all words in the dictionary.	$\Theta(n^2)$ nodes (e.g., for string with all distinct characters $a_1 a_2 \dots a_n$, total nodes = $\frac{n(n+1)}{2}$).
Construction Complexity	$O(N)$ time by inserting words sequentially character-by-character.	$O(n^2)$ time and space for standard trie (reduced to $O(n)$ time/space in a compressed Suffix Tree via Ukkonen's algorithm).
Primary Query Types	• Exact dictionary word lookup.	

- Dictionary prefix search / auto-complete.
- Lexicographic sorting of dictionary keys. | • Arbitrary **substring search** in text S in $O(m)$ time.
- Longest Repeated Substring.
- Longest Common Substring across multiple texts.
- Suffix and prefix matching within a single document. |

Q.2 (a) Independent Set, Set Packing, and Reduction Proof [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.3)

1. Formal Decision Problem Definitions

- **Independent Set Problem:**
- **Instance:** An undirected graph $G = (V, E)$ and an integer $k \leq |V|$.

- **Question:** Does there exist a subset of vertices $S \subseteq V$ with $|S| \geq k$ such that no two vertices in S are joined by an edge in E (i.e., $\forall u, v \in S, (u, v) \notin E$)?
 - **Set Packing Problem:**
 - **Instance:** A finite universe U , a family of subsets $\mathcal{S} = \{S_1, S_2, \dots, S_m\}$ where each $S_i \subseteq U$, and an integer $k \leq m$.
 - **Question:** Does there exist a subcollection of at least k subsets $\mathcal{S}' \subseteq \mathcal{S}$ ($|\mathcal{S}'| \geq k$) that are **pairwise disjoint** (i.e., $\forall S_i, S_j \in \mathcal{S}'$ with $i \neq j, S_i \cap S_j = \emptyset$)?
-

2. Reduction Construction (Independent Set $\leq_P$ Set Packing)

Given an instance of Independent Set $\langle G = (V, E), k \rangle$:

1. **Universe (U):** Define the universe U as the set of all edges of G :

$$U = E.$$

2. **Family of Subsets ($\mathcal{S}$):** For every vertex $v \in V$, construct a subset $S_v \in \mathcal{S}$ containing all edges incident to v :

$$S_v = \{e \in E \mid v \in e\} \quad \forall v \in V.$$

Thus, $\mathcal{S} = \{S_v \mid v \in V\}$ with $|\mathcal{S}| = |V|$.

3. **Threshold Parameter:** Set the packing quota:

$$k' = k.$$

Computational Complexity: Extracting incident edges for each vertex takes $O(|V| + |E|)$ time, which is strictly deterministic polynomial (linear) time.

3. Correctness Proof: Equivalence of Independent Set and Disjoint Subsets

Claim: $S \subseteq V$ is an independent set in $G \iff \mathcal{S}' = \{S_v \mid v \in S\}$ is a family of pairwise disjoint subsets in $\mathcal{S}$.

Forward Direction ($\implies$):

1. Assume $S \subseteq V$ is an independent set in G with $|S| \geq k$.
2. Consider any two distinct vertices $u, v \in S$.
3. By the definition of an independent set, vertices u and v are **not** adjacent:

$$(u, v) \notin E.$$

4. Now, examine the intersection of their corresponding subsets:

$$S_u \cap S_v = \{e \in E \mid e \text{ is incident to } u\} \cap \{e \in E \mid e \text{ is incident to } v\}.$$

5. An edge e belongs to $S_u \cap S_v$ if and only if e connects both u and v simultaneously. In an undirected graph, the only edge connecting u and v is (u, v) .
6. But from (3), $(u, v) \notin E$. Therefore, no edge can belong to both S_u and S_v :

$$S_u \cap S_v = \emptyset.$$

7. Since this holds for every distinct pair in S , the subcollection $\mathcal{S}' = \{S_v \mid v \in S\}$ consists of $|\mathcal{S}'| = |S| \geq k$ pairwise disjoint subsets. Thus, $(U, \mathcal{S}, k)$ is a YES-instance for Set Packing.

Reverse Direction ($\impliedby$):

1. Assume there exists a subcollection $\mathcal{S}' \subseteq \mathcal{S}$ of $|\mathcal{S}'| \geq k$ pairwise disjoint subsets.
2. Define the corresponding vertex subset:

$$S = \{v \in V \mid S_v \in \mathcal{S}'\} \subseteq V \quad \text{with } |S| = |\mathcal{S}'| \geq k.$$

3. We prove that S is an independent set by contradiction.

Suppose S is **not** an independent set. Then there must exist two distinct vertices $u, v \in S$ such that:

$$e^* = (u, v) \in E.$$

4. By definition of the reduction:

- Because u is an endpoint of e^* , $e^* \in S_u$.
- Because v is an endpoint of e^* , $e^* \in S_v$.

5. Therefore:

$$e^* \in S_u \cap S_v \implies S_u \cap S_v \neq \emptyset.$$

6. This contradicts the assumption that the subsets in $\mathcal{S}'$ are pairwise disjoint.

7. Hence, no two vertices in S can be joined by an edge in E . S is a valid independent set of size $|S| \geq k$. ■

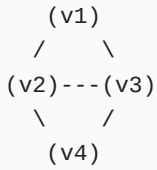
Q.2 (b) Set Packing Construction & Maximal Independent Set on Graph G [8 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.3)

1. Graph Specification

Let $G = (V, E)$ with:

- $V = \{v_1, v_2, v_3, v_4\}$ ($|V| = 4$)
- $E = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_4)\}$ ($|E| = 5$)



(Notice: G is a complete graph K_4 missing only the edge (v_1, v_4)).

2. Construction of Set Packing Instance $(U, \mathcal{S}, k)$ for $k = 2$

1. **Universe** ($U = E$):

$$U = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), (v_2, v_4), (v_3, v_4)\}.$$

(Total elements $|U| = 5$).

2. **Family of Subsets** ($\mathcal{S} = \{S_{v_1}, S_{v_2}, S_{v_3}, S_{v_4}\}$):

- **For vertex v_1 :** Incident edges are (v_1, v_2) and (v_1, v_3) :

$$S_{v_1} = \{(v_1, v_2), (v_1, v_3)\}$$

- **For vertex v_2 :** Incident edges are (v_1, v_2) , (v_2, v_3) , and (v_2, v_4) :

$$S_{v_2} = \{(v_1, v_2), (v_2, v_3), (v_2, v_4)\}$$

- **For vertex v_3 :** Incident edges are (v_1, v_3) , (v_2, v_3) , and (v_3, v_4) :

$$S_{v_3} = \{(v_1, v_3), (v_2, v_3), (v_3, v_4)\}$$

- **For vertex v_4 :** Incident edges are (v_2, v_4) and (v_3, v_4) :

$$S_{v_4} = \{(v_2, v_4), (v_3, v_4)\}$$

3. Target Parameter:

$$k = 2.$$

3. Pairwise Disjointness Analysis across all $\binom{4}{2} = 6$ Subset Pairs

We systematically compute the intersection between every pair of subsets in $\mathcal{S}$:

Subset Pair	Elements in Intersection	Disjoint?	Reason / Edge Conflict in G
(S_{v_1}, S_{v_2})	$S_{v_1} \cap S_{v_2} = \{(v_1, v_2)\}$	No	Vertices v_1 and v_2 share edge (v_1, v_2)
(S_{v_1}, S_{v_3})	$S_{v_1} \cap S_{v_3} = \{(v_1, v_3)\}$	No	Vertices v_1 and v_3 share edge (v_1, v_3)
(S_{v_1}, S_{v_4})	$S_{v_1} \cap S_{v_4} = \emptyset$	YES	No edge exists between v_1 and v_4
(S_{v_2}, S_{v_3})	$S_{v_2} \cap S_{v_3} = \{(v_2, v_3)\}$	No	Vertices v_2 and v_3 share edge (v_2, v_3)
(S_{v_2}, S_{v_4})	$S_{v_2} \cap S_{v_4} = \{(v_2, v_4)\}$	No	Vertices v_2 and v_4 share edge (v_2, v_4)
(S_{v_3}, S_{v_4})	$S_{v_3} \cap S_{v_4} = \{(v_3, v_4)\}$	No	Vertices v_3 and v_4 share edge (v_3, v_4)

4. Mapping to the Maximum / Maximal Independent Set in G

- Disjoint Subcollection in Set Packing:

$$\mathcal{S}^* = \{S_{v_1}, S_{v_4}\} \quad (|\mathcal{S}^*| = 2 = k).$$

Since $S_{v_1} \cap S_{v_4} = \emptyset$, this is the **unique** solution of size 2 to the Set Packing problem on this instance.

- Corresponding Vertex Subset in Graph G :

$$S^* = \{v_1, v_4\}.$$

- Independence & Maximality Verification:

1. **Independence:** Vertices v_1 and v_4 do not share an edge in E ($(v_1, v_4) \notin E$).

2. **Maximality:**

- Adding v_2 creates edges (v_1, v_2) and (v_4, v_2) .

- Adding v_3 creates edges (v_1, v_3) and (v_4, v_3) .

No additional vertex can be added without violating independence.

3. **Maximum Cardinality:** In G , $\{v_1, v_2, v_3\}$ forms a triangle (K_3) and $\{v_2, v_3, v_4\}$ forms a triangle (K_3). At most 1 vertex can be picked from each triangle. Thus, $\alpha(G) \leq 2$.

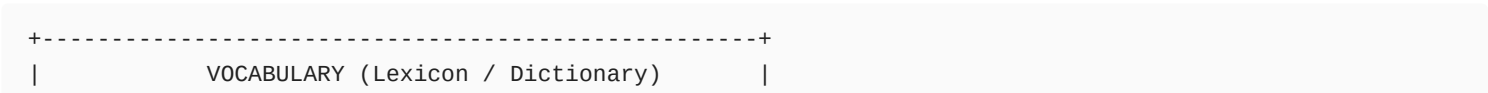
Hence, $S^* = v_1, v_4$ is the **maximum (and maximal) independent set** of graph G , mapped directly from the unique disjoint pair $\{S_{v_1}, S_{v_4}\}$.

Q.3 (a) Inverted File Architecture, Storage Layout, and Boolean Merging [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. Inverted File Architecture and Storage Components

An **Inverted File** (or Inverted Index) is the standard data structure used by search engines for full-text search. It decouples the document corpus into two primary physical storage layers:



Term (t)	Document Frequency (df)	Pointer
algorithm	100	addr_1 --+----> [Postings File / Disk Blocks]
complexity	10	addr_2 --+----> Postings for "complexity"
data	500	addr_3 --+----> Postings for "data"

2. Component Details

A. Vocabulary Search Structure (Dictionary)

The vocabulary stores every distinct word token appearing in the collection. For each term t , it maintains:

- **The Term String** (or an offset into a shared character pool).
- **Document Frequency (df_t)**: The total number of documents containing t (used for query optimization and IDF computation).
- **Posting Pointer**: A 64-bit byte offset or pointer referencing the start of its postings list on disk.

Common Search Structures for the Lexicon:

1. **Hash Table**: Provides expected $O(1)$ lookup time for single terms. *Limitation*: Does not support prefix, range, or wildcard searches.
2. **B+ Tree**: Keeps the vocabulary sorted lexicographically on disk. Lookup takes $O(\log |\mathcal{V}|)$, providing support for range queries and prefix searches.
3. **Compressed Trie**: Stores terms in memory with $O(m)$ lookup (where m is the query term length). Highly space-efficient through common prefix sharing, supporting autocomplete and fast string prefixes.

B. Physical Layout of Occurrence (Postings) Lists

A postings list stores the sequence of documents in which term t appears.

- **Record Structure**: Each entry (posting) contains at minimum:

$$\left(\text{DocID}, \text{TermFrequency}, [\text{Positions}] \right).$$

- **Physical Organization**:
- **Sorted Order**: Postings are stored **strictly in ascending order of DocID** ($d_1 < d_2 < \dots < d_k$). This sorted invariant enables linear two-pointer merge algorithms.
- **Contiguous Allocation**: The postings for a term are stored contiguously in disk blocks to maximize sequential I/O transfer rates.
- **Gap (Delta) Encoding**: Instead of raw 32-bit DocIDs (d_1, d_2, d_3), the system stores the differences: $\Delta_1 = d_1$, $\Delta_2 = d_2 - d_1$, $\Delta_3 = d_3 - d_2$. Small integers are compressed using variable-byte (VByte) or Elias coding.
- **Skip Pointers**: Skip links are placed at intervals of $\approx \sqrt{df}$ postings to allow pointer jumping without scanning every intermediate record.

3. Postings Merging for Boolean Operations

Let L_1 and L_2 be two sorted postings lists of lengths $|L_1|$ and $|L_2|$, with pointers p_1 and p_2 .

A. Boolean AND ($L_1 \cap L_2$): Intersection

- **Logic**: Finds documents that appear in **both** lists.
- **Algorithm**:

```

while p1 != end and p2 != end:
    if L1[p1].docID == L2[p2].docID:
        Append L1[p1].docID to Result
        p1 = p1 + 1;  p2 = p2 + 1
    else if L1[p1].docID < L2[p2].docID:
        p1 = p1 + 1    (or skip forward using skip pointers)
    else:
        p2 = p2 + 1

```

- **Complexity:** $O(|L_1| + |L_2|)$ time (reduced to $O(|L_1| \log |L_2|)$ or $O(\sqrt{|L|})$ using skip lists).

B. Boolean OR ($L_1 \cup L_2$): Union

- **Logic:** Finds documents that appear in **at least one** of the lists.
- **Algorithm:**

```

while p1 != end and p2 != end:
    if L1[p1].docID < L2[p2].docID:
        Append L1[p1].docID;  p1 = p1 + 1
    else if L2[p2].docID < L1[p1].docID:
        Append L2[p2].docID;  p2 = p2 + 1
    else:
        Append L1[p1].docID;  p1 = p1 + 1;  p2 = p2 + 1
Append any remaining elements from non-exhausted list

```

- **Complexity:** $O(|L_1| + |L_2|)$ time.

C. Boolean NOT / Difference ($L_1 \setminus L_2$): Negation

- **Logic:** Evaluates $L_1 \setminus L_2$ (documents in L_1 that do not appear in L_2).
- **Algorithm:**

```

while p1 != end and p2 != end:
    if L1[p1].docID < L2[p2].docID:
        Append L1[p1].docID to Result;  p1 = p1 + 1
    else if L1[p1].docID > L2[p2].docID:
        p2 = p2 + 1
    else:
        p1 = p1 + 1;  p2 = p2 + 1    // Exclude match!
Append remaining elements of L1 to Result

```

- **Complexity:** $O(|L_1| + |L_2|)$ time.

Q.3 (b) Numerical Evaluation: IDF, TF-IDF Vector, and Cosine Similarity [8 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. Given Data

- Total documents: $N = 1000$
- Document Frequencies:
- $df(t_1) = 100$ ("algorithm")
- $df(t_2) = 10$ ("complexity")
- $df(t_3) = 1000$ ("the")
- Formula: $IDF(t) = \log_{10} \left(\frac{N}{df_t} \right)$

2. Computation of IDF Weights

- For t_1 ("algorithm"):

$$\text{IDF}(t_1) = \log_{10} \left(\frac{1000}{100} \right) = \log_{10}(10) = \mathbf{1.0}$$

- For t_2 ("complexity"):

$$\text{IDF}(t_2) = \log_{10} \left(\frac{1000}{10} \right) = \log_{10}(100) = \mathbf{2.0}$$

- For t_3 ("the"):

$$\text{IDF}(t_3) = \log_{10} \left(\frac{1000}{1000} \right) = \log_{10}(1) = \mathbf{0.0}$$

(Note: $\text{IDF}(t_3) = 0$ illustrates how stop words are naturally assigned zero weight).

3. TF-IDF Weight Vector for Document D_A

Given term frequencies in D_A :

- $\text{TF}(t_1, D_A) = 3$
- $\text{TF}(t_2, D_A) = 1$

Calculating component weights $w_{t,D_A} = \text{TF}(t, D_A) \times \text{IDF}(t)$:

- $w_{t_1,D_A} = 3 \times 1.0 = \mathbf{3.0}$
- $w_{t_2,D_A} = 1 \times 2.0 = \mathbf{2.0}$

$$\vec{w}_{D_A} = (\mathbf{w}_{t_1,D_A}, \mathbf{w}_{t_2,D_A}) = (\mathbf{3.0}, \mathbf{2.0})$$

4. Cosine Similarity Between D_A and Query Q

Given Query Vector across (t_1, t_2) :

$$\vec{w}_Q = (1, 2)$$

Step 1: Compute Dot Product ($\vec{w}_{D_A} \cdot \vec{w}_Q$)

$$\vec{w}_{D_A} \cdot \vec{w}_Q = (3 \times 1) + (2 \times 2) = 3 + 4 = \mathbf{7.0}$$

Step 2: Compute Euclidean Norms (Magnitudes)

- Norm of D_A :

$$\|\vec{w}_{D_A}\| = \sqrt{3^2 + 2^2} = \sqrt{9 + 4} = \sqrt{\mathbf{13}} \approx 3.60555$$

- Norm of Q :

$$\|\vec{w}_Q\| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{\mathbf{5}} \approx 2.23607$$

Step 3: Compute Denominator Product

$$\|\vec{w}_{D_A}\| \|\vec{w}_Q\| = \sqrt{13} \times \sqrt{5} = \sqrt{\mathbf{65}} \approx 8.06226$$

Step 4: Compute Cosine Similarity

$$\text{CosineSim}(D_A, Q) = \frac{\vec{w}_{D_A} \cdot \vec{w}_Q}{\|\vec{w}_{D_A}\| \|\vec{w}_Q\|} = \frac{7}{\sqrt{65}}$$

$$\text{CosineSim}(\mathbf{D}_A, \mathbf{Q}) = \frac{7}{\sqrt{65}} \approx \mathbf{0.8682} \quad (\mathbf{86.82\%} \text{ similarity})$$

Q.4 (a) The Accounting Method and Stack Amortized Analysis [7 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Definition of the Accounting Method

In the **accounting method** of amortized analysis:

- We assign artificial charges, termed **amortized costs** ($\hat{c}_i$), to each operation in a data structure.
- When an operation's amortized cost exceeds its actual cost ($\hat{c}_i > c_i$), the difference $\hat{c}_i - c_i$ is treated as **credit** deposited in the data structure, associated either with specific data items or an overall reserve fund.
- When an expensive operation occurs whose actual cost exceeds its amortized cost ($c_j > \hat{c}_j$), the deficit $c_j - \hat{c}_j$ is paid for by withdrawing accumulated credit.
- **The Credit Invariant:** For the amortized cost to be a valid upper bound on the actual cost, the total accumulated credit must remain non-negative after **every** operation in any sequence:

$$\text{Credit}_{\mathbf{n}} = \sum_{i=1}^{\mathbf{n}} \hat{\mathbf{c}}_i - \sum_{i=1}^{\mathbf{n}} \mathbf{c}_i \geq \mathbf{0} \quad \forall \mathbf{n} \geq \mathbf{0} \quad \implies \quad \sum_{i=1}^{\mathbf{n}} \mathbf{c}_i \leq \sum_{i=1}^{\mathbf{n}} \hat{\mathbf{c}}_i.$$

2. Credit Assignment for Stack Operations

Consider an initially empty stack S supporting:

- $\text{PUSH}(S, x)$: Actual cost $c = 1$ unit.
- $\text{POP}(S)$: Actual cost $c = 1$ unit.
- $\text{MULTIPOP}(S, k)$: Actual cost $c = \min(|S|, k)$ units.

We assign the amortized costs:

$$\hat{\mathbf{c}}_{\text{PUSH}} = \mathbf{2} \text{ units}$$

$$\hat{\mathbf{c}}_{\text{POP}} = \mathbf{0} \text{ units}$$

$$\hat{\mathbf{c}}_{\text{MULTIPOP}} = \mathbf{0} \text{ units}$$

3. Formal Proof of the $O(1)$ Amortized Bound

State Credit Invariant:

Invariant: Every element currently residing in the stack carries exactly 1 unit of stored credit.

Let the total credit in the stack after operation t be C_t . By the invariant:

$$C_t = |S_t|.$$

We prove by induction on the sequence length t that $C_t \geq 0$ for all $t \geq 0$:

1. Base Case ($t = 0$):

The stack is initially empty ($|S_0| = 0$).

$$C_0 = |S_0| = 0 \geq 0.$$

The invariant holds.

2. **Inductive Step** ($t \rightarrow t + 1$): Assume $C_t = |S_t| \geq 0$.

- **Case 1: Operation is PUSH(S, x)**
- Actual cost: $c_{t+1} = 1$.
- Amortized cost charged: $\hat{c}_{t+1} = 2$.
- Credit deposited: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 2 - 1 = +1$.
- This 1 unit is placed directly on the newly pushed element x .
- New credit:

$$C_{t+1} = C_t + 1 = |S_t| + 1 = |S_{t+1}| \geq 0.$$

The invariant holds.

- **Case 2: Operation is POP(S)**
- Actual cost: $c_{t+1} = 1$.
- Amortized cost charged: $\hat{c}_{t+1} = 0$.
- Credit consumed: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 0 - 1 = -1$.
- The actual cost of 1 is paid for by consuming the 1 unit of credit residing on the popped element.
- New credit:

$$C_{t+1} = C_t - 1 = |S_t| - 1 = |S_{t+1}| \geq 0.$$

The invariant holds.

- **Case 3: Operation is MULTIPOP(S, k)**
- Let $m = \min(|S_t|, k)$ be the number of elements popped.
- Actual cost: $c_{t+1} = m$.
- Amortized cost charged: $\hat{c}_{t+1} = 0$.
- Credit consumed: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 0 - m = -m$.
- Each of the m popped elements had 1 unit of credit stored on it, totaling m units of credit. This credit pays for the m pop operations.
- New credit:

$$C_{t+1} = C_t - m = |S_t| - m = |S_{t+1}| \geq 0.$$

The invariant holds.

Conclusion:

Because the stack size $|S_n| \geq 0$ at all times, the total accumulated credit satisfies:

$$C_n = \sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i = |S_n| \geq 0 \quad \forall n \geq 0.$$

Hence, the total actual cost is bounded by:

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i \leq 2n = O(n).$$

This proves that the amortized cost per operation is $\leq 2 = O(1)$.

Q.4 (b) Design and Accounting Analysis of Binary Counter with RESET [8 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2, Exercise 16.2-2)

1. The Design Challenge

In a naive k -bit counter, a `RESET` operation clears all k bits by running a loop of size k . If an adversary alternates `INCREMENT` and `RESET`, each pair would cost $\Theta(k)$, resulting in $\Theta(nk)$ total time. To achieve $O(n)$ total time, `RESET` must only touch the bits that have been actively set.

2. Tracking Variable Maintained

We augment the k -bit array $A[0 \dots k-1]$ with an integer tracking pointer:

max_bit

- **Semantics:** `max_bit` stores the **highest index of any bit in A currently containing a 1**.
- **Initialization:** When the counter is $000 \dots 0_2$, `max_bit` = -1 .

Algorithm Implementations:

```
INCREMENT(A, max_bit, k):
    i = 0
    while i < k and A[i] == 1:
        A[i] = 0
        i = i + 1
    if i < k:
        A[i] = 1
        if i > max_bit:
            max_bit = i
    // Actual cost = (number of 1s flipped to 0) + 1

RESET(A, max_bit):
    for i = 0 to max_bit:
        A[i] = 0
    max_bit = -1
    // Actual cost = max_bit + 1
```

- **Crucial Invariant:** To advance `max_bit` to index h , the counter value must reach at least 2^h . Therefore, between two successive `RESET` calls, if the highest bit reached is $h \geq 0$, there must have been **at least** $2^h \geq h + 1$ preceding `INCREMENT` operations.

3. Amortized Cost Assignment via Accounting Method

Let the unit of actual cost be 1 bit inspection/flip.

We assign the following amortized costs:

$$\hat{c}_{\text{INCREMENT}} = \mathbf{3 \text{ units}} \quad (O(1))$$

$$\hat{c}_{\text{RESET}} = \mathbf{0 \text{ units}} \quad (O(1))$$

Fund Allocation for the 3 Units Charged to `INCREMENT`:

1. 1 unit pays immediately for the actual flip of the bit from $0 \rightarrow 1$.
2. 1 unit is deposited on the newly set 1 bit to pay for its future flip from $1 \rightarrow 0$ during subsequent `INCREMENT` operations.
3. 1 unit is deposited into a global **"Reset Bank"**.

4. Proof of Credit Nonnegativity

Let the total credit in the counter system after t operations be C_t .

The system maintains two distinct credit reserves:

1. **Bit Credit:** 1 unit on every bit currently holding 1 ($= b_t$, where b_t is the number of 1-bits).
2. **Reset Fund (R_t):** Contains 1 unit from every `INCREMENT` operation executed since the last `RESET`.

$$C_t = b_t + R_t.$$

We prove that $C_t \geq 0$ for all $t \geq 0$:

- **Base Case ($t = 0$):**
Counter is empty: $b_0 = 0, R_0 = 0 \implies C_0 = 0 \geq 0$.
- **Inductive Step ($t \rightarrow t + 1$):**
- **Case 1: Operation is `INCREMENT`**
- Let m bits flip from $1 \rightarrow 0$ and 1 bit flip from $0 \rightarrow 1$.
- Actual cost: $c_{t+1} = m + 1$.
- Amortized cost charged: $\hat{c}_{t+1} = 3$.
- Net credit change: $\Delta C = \hat{c}_{t+1} - c_{t+1} = 3 - (m + 1) = 2 - m$.
- The m bits flipping $1 \rightarrow 0$ release their m units of bit credit.
- 1 unit of credit is placed on the new 1-bit, and 1 unit is added to the Reset Fund:

$$R_{t+1} = R_t + 1, \quad b_{t+1} = b_t - m + 1.$$

$$C_{t+1} = b_{t+1} + R_{t+1} \geq 0.$$

The invariant is preserved.

- **Case 2: Operation is `RESET`**
- Actual cost: $c_{t+1} = \text{max_bit} + 1$.
- Amortized cost charged: $\hat{c}_{t+1} = 0$.
- Deficit to cover: $c_{t+1} - \hat{c}_{t+1} = \text{max_bit} + 1$.
- Let r be the number of `INCREMENT` operations executed since the previous reset.
- Since each increment added 1 unit to R , the Reset Fund contains:

$$R_t = r \text{ units.}$$

- To reach index `max_bit`, the counter value must have been at least $2^{\text{max_bit}}$:

$$r \geq 2^{\text{max_bit}} \geq \text{max_bit} + 1 \quad (\forall \text{max_bit} \geq 0).$$

- Therefore:

$$R_t \geq \text{max_bit} + 1 = c_{t+1}.$$

- The Reset Fund completely pays for the loop in `RESET` !
- After `RESET` : $\text{max_bit} = -1, b_{t+1} = 0$, and the remaining Reset Fund is:

$$R_{t+1} = R_t - (\text{max_bit} + 1) \geq 0.$$

$$C_{t+1} = 0 + R_{t+1} \geq 0.$$

The invariant is preserved.

5. Total Worst-Case Time Bound

Since the accumulated credit is non-negative at every step ($C_n \geq 0$):

$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i \leq 3n = \mathbf{O(n)}.$$

Any sequence of n `INCREMENT` and `RESET` operations on an initially zero counter executes in $O(n)$ **total worst-case time**.

Q.5 (a) Suffix Trie Substring Search in $O(m)$ Time and Genomic Infeasibility [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5; Ref. [4], Drozdek, Ch. 7)

1. Why Substring Search Takes $O(m)$ Time Independent of Text Length n

The Fundamental Substring Theorem:

Any substring of a string S is a prefix of some suffix of S .

In a suffix trie constructed for a string S of length n :

1. The trie stores all n suffixes of S (appended with a unique terminator $\$$):

$$\{S[0 \dots n-1]\$, S[1 \dots n-1]\$, \dots, S[n-1 \dots n-1]\$ \}.$$

2. Since every path from the root to any node spells out a prefix of one or more suffixes, **every valid substring of S corresponds to a unique path starting at the root.**

Search Mechanics for Pattern P of Length m :

- Start at the root node.
- For each character $P[k]$ for $k = 0, 1, \dots, m-1$:
- Check if an outgoing edge labeled $P[k]$ exists from the current node.
- If the edge exists, follow it to the child node.
- If the edge does not exist, the pattern P is **not a substring** of S (terminate and return `False`).
- If all m characters of P are successfully matched, the algorithm terminates at some node v and returns `True`.
(Note: Node v does not need to be a leaf; finding P at an internal node confirms that P is a valid substring of S).

Complexity:

At each character, array indexing into the child array takes $O(1)$ time (or $O(|\Sigma|)$). Since the path has length exactly m , the total search time is strictly:

$$O(m \cdot |\Sigma|) = O(m) \text{ time} \quad (\text{for constant alphabet } \Sigma),$$

which is completely independent of the length of the text n .

2. Why Naive Suffix Tries are Unsuitable for Large Genomic Sequences

In bioinformatics, genomic sequences (DNA) consist of billions of nucleotides over the alphabet $\Sigma = \{'A', 'C', 'G', 'T'\}$.

1. **Quadratic Space Complexity $\Theta(n^2)$:**

- A naive suffix trie contains a node for every individual character of every suffix.
- For a sequence of length n with distinct substrings, the total number of nodes is $\Theta(n^2)$.
- The human genome contains $n \approx 3.2 \times 10^9$ base pairs. A quadratic space requirement would demand:

$$n^2 \approx (3.2 \times 10^9)^2 \approx 10^{19} \text{ nodes},$$

which is physically impossible to store on any modern memory system.

- Even for a small bacterial genome ($n \approx 5 \times 10^6$), $n^2 \approx 2.5 \times 10^{13}$ nodes, requiring petabytes of RAM.

2. **Pointer Overhead:**

- Each trie node requires pointers for its outgoing edges ($|\Sigma| = 4$ plus terminal $\$$). In a 64-bit architecture, each pointer takes 8 bytes, meaning each node consumes at least $5 \times 8 = 40$ bytes of memory.
- This structural overhead exacerbates the quadratic node count.

3. **Bioinformatics Solutions:**

Genomic indexing uses space-efficient alternatives:

- **Suffix Trees:** Compacts paths into $O(n)$ nodes.
- **Suffix Arrays:** Stores sorted suffix start positions in an integer array of size $O(n)$ (≈ 12 GB for human genome).
- **FM-Index (Burrows-Wheeler Transform + Rank Dictionaries):** Compresses the entire human genome into just $\approx 2\text{--}4$ GB of RAM, allowing whole-genome alignment on personal computers (e.g., Bowtie, BWA).

Q.5 (b) Suffix Trie for $S = \text{"BANANAS"}$ and Substring Search Trace [8 Marks]

1. List of All Suffixes of $S\$ = \text{"BANANAS\$"}$

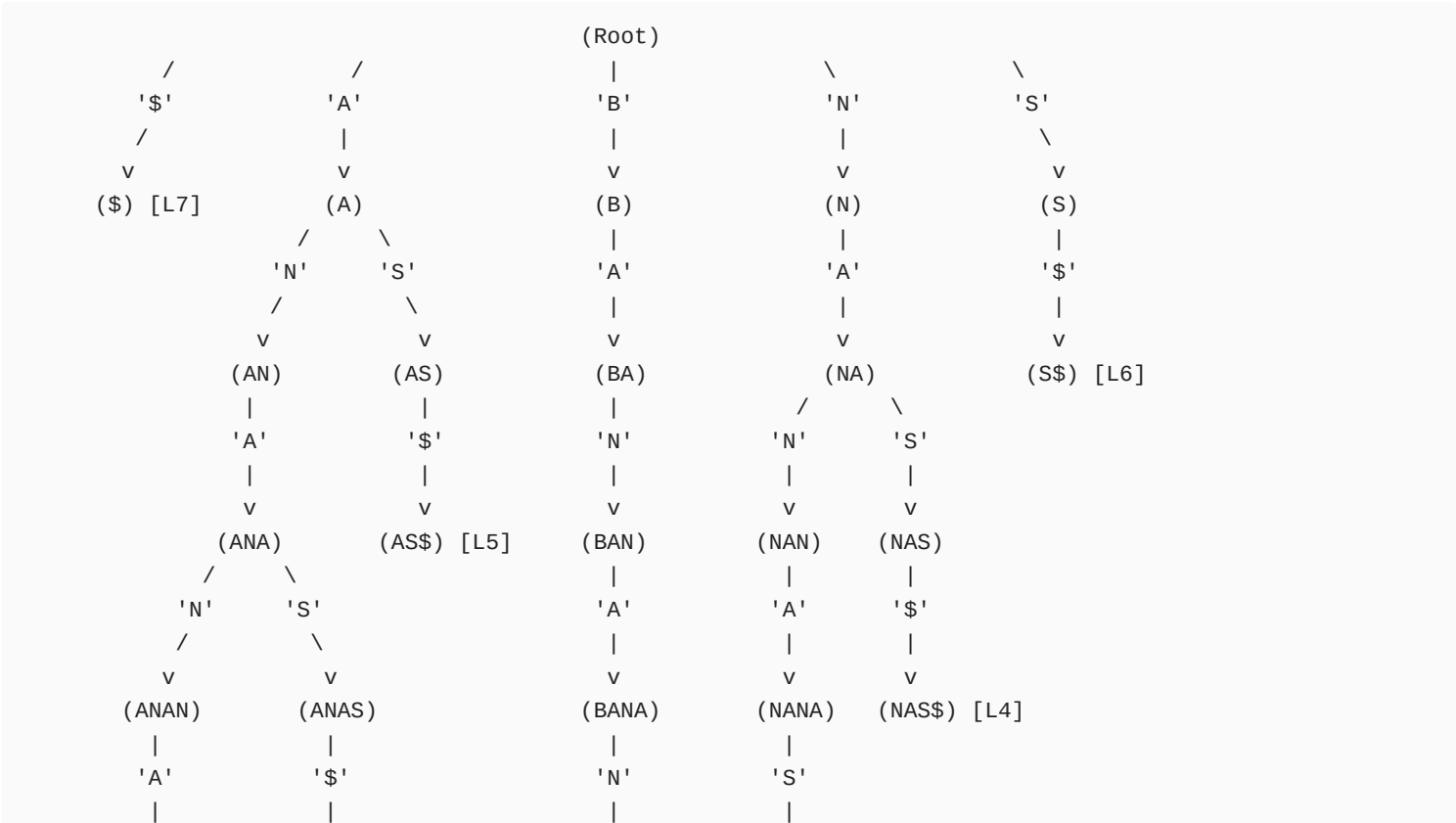
Text: $S = \text{"BANANAS"}$, Length $n = 7$.

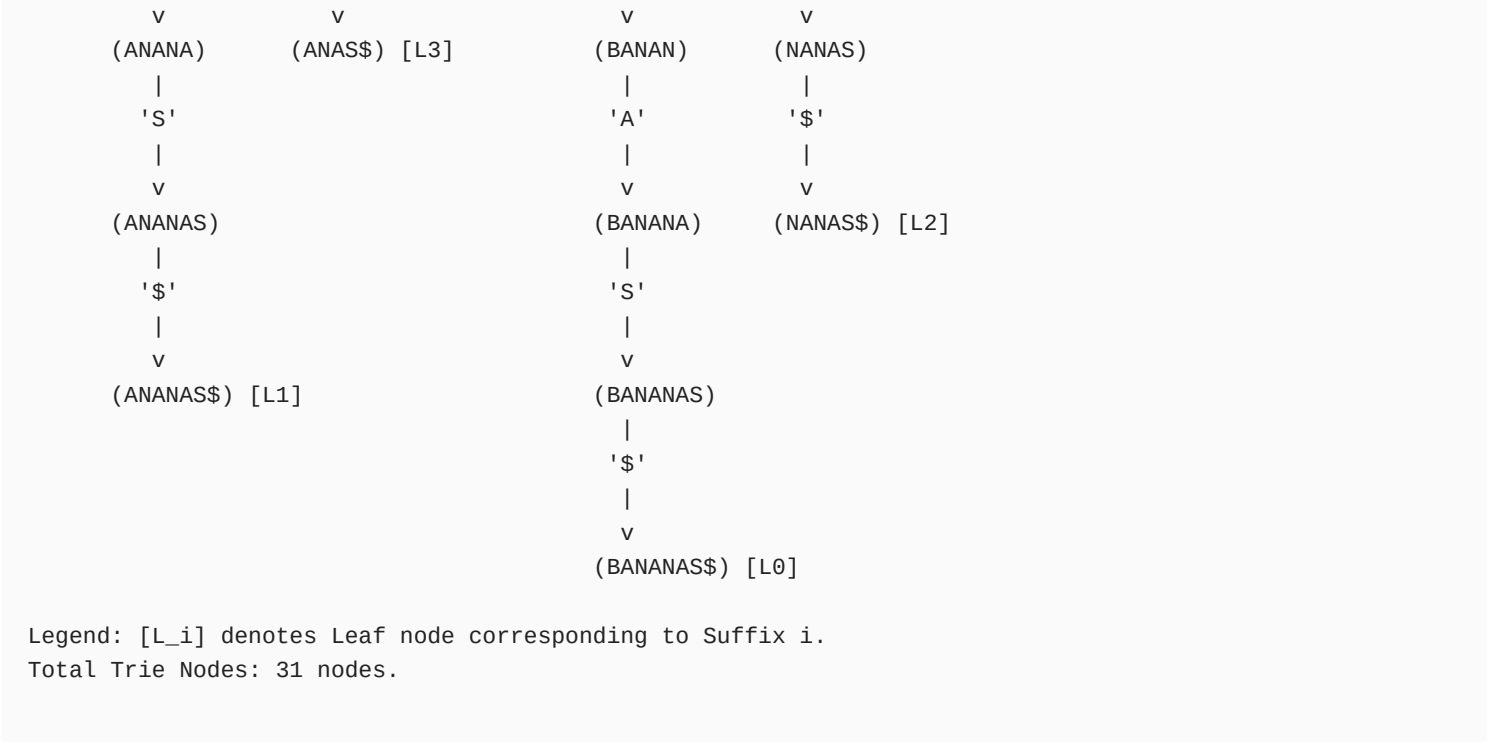
Terminated text: $S\$ = \text{"BANANAS\$"}$, Total Length = 8.

Suffix Index (i)	Suffix $S[i \dots 7]\$$	Length
0	"BANANAS\$"	8
1	"ANANAS\$"	7
2	"NANAS\$"	6
3	"ANAS\$"	5
4	"NAS\$"	4
5	"AS\$"	3
6	"S\$"	2
7	"\$\\"	1

2. Suffix Trie Diagram for "BANANAS\$"

In a standard suffix trie, every edge represents a single character:





3. Substring Search Trace for Pattern $P = \text{"NAN"}$

- **Query Pattern:** $P = \text{"NAN"}$, Length $m = 3$.
- **Pattern Array:** $P[0] = \text{'N'}$, $P[1] = \text{'A'}$, $P[2] = \text{'N'}$.

Step-by-Step Traversal:

Step	Current Node Prefix	Target Character $P[k]$	Edge Lookup	Result / Next Visited Node
0	Root ("")	—	—	Start search at Root
1	Root ("")	$P[0] = \text{'N'}$	Edge 'N' exists?	Yes $\implies$ Traverse edge 'N' to visit Node("N")
2	Node("N")	$P[1] = \text{'A'}$	Edge 'A' exists?	Yes $\implies$ Traverse edge 'A' to visit Node("NA")
3	Node("NA")	$P[2] = \text{'N'}$	Edge 'N' exists?	Yes $\implies$ Traverse edge 'N' to visit Node("NAN")

Search Outcome:

- **Path Traversed:**

$$\text{Root} \xrightarrow{\text{'N'}} \text{Node("N")} \xrightarrow{\text{'A'}} \text{Node("NA")} \xrightarrow{\text{'N'}} \text{Node("NAN")}$$

- **Nodes Visited:** 4 nodes in sequence: Root , Node("N") , Node("NA") , and Node("NAN") .
- **Conclusion:** The pattern P is completely consumed after 3 character edge traversals. The algorithm halts at **Node("NAN")** and returns **True** (pattern "NAN" is confirmed to be a substring of "BANANAS").
- **Occurrence Location:** Traversing downward from Node("NAN") reaches leaf Node("NANAS\$") (Suffix 2). This reveals that "NAN" begins at index **2** of S ("BA **NAN** AS").

Q.6 (a) Theoretical Foundation of KMP Failure Function and $O(m)$ Complexity Proof [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.3)

1. Theoretical Foundation of the Failure Function

For a pattern $P = P[0 \dots m - 1]$, the failure function value $f(i)$ is defined as the length of the longest proper prefix of $P[0 \dots i]$ that is also a suffix of $P[0 \dots i]$.

The Recurrence Relation:

$$f(0) = 0$$

$$f(i) = \begin{cases} j + 1 & \text{if } P[i] = P[j] \text{ where } j = f(i - 1) \\ f(j - 1) \text{ recursively} & \text{if } P[i] \neq P[j] \\ 0 & \text{if no match occurs} \end{cases}$$

Theoretical Rationale of the Recurrence:

1. Inductive Extension ($P[i] == P[j]$):

Suppose we already know $f(i - 1) = j$. This guarantees that the prefix of length j ($P[0 \dots j - 1]$) matches the suffix of length j ending at $i - 1$ ($P[i - j \dots i - 1]$).

- To determine $f(i)$, we test whether this matching prefix-suffix can be extended by 1 character: we compare $P[i]$ with the character immediately following the prefix, which is $P[j]$.
- If $P[i] == P[j]$, the matching prefix-suffix extends to length $j + 1$. Thus, $f(i) = j + 1$.

2. Recursive Fallback ($P[i] \neq P[j]$):

If $P[i] \neq P[j]$, we cannot extend the prefix of length j . However, any smaller candidate prefix of $P[0 \dots i]$ that matches a suffix of $P[0 \dots i]$ must also be a proper prefix of $P[0 \dots j - 1]$ that is simultaneously a suffix of $P[0 \dots j - 1]$.

- The next longest such candidate is given by $f(j - 1)$.
- We update $j \leftarrow f(j - 1)$ and re-evaluate $P[i] == P[j]$.
- This recursion continues until either a character match is found (yielding $f(i) = j + 1$) or j falls to 0 with no match, in which case $f(i) = 0$.

2. Proof that Failure Function Construction Runs in $O(m)$ Time

Consider the standard iterative precomputation algorithm:

```
Algorithm ComputeFailureFunction(P, m):
1.  f[0] = 0
2.  j = 0
3.  for i = 1 to m - 1:
4.      while j > 0 and P[i] != P[j]:
5.          j = f[j - 1]
6.      if P[i] == P[j]:
7.          j = j + 1
8.      f[i] = j
```

Amortized Analysis via the Potential Method:

Define a potential function Φ equal to the value of pointer j :

$$\Phi = j.$$

- **Initial State** ($i = 0$): $j = 0 \implies \Phi_0 = 0$.
- **Non-negativity**: Because j represents the length of a prefix, $j \geq 0 \implies \Phi \geq 0$ at all times.

Accounting across Loop Iterations:

1. Increments of j (Lines 6–7):

In each iteration of the outer `for` loop (Line 3), j increases by **at most 1** (only when $P[i] == P[j]$).

- Since the outer loop executes exactly $m - 1$ times, the pointer j can increase by at most 1 per iteration.

- **Total increases in j across the entire algorithm $\leq m - 1$.**

2. **Decrements of j (Lines 4–5):**

Each execution of the inner `while` loop sets $j \leftarrow f[j - 1]$.

- By definition of proper prefix-suffix, $f[j - 1] < j$. Thus, each iteration of the while loop **strictly decreases j** by at least 1.
- Because j starts at 0 and never becomes negative ($j \geq 0$), the total number of decrements of j can never exceed the total number of increments.
- Consequently, the inner `while` loop can execute **at most $m - 1$ times across all outer iterations combined**.

Total Time Summation:

$$\begin{aligned} \text{Total Operations} &= (\text{Outer loop iterations}) + (\text{Total while loop executions}) \\ &\leq (m - 1) + (m - 1) \\ &= 2m - 2 = \mathbf{O(m)}. \end{aligned}$$

Thus, constructing the failure function runs in $O(m)$ **deterministic time**. ■

Q.6 (b) Failure Function Construction and KMP Trace for $P = \text{”ABAABAC”}$ [8 Marks]

1. Complete Failure Function Table $f[0 \dots 6]$ for $P = \text{”ABAABAC”}$

Pattern length: $m = 7$.

Indices: $P[0] = \text{'A'}$, $P[1] = \text{'B'}$, $P[2] = \text{'A'}$, $P[3] = \text{'A'}$, $P[4] = \text{'B'}$, $P[5] = \text{'A'}$, $P[6] = \text{'C'}$.

i	Substring $P[0 \dots i]$	Proper Prefixes	Proper Suffixes	Longest Common Prefix-Suffix	$f(i)$
0	"A"	$\emptyset$	$\emptyset$	None	0
1	"AB"	{ "A" }	{ "B" }	None	0
2	"ABA"	{ "A", "AB" }	{ "A", "BA" }	"A"	1
3	"ABAA"	{ "A", "AB", "ABA" }	{ "A", "AA", "BAA" }	"A"	1
4	"ABAAB"	{ "A", "AB", "ABA", "ABAA" }	{ "B", "AB", "AAB", "BAAB" }	"AB"	2
5	"ABAABA"	{ "A", "AB", "ABA", ... }	{ "A", "BA", "ABA", "AABA", ... }	"ABA"	3
6	"ABAABAC"	{ "A", "AB", ... }	{ "C", "AC", "BAC", "ABAC", ... }	None	0

$$\mathbf{f = [0, 0, 1, 1, 2, 3, 0]}$$

2. KMP Execution Trace on $T = \text{”ABAABABAAABAABAC”}$

- **Text T ($n = 16$):**

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Char:	A	B	A	A	B	A	B	A	A	A	B	A	A	B	A	C

- **Pattern P ($m = 7$):** "ABAABAC"
- **Failure Table f :** [0, 0, 1, 1, 2, 3, 0]

Complete Step-by-Step Execution Table:

Step	Text Index (i)	$T[i]$	Pattern Index (j)	$P[j]$	Match Status	Action / State Transition
1	0	'A'	0	'A'	Match	$i \leftarrow 1, j \leftarrow 1$
2	1	'B'	1	'B'	Match	$i \leftarrow 2, j \leftarrow 2$
3	2	'A'	2	'A'	Match	$i \leftarrow 3, j \leftarrow 3$
4	3	'A'	3	'A'	Match	$i \leftarrow 4, j \leftarrow 4$
5	4	'B'	4	'B'	Match	$i \leftarrow 5, j \leftarrow 5$
6	5	'A'	5	'A'	Match	$i \leftarrow 6, j \leftarrow 6$
7	6	'B'	6	'C'	MISMATCH	$j > 0 \implies j \leftarrow f[6 - 1] = f[5] = \mathbf{3}; (i \text{ stays } 6)$
8	6	'B'	3	'A'	MISMATCH	$j > 0 \implies j \leftarrow f[3 - 1] = f[2] = \mathbf{1}; (i \text{ stays } 6)$
9	6	'B'	1	'B'	Match	$i \leftarrow 7, j \leftarrow 2$
10	7	'A'	2	'A'	Match	$i \leftarrow 8, j \leftarrow 3$
11	8	'A'	3	'A'	Match	$i \leftarrow 9, j \leftarrow 4$
12	9	'A'	4	'B'	MISMATCH	$j > 0 \implies j \leftarrow f[4 - 1] = f[3] = \mathbf{1}; (i \text{ stays } 9)$
13	9	'A'	1	'B'	MISMATCH	$j > 0 \implies j \leftarrow f[1 - 1] = f[0] = \mathbf{0}; (i \text{ stays } 9)$
14	9	'A'	0	'A'	Match	$i \leftarrow 10, j \leftarrow 1$
15	10	'B'	1	'B'	Match	$i \leftarrow 11, j \leftarrow 2$
16	11	'A'	2	'A'	Match	$i \leftarrow 12, j \leftarrow 3$
17	12	'A'	3	'A'	Match	$i \leftarrow 13, j \leftarrow 4$
18	13	'B'	4	'B'	Match	$i \leftarrow 14, j \leftarrow 5$
19	14	'A'	5	'A'	Match	$i \leftarrow 15, j \leftarrow 6$
20	15	'C'	6	'C'	MATCH	Complete Pattern Match! ($j = m - 1 = 6$)

3. Summary of Mismatch Fallbacks and Match Result

- Mismatch Fallback Transitions:

- At Step 7:** $T[6] = \text{'B'} \neq P[6] = \text{'C'} \implies j \leftarrow f[5] = \mathbf{3}$
- At Step 8:** $T[6] = \text{'B'} \neq P[3] = \text{'A'} \implies j \leftarrow f[2] = \mathbf{1}$
- At Step 12:** $T[9] = \text{'A'} \neq P[4] = \text{'B'} \implies j \leftarrow f[3] = \mathbf{1}$
- At Step 13:** $T[9] = \text{'A'} \neq P[1] = \text{'B'} \implies j \leftarrow f[0] = \mathbf{0}$

- Starting Index Calculation:

$$\text{Start Index} = i - m + 1 = 15 - 7 + 1 = \mathbf{9}.$$

- Verification:

$$T[9 \dots 15] = \text{"ABAABAC"} = P.$$

The final match is located at **starting index 9**.

Q.1 (a) Performance Improvement of KMP over Brute Force [3 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.1, 12.3.3)

- Elimination of Text Backtracking:

When a mismatch occurs at $T[i] \neq P[j]$, the Brute Force algorithm discards all previous comparisons, resetting the text

- pointer back to $i - j + 1$ and the pattern pointer to 0. In contrast, KMP **never retreats the text pointer i** ; i is strictly non-decreasing ($\Delta i \geq 0$).
2. **Exploitation of Pattern Self-Symmetry:**
KMP precomputes the failure function (f /LPS table) in $O(m)$ time. Upon a mismatch, it uses $f[j - 1]$ to slide the pattern to align the longest valid proper prefix with the already matched text, resuming comparisons directly at $T[i]$ with $P[f[j - 1]]$.
3. **Worst-Case Complexity Leap:**
On repetitive strings (e.g., $T = a^n$, $P = a^{m-1}b$), Brute Force degrades to quadratic $O(\mathbf{n} \cdot \mathbf{m})$ time, whereas KMP guarantees linear deterministic $O(n + m)$ time.

Q.1 (b) Role of an Oracle in Reductions and Invariance to Oracle Complexity [3 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1)

1. Definition and Role of an Oracle

In computational complexity theory, an **oracle** (or "black box") for a decision problem X is an abstract computational device capable of solving any instance of X in **a single step ($O(1)$ time)**.

- In a polynomial-time reduction $Y \leq_P X$, the reduction algorithm for Y is permitted to query the oracle for X a polynomial number of times.

2. Why the Oracle's Internal Complexity Does Not Affect Reduction Validity

- Relative Complexity Metric:** The reduction $Y \leq_P X$ does **not** assert that X is easy; it asserts a conditional, relative relationship: "*If X were solvable in polynomial time, then Y would also be solvable in polynomial time.*"
- Mathematical Definition:** By definition of Turing/Karp reductions, each oracle invocation is assigned a nominal cost of 1 unit of time. Whether problem X requires exponential time, doubly-exponential time, or is undecidable has **zero bearing** on whether Y reduces to X in polynomial time.

Q.1 (c) Trie Child Representation: Fixed Array vs. Dynamic Hash Map [4 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.1)

Dimension	Fixed Array (<code>TrieNode* children[26]</code>)	Dynamic Hash Map (<code>unordered_map<char, TrieNode*></code>)
Lookup Time	Guaranteed Worst-Case $O(1)$:	

Direct memory offset calculation (`index = c - 'a'`). Extremely fast; no hash collisions; cache-friendly. | **Average $O(1)$, Worst-Case $O(k)$:**

Requires computing a hash code and resolving bucket collisions. Slower constant factor due to pointer dereferencing. | **Memory Consumption | High / Wasteful for Sparse Nodes:**

Allocates 26 pointers ($26 \times 8 = 208$ bytes on 64-bit systems) at **every node**, even if the node has only 1 child. Infeasible for large alphabets (e.g., Unicode). | **Compact / Proportional to Degree:**

Allocates memory strictly for edges that actually exist ($O(\text{out-degree})$). Highly efficient for sparse trees or very large alphabets. |

Q.1 (d) Optimization vs. Decision Problems & Binary Search using an Oracle [5 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1)

1. Optimization vs. Decision Problems

- **Optimization Problem:** Seeks the **best possible numerical value or structure** among all feasible candidates (e.g., "*Find the maximum size of an independent set in graph G ,*" denoted $\alpha(G)$).
 - **Decision Problem:** Poses a predicate requiring a **binary YES/NO answer** against a given threshold k (e.g., "*Does G contain an independent set of size at least k ?*").
-

2. Finding Maximum Independent Set in $O(\log |V|)$ Oracle Queries via Binary Search

Let $G = (V, E)$ have $n = |V|$ vertices.

The size of any independent set S is an integer bounded by:

$$0 \leq |S| \leq n.$$

Monotonicity Property:

Let $\text{Oracle}(G, k)$ return YES if G has an independent set of size $\geq k$, and NO otherwise:

- If $\text{Oracle}(G, k) = \text{YES}$, then G also contains independent sets of size $k - 1, k - 2, \dots, 0$ (any subset of an independent set is independent).
- If $\text{Oracle}(G, k) = \text{NO}$, G cannot contain an independent set of size $k + 1, k + 2, \dots, n$.

The oracle's answers as k ranges from 0 to n form a sorted boolean sequence:

$$\underbrace{\text{YES, YES, } \dots, \text{ YES}}_{k \leq \alpha(G)}, \quad \underbrace{\text{NO, NO, } \dots, \text{ NO}}_{k > \alpha(G)}$$

Binary Search Algorithm:

```
Algorithm Find_Max_Independent_Set_Size(G, n):
    low = 0
    high = n
    max_size = 0

    while low <= high:
        mid = floor((low + high) / 2)
        if Oracle(G, mid) == YES:
            max_size = mid           // mid is feasible; attempt larger
            low = mid + 1
        else:
            high = mid - 1          // mid is infeasible; search smaller

    return max_size
```

Query Complexity:

The search interval $[0, n]$ is halved at each step. The exact number of decision oracle invocations is:

$$\lceil \log_2(n + 1) \rceil = O(\log |V|) \text{ queries.}$$

Q.1 (e) Accounting Proof of Credit Nonnegativity for $\hat{c}_i = 3$ [5 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Cost Model & Amortized Assignment

- **Actual Cost (c_i):**

$$c_i = \begin{cases} i & \text{if } i = 2^j \text{ for some integer } j \geq 0 \\ 1 & \text{otherwise} \end{cases}$$

- **Assigned Amortized Cost:** $\hat{c}_i = 3$ for every operation $i \geq 1$.

2. Total Amortized Cost

For any sequence of n operations:

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n 3 = 3n.$$

3. Total Actual Cost Summation

Let $k = \lfloor \log_2 n \rfloor$ be the highest power of 2 such that $2^k \leq n$.

The operations where cost is a power of 2 are: $2^0, 2^1, 2^2, \dots, 2^k$ (total $k + 1$ operations).

- Number of non-power-of-2 operations: $n - (k + 1)$, each costing 1.
- Sum of power-of-2 operations: $\sum_{j=0}^k 2^j = 2^{k+1} - 1$.

$$\begin{aligned} \sum_{i=1}^n c_i &= (n - (k + 1)) \cdot 1 + \sum_{j=0}^k 2^j \\ &= n - k - 1 + (2^{k+1} - 1) \\ &= n + 2^{k+1} - k - 2. \end{aligned}$$

4. Proof that Accumulated Credit Remains Nonnegative ($\text{Credit}_n \geq 0$)

The accumulated credit after n operations is:

$$\begin{aligned} \text{Credit}_n &= \sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i \\ &= 3n - (n + 2^{k+1} - k - 2) \\ &= 2n - 2^{k+1} + k + 2. \end{aligned}$$

We evaluate the terms:

1. By definition of $k = \lfloor \log_2 n \rfloor$:

$$2^k \leq n < 2^{k+1} \implies 2n \geq 2 \cdot 2^k = 2^{k+1} \implies 2n - 2^{k+1} \geq 0.$$

2. Since $n \geq 1$, $k = \lfloor \log_2 n \rfloor \geq 0$, which implies:

$$k + 2 \geq 2.$$

Combining both inequalities:

$$\text{Credit}_n = \underbrace{(2n - 2^{k+1})}_{\geq 0} + \underbrace{(k + 2)}_{\geq 2} \geq 2 > 0 \quad \forall n \geq 1.$$

The stored credit is **strictly positive (≥ 2) for all $n \geq 1$** , completely proving that credit never drops below zero.

Q.1 (f) Text Processing Operations in Document Indexing [5 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

[Raw Text] ---> (Tokenization) ---> (Stop-Word Removal) ---> (Stemming) ---> [Index Postings]

1. Tokenization:

- **Definition:** The lexical analysis phase that segments a continuous character stream into distinct linguistic tokens (words), stripping punctuation, whitespace, and formatting tags.
- **Example:** Raw text "Search-engine design, e.g., KMP & Tries!" $\implies$ Tokens: ["search", "engine", "design", "e.g.", "kmp", "tries"].

2. Stop-Word Removal:

- **Definition:** Filtering out high-frequency functional words (articles, prepositions, pronouns) that appear ubiquitously across all documents and carry virtually zero discriminative semantic value, reducing inverted index size by 30--40%.
- **Example:** Sentence ["the", "efficiency", "of", "a", "search", "engine"] $\implies$ After filtering: ["efficiency", "search", "engine"] (removing "the", "of", "a").

3. Stemming:

- **Definition:** The morphological process of stripping affixes (prefixes/suffixes) to reduce inflectional or derived word variants to a common base root (e.g., using the Porter Stemmer), ensuring that a search for one grammatical form retrieves documents containing all related forms.
- **Example:** Words ["connecting", "connection", "connections", "connected"] $\implies$ All reduced to the common stem: "connect".

Q.1 (g) Isolated Vertices in the Reduction Vertex Cover $\leq_P$ Set Cover [5 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.3)

In the reduction from Vertex Cover to Set Cover:

- Universe $U = E$ (the set of all edges in G).
- Subset family $\mathcal{S} = \{S_v \mid v \in V\}$, where $S_v = \{e \in E \mid e \text{ is incident to } v\}$.
- Budget quota $k' = k$.

Behavior with an Isolated Vertex ($u \in V$ with $\deg(u) = 0$):

If u is an isolated vertex, it has no incident edges. Its corresponding subset in $\mathcal{S}$ is:

$$S_u = \emptyset.$$

Why Correctness is Preserved:

1. Irrelevance to Edge Coverage in G :

An isolated vertex covers zero edges in G . A graph G has a vertex cover of size $\leq k$ if and only if the subgraph $G \setminus \{u\}$ has a vertex cover of size $\leq k$. Including an isolated vertex in a vertex cover consumes budget without covering any edge.

2. Inutility in Set Cover:

The subset $S_u = \emptyset$ contains zero elements of the universe $U = E$. Selecting S_u in a candidate set cover $\mathcal{C} \subseteq \mathcal{S}$ increments the subset count by 1 without covering any element of U .

- Any valid set cover $\mathcal{C}$ that covers U satisfies:

$$\bigcup_{S_v \in \mathcal{C} \setminus \{S_u\}} S_v = \bigcup_{S_v \in \mathcal{C}} S_v = U.$$

- If a set cover of size $\leq k'$ exists using S_u , dropping S_u yields a smaller valid set cover of size $\leq k' - 1$.

3. Soundness & Completeness:

- ($\implies$) If G has a vertex cover C with $|C| \leq k$, we can choose C to contain no isolated vertices. The corresponding subsets $\{S_v \mid v \in C\}$ cover $U = E$ with cardinality $\leq k$.
- ($\impliedby$) If $\mathcal{S}$ has a set cover $\mathcal{C}$ of size $\leq k$, any empty subset $S_u = \emptyset$ in $\mathcal{C}$ can be replaced by an arbitrary non-empty subset (or omitted), directly mapping back to a valid vertex cover in G of size $\leq k$.

Thus, empty sets $S_u = \emptyset$ are completely benign and do not compromise the correctness of the reduction.

Q.2 (a) Requirements for NP-Completeness and the Role of Transitivity [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.4)

1. The Two Formal Requirements for NP-Completeness

To prove that a decision problem (or language) X is **NP-complete**, one must establish two conditions:

1. Requirement 1: Membership in NP ($X \in \text{NP}$)

- There must exist a deterministic polynomial-time algorithm $B(s, t)$ (a **certifier**) and a polynomial $p(\cdot)$ such that for every input string s :

$$s \in X \iff \exists t \text{ with } |t| \leq p(|s|) \text{ such that } B(s, t) = \text{"yes"}.$$

- Intuitively, a candidate solution (certificate) can be **verified** in deterministic polynomial time.

2. Requirement 2: Hardness for NP (X is NP-hard)

- Every problem A in the complexity class **NP** must be polynomial-time reducible to X :

$$\forall A \in \text{NP}, \quad A \leq_P X.$$

- Intuitively, X is at least as hard as any problem in **NP**.

2. The Role of Transitivity ($Y \leq_P X$) in NP-Completeness Proofs

The Challenge of the Direct Definition:

The literal definition of NP-hardness requires demonstrating an infinite family of reductions—namely, reducing *every single problem* $A \in \text{NP}$ to X . Directly constructing an infinite number of reductions from scratch for each new problem is impossible.

The Transitivity Lemma:

Polynomial-time reduction ($\leq_P$) is a **transitive relation**:

$$\text{If } A \leq_P Y \text{ and } Y \leq_P X, \text{ then } A \leq_P X.$$

• Proof of Transitivity:

If $A \leq_P Y$, there exists a polynomial-time reduction f taking time $O(|s|^c)$.

If $Y \leq_P X$, there exists a polynomial-time reduction g taking time $O(|s|^d)$.

The composition $(g \circ f)(s) = g(f(s))$ maps instances of A directly to instances of X . Since the size $|f(s)| = O(|s|^c)$, the composed reduction $g(f(s))$ runs in time $O((|s|^c)^d) = O(|s|^{cd})$, which is strictly **polynomial**.

```
Any Problem A in NP  -----[ Cook-Levin / Known ]-----> Known NP-Complete Problem Y
                                                                |
                                                                | (Single Reduction)
                                                                v
                                                                Target Problem X
=====>
By Transitivity: A <=_P X for ALL A in NP!
```

How Transitivity Streamlines Hardness Proofs:

1. Suppose we select a problem Y that is **already known to be NP-complete** (e.g., Circuit-SAT or 3-SAT).
2. By definition of Y 's NP-completeness, we already know that **every** problem $A \in \mathbf{NP}$ reduces to Y ($A \leq_P Y$).
3. Therefore, to prove that a new target problem X is NP-hard, we only need to construct a **single polynomial-time reduction**:

$$\mathbf{Y} \leq_P \mathbf{X}.$$

4. By transitivity, for every $A \in \mathbf{NP}$, the chain $A \leq_P Y \leq_P X$ guarantees that $A \leq_P X$. This establishes that X is NP-hard without ever referencing the original computational models of NP.

Q.2 (b) Gate Conversion to CNF and Equivalence Proof [8 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.4)

In the reduction from Circuit-SAT to 3-SAT, each gate is represented by a boolean variable for its output wire and variables for its input wires. To preserve the circuit's operation, we enforce the condition that the output variable takes a value consistent with the gate's logic under any satisfying truth assignment.

1. Conversion of an AND Gate: $v = u \wedge w$

- **Logical Equivalence:** $v \iff (u \wedge w)$
- **Bidirectional Decomposition:**

$$1. v \implies (u \wedge w):$$

$$\neg v \vee (u \wedge w) \equiv (\neg v \vee u) \wedge (\neg v \vee w)$$

$$2. (u \wedge w) \implies v:$$

$$\neg(u \wedge w) \vee v \equiv (\neg u \vee \neg w \vee v)$$

Resulting CNF Formula for AND Gate:

$$\Phi_{\text{AND}}(\mathbf{u}, \mathbf{w}, \mathbf{v}) = (\neg \mathbf{u} \vee \neg \mathbf{w} \vee \mathbf{v}) \wedge (\mathbf{u} \vee \neg \mathbf{v}) \wedge (\mathbf{w} \vee \neg \mathbf{v})$$

(Note: The 2-literal clauses can be padded to 3 literals using the zero-enforced helper variable z_1 : $(u \vee \neg v \vee z_1) \wedge (w \vee \neg v \vee z_1)$).

2. Conversion of a NOT Gate: $v = \neg u$

- **Logical Equivalence:** $v \iff \neg u$
- **Bidirectional Decomposition:**

$$1. v \implies \neg u:$$

$$\neg v \vee \neg u \equiv (\neg u \vee \neg v)$$

$$2. \neg u \implies v:$$

$$\neg(\neg u) \vee v \equiv (\mathbf{u} \vee \mathbf{v})$$

Resulting CNF Formula for NOT Gate:

$$\Phi_{\text{NOT}}(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \vee \mathbf{v}) \wedge (\neg \mathbf{u} \vee \neg \mathbf{v})$$

(Note: Padded to 3 literals via z_1 : $(u \vee v \vee z_1) \wedge (\neg u \vee \neg v \vee z_1)$).

3. Correctness Proof: Equivalence Between CNF Satisfaction and Gate Logic

A. Proof for the AND Gate (Φ_{AND} is satisfied $\iff v = u \wedge w$)

We evaluate the three clauses across all $2^3 = 8$ possible truth assignments for (u, w, v) :

- $C_1 = (\neg u \vee \neg w \vee v)$
- $C_2 = (u \vee \neg v)$
- $C_3 = (w \vee \neg v)$

u	w	v	Gate Logic: $u \wedge w$	Match?	Clause C_1	Clause C_2	Clause C_3	$\Phi_{\text{AND}} = C_1 \wedge C_2 \wedge C_3$
0	0	0	0	Yes	1	1	1	1 (True)
0	0	1	0	No	1	0	0	0 (False)
0	1	0	0	Yes	1	1	1	1 (True)
0	1	1	0	No	1	0	1	0 (False)
1	0	0	0	Yes	1	1	1	1 (True)
1	0	1	0	No	1	1	0	0 (False)
1	1	0	1	No	0	1	1	0 (False)
1	1	1	1	Yes	1	1	1	1 (True)

- Analysis:**
- When $v = u \wedge w$ (rows 1, 3, 5, 8), all three clauses evaluate to True ($\Phi_{\text{AND}} = 1$).
- When $v \neq u \wedge w$ (rows 2, 4, 6, 7), at least one clause evaluates to False ($\Phi_{\text{AND}} = 0$).
Specifically:
 - If $u = 1, w = 1$, but $v = 0$, C_1 is violated ($0 \vee 0 \vee 0 = 0$).
 - If $v = 1$, but $u = 0$, C_2 is violated ($0 \vee 0 = 0$).
 - If $v = 1$, but $w = 0$, C_3 is violated ($0 \vee 0 = 0$).

Therefore, $\Phi_{\text{AND}}(u, w, v) = \text{True} \iff v = u \wedge w$. ■

B. Proof for the NOT Gate (Φ_{NOT} is satisfied $\iff v = \neg u$)

We evaluate the two clauses across all $2^2 = 4$ possible truth assignments for (u, v) :

- $C_A = (u \vee v)$
- $C_B = (\neg u \vee \neg v)$

u	v	Gate Logic: $\neg u$	Match?	Clause $C_A = (u \vee v)$	Clause $C_B = (\neg u \vee \neg v)$	$\Phi_{\text{NOT}} = C_A \wedge C_B$
0	0	1	No	0	1	0 (False)
0	1	1	Yes	1	1	1 (True)
1	0	0	Yes	1	1	1 (True)
1	1	0	No	1	0	0 (False)

- Analysis:**
- When $v = \neg u$ (rows 2 and 3), both clauses evaluate to True ($\Phi_{\text{NOT}} = 1$).
- When $v = u = 0$, clause C_A evaluates to False ($0 \vee 0 = 0$).
- When $v = u = 1$, clause C_B evaluates to False ($0 \vee 0 = 0$).

Therefore, $\Phi_{\text{NOT}}(u, v) = \text{True} \iff v = \neg u$. ■

Q.3 (a) Comparison: Standard Trie, Compressed Trie, and Suffix Trie [7 Marks]

Dimension	Standard Trie	Compressed Trie (Radix Tree)	Suffix Trie
1. Input Domain	A set / dictionary of k independent strings:		

$\mathcal{W} = \{S_1, S_2, \dots, S_k\}$. | A set / dictionary of k independent strings:

$\mathcal{W} = \{S_1, S_2, \dots, S_k\}$. | A **single string** S of length n (appended with a unique terminator $\$$):

stores all n suffixes of S . |

| **2. Edge Label Type** | Exactly **one character** ($c \in \Sigma$) per edge. | Non-empty **substrings** of characters (or integer index pairs ($start, end$)). | Exactly **one character** ($c \in \Sigma \cup \{\$\}$) per edge. |

| **3. Node Count Complexity** | $O(N)$ **nodes**, where $N = \sum_{i=1}^k \|S_i\|$ is the total length of all words.

(At most $N + 1$ nodes). | $O(k)$ **nodes** (at most $2k$ nodes).

Bounded strictly by the number of keys k , **independent of word lengths**. | $\Theta(n^2)$ **nodes** in the worst case

(e.g., $1 + \frac{n(n+1)}{2}$ nodes for a string of distinct characters). |

| **4. Query Operations Supported** | • Exact dictionary word lookup in $O(m)$ time.

- Prefix search & auto-complete.
- Lexicographic sorting of dictionary keys. | • Exact dictionary word lookup in $O(m)$ time.
- Prefix search & auto-complete.
- Longest common prefix of dictionary keys. | • Arbitrary **substring search** in text S in $O(m)$ time.
- Longest Repeated Substring.
- Longest Common Substring across multiple texts.

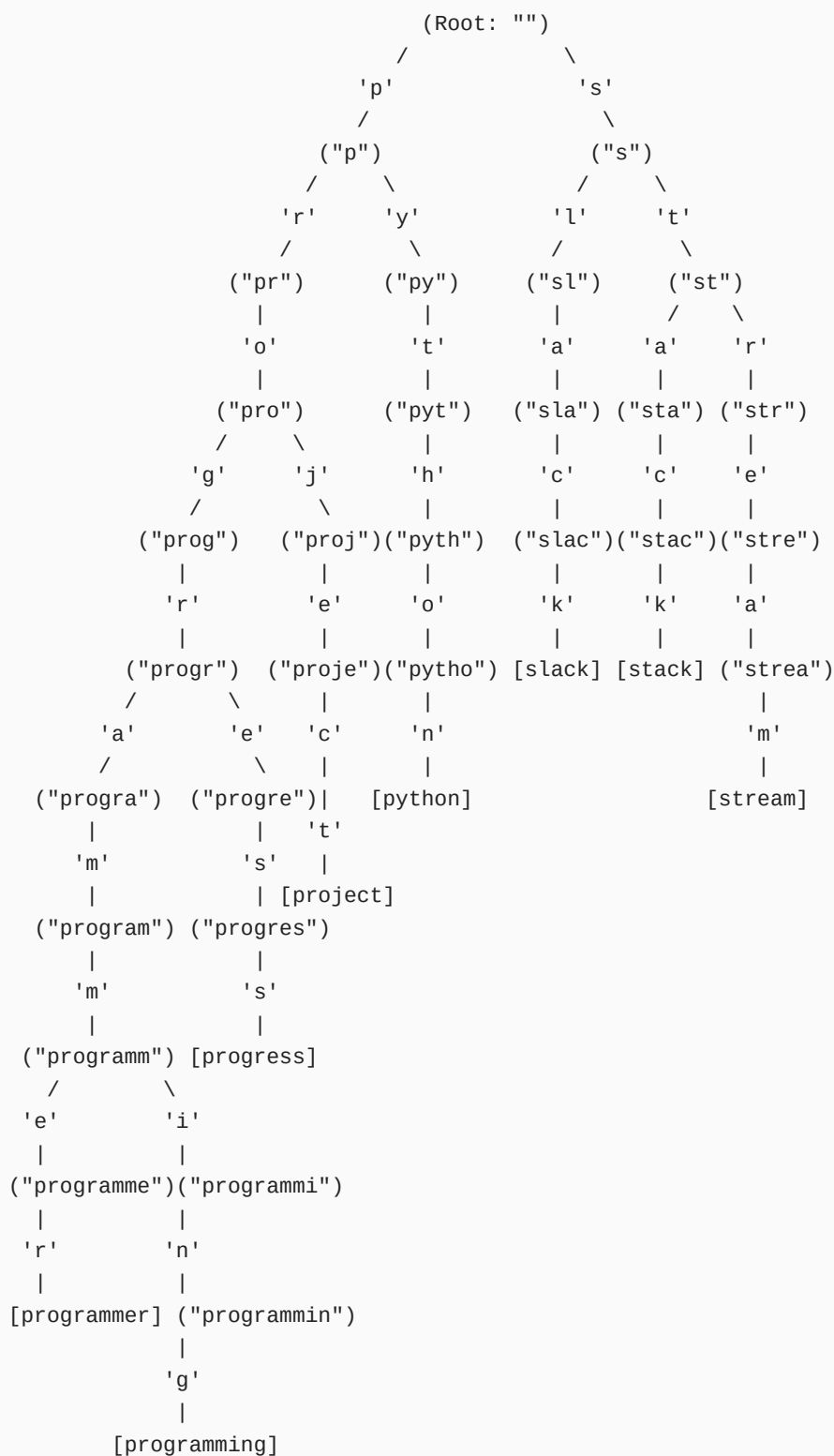
• Substring occurrence frequency and locations. |
| **5. Auxiliary Space Consumption** | **High:** $O(N \cdot \|\Sigma\|)$ memory overhead due to allocating child pointers at every single character node. | **Minimal:** $O(k \cdot \|\Sigma\|)$ pointer overhead. Eliminates all non-branching redundant chains. | **Prohibitive:** $\Theta(n^2 \cdot \|\Sigma\|)$ pointer space in naive form (impractical for large texts/genomes). |

Q.3 (b) Standard Trie vs. Compressed Trie for Word Set $\mathcal{W}$ [8 Marks]

Given dictionary of $k = 8$ words:

$$\mathcal{W} = \{ \text{"programmer"}, \text{"programming"}, \text{"progress"}, \text{"project"}, \text{"python"}, \text{"slack"}, \text{"stack"}, \text{"stream"} \}$$

In the standard trie, words share common prefix paths character-by-character:



- **Prefix "p" tree:**
- Common branch "p" → "pr" → "pro": **3 nodes**
- Sub-branch "prog" → "progr": **2 nodes**
- Split to "program": "progra" → "program" → "programm": **3 nodes**
- Split to "programmer": "programme" → "programmer": **2 nodes**
- Split to "programming": "programmi" → "programmin" → "programming": **3 nodes**
- Split to "progress": "progre" → "progres" → "progress": **3 nodes**
- Split from "pro" to "project": "proj" → "proje" → "projec" → "project": **4 nodes**

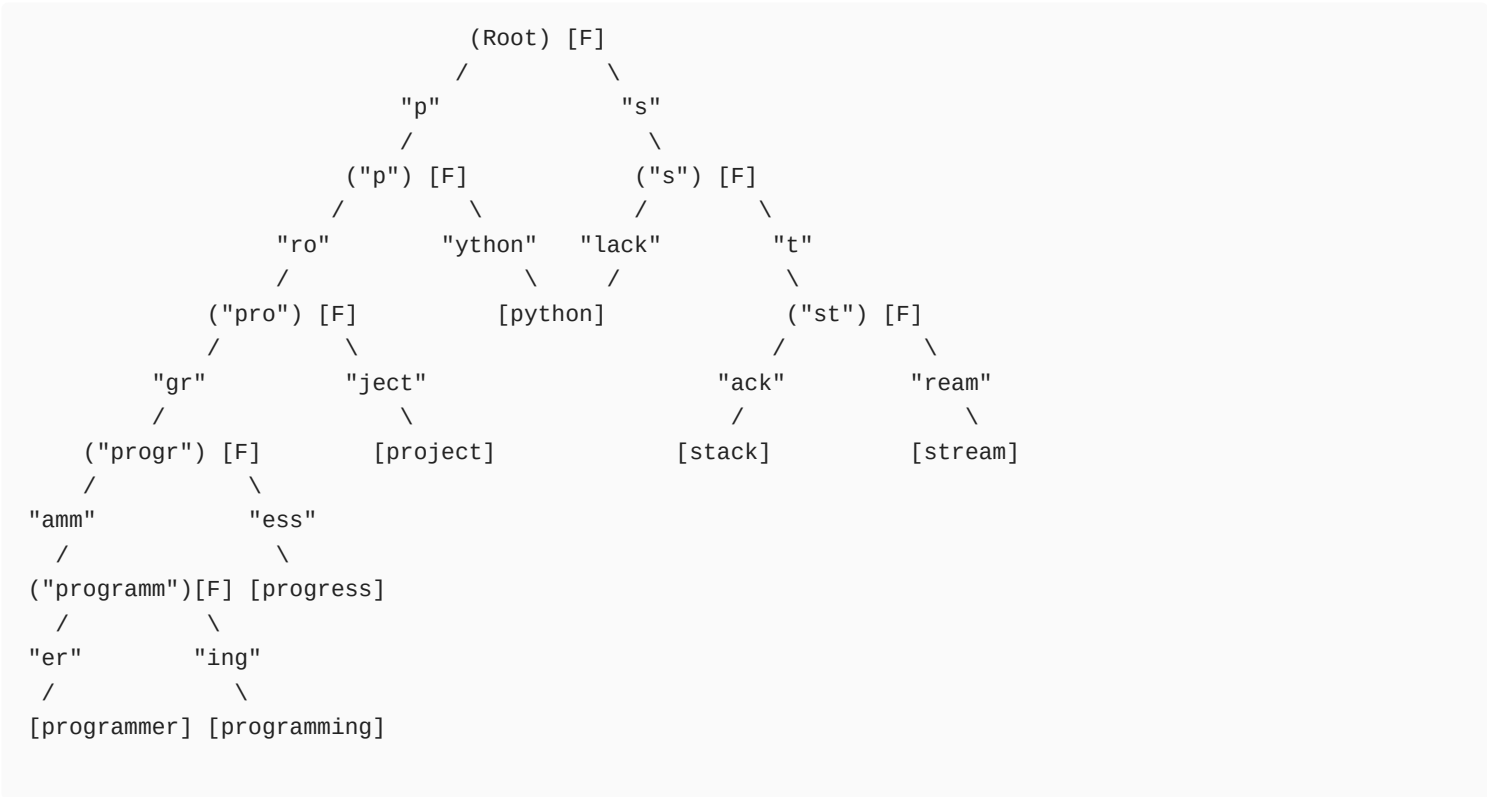
- Sub-branch from "p" to "python": "py" → "pyt" → "pyth" → "pytho" → "python" : **5 nodes**
(Total nodes under "p": $3 + 2 + 3 + 2 + 3 + 3 + 4 + 5 = 25$ nodes)
- **Prefix "s" tree:**
- Common node "s" : **1 node**
- Sub-branch to "slack": "sl" → "sla" → "slac" → "slack" : **4 nodes**
- Common node "st" : **1 node**
- Split to "stack": "sta" → "stac" → "stack" : **3 nodes**
- Split to "stream": "str" → "stre" → "strea" → "stream" : **4 nodes**
(Total nodes under "s": $1 + 4 + 1 + 3 + 4 = 13$ nodes)

Total Character Nodes (excluding Root) = **25 + 13 = 38 nodes.**

Total Nodes (including Root) = $38 + 1 =$ **39 nodes.**

2. Conversion to Compressed Trie

Chains of internal nodes with out-degree 1 that are not terminal words are compressed into single edges labeled by substrings:



Node and Edge Inventory of the Compressed Trie:

- **Root Node:** Root ("") (1 node)
- **Internal Branching Nodes (degree ≥ 2):**
 1. ("p") : branches via "ro" and "ython"
 2. ("pro") : branches via "gr" and "ject"
 3. ("progr") : branches via "amm" and "ess"
 4. ("programm") : branches via "er" and "ing"
 5. ("s") : branches via "lack" and "t"
 6. ("st") : branches via "ack" and "ream"
 (Total internal branching nodes = 6)
- **Terminal Leaf Nodes (Dictionary Words):**
 1. [programmer]
 2. [programming]

3. [progress]
 4. [project]
 5. [python]
 6. [slack]
 7. [stack]
 8. [stream]
- (Total leaf nodes = 8)

Total Nodes in Compressed Trie = 1 (root) + 6 (internal) + 8 (leaves) = 15 nodes.

Character Nodes (excluding Root) = 15 - 1 = 14 nodes.

3. Calculation of Percentage Reduction in Node Count

A. Reduction Based on Total Node Count (Including Root):

- **Initial Standard Trie Nodes:** 39
- **Compressed Trie Nodes:** 15
- **Absolute Nodes Eliminated:** $39 - 15 = 24$ nodes

$$\text{Percentage Reduction} = \frac{39 - 15}{39} \times 100\% = \frac{24}{39} \times 100\% = \mathbf{61.54\%}.$$

B. Reduction Based on Character Nodes (Excluding Root):

- **Initial Standard Trie Character Nodes:** 38
- **Compressed Trie Character Nodes:** 14
- **Absolute Nodes Eliminated:** $38 - 14 = 24$ nodes

$$\text{Percentage Reduction} = \frac{38 - 14}{38} \times 100\% = \frac{24}{38} \times 100\% = \mathbf{63.16\%}.$$

Q.4 (a) Accounting Analysis of a k -Bit Binary Counter [7 Marks]

(Ref. [2], CLRS 4th ed., Sec. 16.2)

1. Cost Assignments in the Accounting Method

Let the actual cost of inspecting and flipping any single bit be $c = 1$ unit.

In an `INCREMENT` operation on a k -bit counter $A[0 \dots k-1]$:

- A sequence of m_t consecutive 1s starting at index 0 flip to 0 ($1 \rightarrow 0$).
- At most one 0 flips to 1 ($0 \rightarrow 1$).
- Actual cost: $c_t = m_t + 1$.

We assign the following amortized costs:

1. Flipping a bit from 0 $\rightarrow$ 1:

$$\hat{c}_{0 \rightarrow 1} = \mathbf{2 \text{ units}}$$

- 1 unit pays immediately for the actual physical flip ($0 \rightarrow 1$).
- 1 unit is deposited directly on that specific bit as **stored credit**.

2. Flipping a bit from 1 $\rightarrow$ 0:

$$\hat{c}_{1 \rightarrow 0} = \mathbf{0 \text{ units}}$$

- The actual physical cost of 1 unit is completely paid for by consuming the 1 unit of credit already residing on that bit from

when it was originally set to 1.

Since each `INCREMENT` performs at most one $0 \rightarrow 1$ flip and m_t flips of $1 \rightarrow 0$:

$$\hat{c}_t = 2 + \mathbf{m}_t \cdot \mathbf{0} = 2 \text{ units } (O(1)).$$

2. Proof of the Credit Invariant: $\text{Credit}_t = b_t$

Let:

- b_t denote the number of 1-bits in the counter after operation t .
- $\text{Credit}_t = \sum_{i=1}^t \hat{c}_i - \sum_{i=1}^t c_i$ denote the total accumulated credit after operation t .

State Invariant Statement:

$$\text{Credit}_t = b_t \quad \forall t \geq 0.$$

(Every bit currently set to 1 holds exactly 1 unit of credit; bits set to 0 hold 0 credit).

Proof by Mathematical Induction:

- Base Case ($t = 0$):**
The counter starts at $000 \dots 0_2$.
The number of 1-bits is $b_0 = 0$.
Total actual cost $\sum c_0 = 0$, total amortized cost $\sum \hat{c}_0 = 0 \implies \text{Credit}_0 = 0$.
$$\text{Credit}_0 = b_0 = 0 \geq 0.$$

The base case holds.

- Inductive Hypothesis:** Assume that after $t - 1$ operations:

$$\text{Credit}_{t-1} = b_{t-1}.$$

- Inductive Step ($t - 1 \rightarrow t$):** Consider the t -th `INCREMENT` operation.
Suppose this operation flips m_t bits from $1 \rightarrow 0$ and 1 bit from $0 \rightarrow 1$:

1. New count of 1-bits (b_t):

$$b_t = b_{t-1} - m_t + 1.$$

2. Actual and Amortized Costs:

$$c_t = m_t + 1, \quad \hat{c}_t = 2.$$

3. Net change in credit during step t (ΔCredit_t):

$$\Delta \text{Credit}_t = \hat{c}_t - c_t = 2 - (m_t + 1) = 1 - m_t.$$

4. Total accumulated credit after step t (Credit_t):

$$\begin{aligned} \text{Credit}_t &= \text{Credit}_{t-1} + \Delta \text{Credit}_t \\ &= b_{t-1} + (1 - m_t) \quad (\text{by Inductive Hypothesis}) \\ &= b_{t-1} - m_t + 1 \\ &= b_t. \end{aligned}$$

Since b_t is the count of 1-bits in the counter, $b_t \geq 0$ for all t .

Thus, $\text{Credit}_t = b_t \geq 0$ holds unconditionally for all $t \geq 0$. ■

3. Derivation of the $O(n)$ Bound for n Operations

Because the accumulated credit $\text{Credit}_n \geq 0$ for any sequence of n operations:

$$\sum_{t=1}^n c_t = \sum_{t=1}^n \hat{c}_t - \text{Credit}_n \leq \sum_{t=1}^n \hat{c}_t.$$

Since each `INCREMENT` is charged an amortized cost $\hat{c}_t \leq 2$:

$$\sum_{t=1}^n \hat{c}_t \leq \sum_{t=1}^n 2 = 2n.$$

Therefore:

$$\sum_{t=1}^n c_t \leq 2n = O(n).$$

The total actual cost of any sequence of n `INCREMENT` operations starting from zero is strictly bounded by $2n$, yielding an amortized cost of $O(1)$ **per operation**.

Q.4 (b) Trace of a 4-Bit Binary Counter Through 10 Successive Increments [8 Marks]

- Initial Counter State ($t = 0$):** $A[3 \dots 0] = 0000_2$, with $\text{Credit}_0 = b_0 = 0$.
- Bits:** $A[0]$ is LSB, $A[3]$ is MSB.
- Credit Formula:** $\text{Credit}_t = \sum_{i=1}^t \hat{c}_i - \sum_{i=1}^t c_i$.

Detailed Step-by-Step Execution Table:

Step (t)	Counter State ($A[3..0]$)	Bit Transitions ($1 \rightarrow 0, 0 \rightarrow 1$)	Actual Flips (c_t)	Number of 1s (b_t)	Amortized Cost ($\hat{c}_t$)	Cumulative Actual ($\sum c_t$)	Cumulative Amortized ($\sum \hat{c}_t$)	Stored Credit (Credit_t)	Invariant Verification ($\text{Credit}_t = b_t \geq 0$)
0	0000	Initial State	—	0	—	0	0	0	$0 = 0 \geq 0 \checkmark$
1	0001	$A[0] : 0 \rightarrow 1$	1	1	2	1	2	1	$1 = 1 \geq 0 \checkmark$
2	0010	$A[0] : 1 \rightarrow 0, A[1] : 0 \rightarrow 1$	2	1	2	3	4	1	$1 = 1 \geq 0 \checkmark$
3	0011	$A[0] : 0 \rightarrow 1$	1	2	2	4	6	2	$2 = 2 \geq 0 \checkmark$
4	0100	$A[0], A[1] : 1 \rightarrow 0, A[2] : 0 \rightarrow 1$	3	1	2	7	8	1	$1 = 1 \geq 0 \checkmark$
5	0101	$A[0] : 0 \rightarrow 1$	1	2	2	8	10	2	$2 = 2 \geq 0 \checkmark$
6	0110	$A[0] : 1 \rightarrow 0, A[1] : 0 \rightarrow 1$	2	2	2	10	12	2	$2 = 2 \geq 0 \checkmark$
7	0111	$A[0] : 0 \rightarrow 1$	1	3	2	11	14	3	$3 = 3 \geq 0 \checkmark$
8	1000	$A[0], A[1], A[2] : 1 \rightarrow 0, A[3] : 0 \rightarrow 1$	4	1	2	15	16	1	$1 = 1 \geq 0 \checkmark$
9	1001	$A[0] : 0 \rightarrow 1$	1	2	2	16	18	2	$2 = 2 \geq 0 \checkmark$
10	1010	$A[0] : 1 \rightarrow 0, A[1] : 0 \rightarrow 1$	2	2	2	18	20	2	$2 = 2 \geq 0 \checkmark$

Verification Summary Across All 10 Steps:

- Total Actual Bit Flips Incurred:** $\sum_{t=1}^{10} c_t = 18$ flips.
- Total Amortized Cost Charged:** $\sum_{t=1}^{10} \hat{c}_t = 10 \times 2 = 20$ units.
- Ending Credit Stored:**

$$\text{Credit}_{10} = 20 - 18 = \mathbf{2 \text{ units.}}$$

4. **Invariant Check at $t = 10$:** Counter is 1010_2 , which contains exactly two 1s ($A[3]$ and $A[1]$), so $b_{10} = 2$.

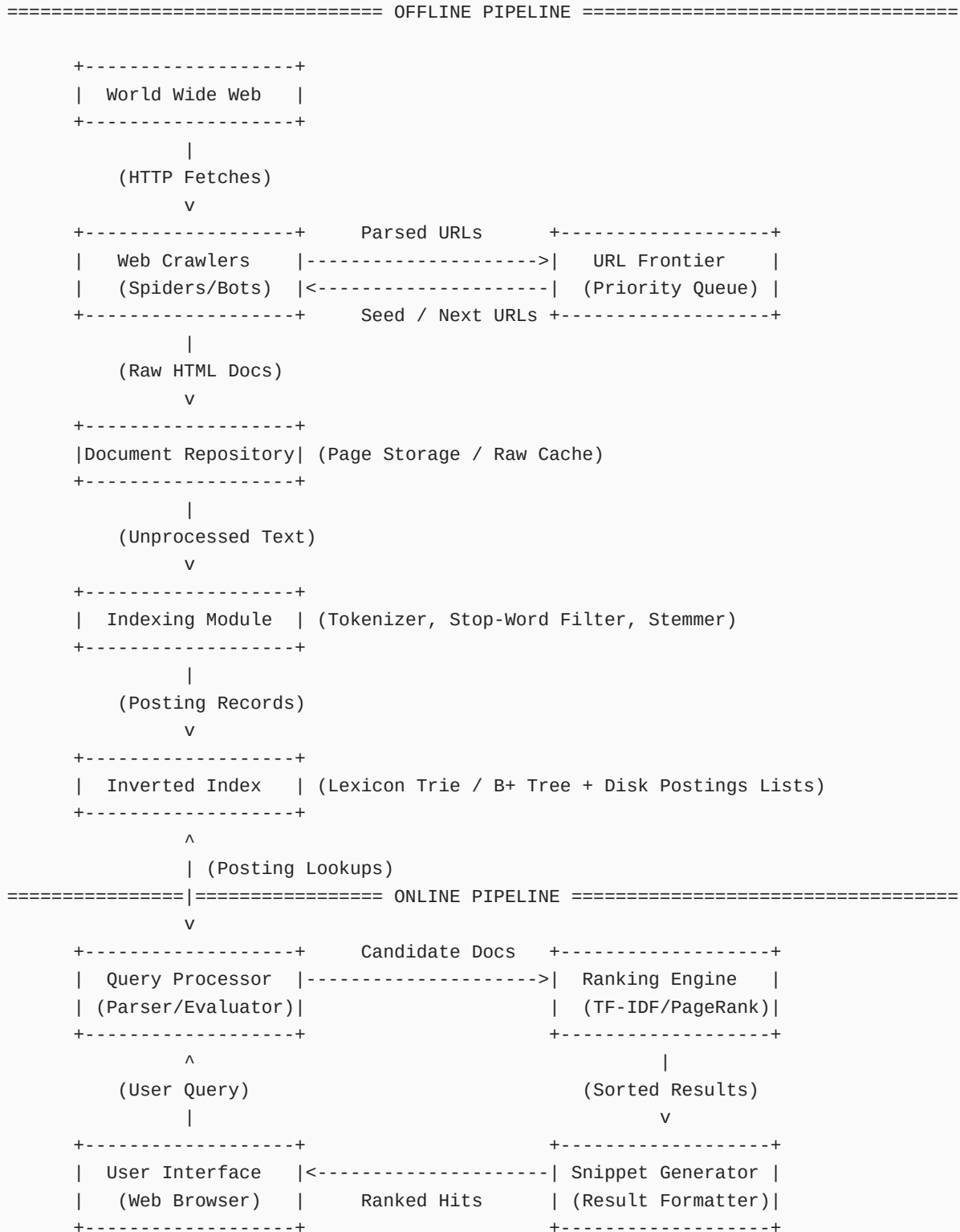
$$\text{Credit}_{10} = \mathbf{b_{10} = 2 \geq 0.}$$

The invariant holds with zero deficits throughout the entire sequence.

Q.5 (a) End-to-End Web Search Engine Architecture & Data Flows [7 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

1. End-to-End Architectural Block Diagram



2. Detailed Data Flows Connecting the Components

1. **Crawlers** $\longleftrightarrow$ **URL Frontier** $\longrightarrow$ **Document Repository**:
- The **Web Crawler** dequeues target URLs from the **URL Frontier** (enforcing politeness delays and crawl prioritization).
 - It issues HTTP/HTTPS requests to web servers, downloads the raw HTML pages, and streams them into the **Document Repository**.
 - It parses outgoing links from the fetched HTML and enqueues newly discovered, normalized URLs back into the **URL Frontier**.
2. **Document Repository** $\longrightarrow$ **Indexing Module**:
- The raw documents are fed in batches from the repository into the **Indexing Module**.
 - The indexing module strips HTML markup and applies natural language processing: **tokenization** (word segmentation), **stop-word removal** (eliminating uninformative words), and **stemming** (reducing inflected variants to a canonical root).
3. **Indexing Module** $\longrightarrow$ **Inverted Index**:
- The stream of $(\text{term}, \text{docID}, \text{position})$ tuples generated by the indexing module is sorted and structured.
 - Distinct terms populate the **Vocabulary (Lexicon)**, while occurrence records are appended to sorted **Postings Lists** on disk/memory, forming the searchable **Inverted Index**.
4. **User** $\longrightarrow$ **Query Processor** $\longleftrightarrow$ **Inverted Index**:
- The user inputs a query through the **User Interface**.
 - The **Query Processor** parses the query string, applies identical tokenization/stemming, and accesses the **Inverted Index** to look up the vocabulary entries and retrieve the posting lists for each query term.
 - It executes list intersections (**AND**), unions (**OR**), or differences (**NOT**).
5. **Query Processor** $\longrightarrow$ **Ranking Engine** $\longrightarrow$ **User Interface**:
- The candidate document set identified by the query processor is passed to the **Ranking Engine**.
 - The ranking engine computes relevance scores using content-based metrics (e.g., **TF-IDF / Cosine Similarity**) combined with static query-independent quality metrics (e.g., **PageRank** from the web graph).
 - The top- k ranked documents pass through the **Snippet Generator** (which extracts context windows and highlights matched keywords) and are delivered to the **User Interface**.
-

Q.5 (b) Step-by-Step Posting List Evaluation [8 Marks]

(Ref. [1], Goodrich et al., Ch. 12.5.3)

Given Posting Lists:

- $L_{\text{cloud}} = [1, 3, 5, 7, 9, 11]$
 - $L_{\text{computing}} = [2, 3, 6, 7, 10, 11]$
 - $L_{\text{security}} = [3, 4, 7, 8, 11, 12]$
-

Query 1: `"cloud" AND "computing"``

Operation: **Intersection** ($L_{\text{cloud}} \cap L_{\text{computing}}$)

We trace the two-pointer merge algorithm:

- Pointer p_1 on L_{cloud} , Pointer p_2 on $L_{\text{computing}}$.

Step	p_1 Val (L_{cloud})	p_2 Val ($L_{\text{computing}}$)	Comparison	Action Taken	Result List
1	1	2	$1 < 2$	Advance p_1	[]
2	3	2	$3 > 2$	Advance p_2	[]

Step	p_1 Val (L_{cloud})	p_2 Val ($L_{computing}$)	Comparison	Action Taken	Result List
3	3	3	$3 == 3$	Match: Append 3 ; advance p_1, p_2	[3]
4	5	6	$5 < 6$	Advance p_1	[3]
5	7	6	$7 > 6$	Advance p_2	[3]
6	7	7	$7 == 7$	Match: Append 7 ; advance p_1, p_2	[3, 7]
7	9	10	$9 < 10$	Advance p_1	[3, 7]
8	11	10	$11 > 10$	Advance p_2	[3, 7]
9	11	11	$11 == 11$	Match: Append 11 ; advance p_1, p_2	[3, 7, 11]

- Both lists exhausted.

Result of Query 1 = [3, 7, 11]

Query 2: `("cloud" AND "computing") AND NOT "security"``

Operation: **Difference** ($L_{Q1} \setminus L_{security}$)

- From Query 1: $L_{Q1} = [3, 7, 11]$
- Negation List: $L_{security} = [3, 4, 7, 8, 11, 12]$

We trace the two-pointer set difference algorithm:

- Pointer q_1 on L_{Q1} , Pointer q_2 on $L_{security}$.

Step	q_1 Val (L_{Q1})	q_2 Val ($L_{security}$)	Comparison	Action Taken	Result List
1	3	3	$3 == 3$	Match in Negation: Discard 3 ; advance q_1, q_2	[]
2	7	4	$7 > 4$	Advance q_2	[]
3	7	7	$7 == 7$	Match in Negation: Discard 7 ; advance q_1, q_2	[]
4	11	8	$11 > 8$	Advance q_2	[]
5	11	11	$11 == 11$	Match in Negation: Discard 11 ; advance q_1, q_2	[]

- List L_{Q1} exhausted. No elements survive.

Result of Query 2 = $\emptyset$ (Empty Set / No Matching Documents)

Query 3: `("cloud" OR "security") AND "computing"``

Step 1: Evaluate Sub-expression $L_{union} = L_{cloud} \cup L_{security}$

Using the two-pointer sorted union merge on:

- $L_{cloud} = [1, 3, 5, 7, 9, 11]$
- $L_{security} = [3, 4, 7, 8, 11, 12]$
- Compare 1 and 3: Append 1 , advance L_{cloud}
- Compare 3 and 3: Append 3 , advance both
- Compare 5 and 4: Append 4 , advance $L_{security}$
- Compare 5 and 7: Append 5 , advance L_{cloud}
- Compare 7 and 7: Append 7 , advance both
- Compare 9 and 8: Append 8 , advance $L_{security}$
- Compare 9 and 11: Append 9 , advance L_{cloud}
- Compare 11 and 11: Append 11 , advance both

$$L_{\text{union}} = [1, 3, 4, 5, 7, 8, 9, 11, 12]$$

Step 2: Intersect with $L_{\text{computing}} = [2, 3, 6, 7, 10, 11]$

Operation: $L_{\text{final}} = L_{\text{union}} \cap L_{\text{computing}}$

Step	u Val (L_{union})	c Val ($L_{\text{computing}}$)	Comparison	Action Taken	Result List
1	1	2	$1 < 2$	Advance L_{union}	[]
2	3	2	$3 > 2$	Advance $L_{\text{computing}}$	[]
3	3	3	$3 == 3$	Match: Append 3 ; advance both	[3]
4	4	6	$4 < 6$	Advance L_{union}	[3]
5	5	6	$5 < 6$	Advance L_{union}	[3]
6	7	6	$7 > 6$	Advance $L_{\text{computing}}$	[3]
7	7	7	$7 == 7$	Match: Append 7 ; advance both	[3, 7]
8	8	10	$8 < 10$	Advance L_{union}	[3, 7]
9	9	10	$9 < 10$	Advance L_{union}	[3, 7]
10	11	10	$11 > 10$	Advance $L_{\text{computing}}$	[3, 7]
11	11	11	$11 == 11$	Match: Append 11 ; advance both	[3, 7, 11]
12	12	end	—	$L_{\text{computing}}$ exhausted $\implies$ Terminate	[3, 7, 11]

$$\text{Result of Query 3} = [3, 7, 11]$$

Q.6 (a) Bidirectional Reductions: Independent Set $\iff$ Vertex Cover [7 Marks]

(Ref. [3], Kleinberg & Tardos, Ch. 8.1, 8.2)

1. Foundational Complement Duality Theorem

Theorem: In any undirected graph $G = (V, E)$, a subset $S \subseteq V$ is an independent set $\iff V \setminus S$ is a vertex cover.

- **Forward Direction ($\implies$):**
Let $S \subseteq V$ be an independent set. Consider an arbitrary edge $e = (u, v) \in E$.
By definition of an independent set, vertices u and v cannot both belong to S .
Therefore, at least one of the endpoints must belong to the complement:

$$u \in (V \setminus S) \quad \text{or} \quad v \in (V \setminus S).$$

Since this condition holds for every edge in E , the complement $V \setminus S$ covers all edges in E and is therefore a valid vertex cover of G .

- **Reverse Direction ($\impliedby$):**
Let $V \setminus S$ be a vertex cover of G . We prove that S is an independent set by contradiction.
Suppose S is not an independent set. Then there must exist two adjacent vertices $u, v \in S$ such that $(u, v) \in E$.
If $u \in S$ and $v \in S$, neither vertex belongs to the complement $V \setminus S$.
Consequently, the edge (u, v) has neither of its endpoints in $V \setminus S$, contradicting the fact that $V \setminus S$ is a vertex cover.
Hence, no two vertices in S can be joined by an edge, and S is an independent set. ■

2. Reduction 1: Independent Set $\leq_P$ Vertex-Cover

- **Instance Transformation:**
Given an arbitrary instance of Independent Set: $\langle G = (V, E), k \rangle$.

Construct the instance of Vertex Cover:

$$\langle \mathbf{G}' = (\mathbf{V}, \mathbf{E}), \mathbf{k}' = |\mathbf{V}| - \mathbf{k} \rangle.$$

- **Parameter Adjustment:**

$$k_{\text{VC}} = |V| - k_{\text{IS}}.$$

- **Computational Complexity:** Graph G is copied directly and $k' = |V| - k$ is computed in $O(|V| + |E|)$ time (strictly polynomial/linear time).
- **Correctness Proof:**

$$\begin{aligned} \langle G, k \rangle \text{ is a YES-instance for Independent Set} &\iff \exists S \subseteq V \text{ such that } S \text{ is an independent set and } |S| \geq k \\ &\iff V \setminus S \text{ is a vertex cover of } G \text{ (by Complement Duality)} \\ &\iff |V \setminus S| = |V| - |S| \leq |V| - k = k' \\ &\iff \langle G', k' \rangle \text{ is a YES-instance for Vertex Cover.} \end{aligned}$$

3. Reduction 2: Vertex-Cover $\leq_P$ Independent Set

- **Instance Transformation:**

Given an arbitrary instance of Vertex Cover: $\langle G = (V, E), k \rangle$.

Construct the instance of Independent Set:

$$\langle \mathbf{G}'' = (\mathbf{V}, \mathbf{E}), \mathbf{k}'' = |\mathbf{V}| - \mathbf{k} \rangle.$$

- **Parameter Adjustment:**

$$k_{\text{IS}} = |V| - k_{\text{VC}}.$$

- **Computational Complexity:** Takes $O(|V| + |E|)$ time.
- **Correctness Proof:**

$$\begin{aligned} \langle G, k \rangle \text{ is a YES-instance for Vertex Cover} &\iff \exists C \subseteq V \text{ such that } C \text{ is a vertex cover and } |C| \leq k \\ &\iff V \setminus C \text{ is an independent set of } G \text{ (by Complement Duality)} \\ &\iff |V \setminus C| = |V| - |C| \geq |V| - k = k'' \\ &\iff \langle G'', k'' \rangle \text{ is a YES-instance for Independent Set.} \end{aligned}$$

4. Explanation of the Parameter Adjustment: $k \longleftrightarrow |V| - k$

1. **Partitioning of the Vertex Set:** In any graph $G = (V, E)$, the vertex set V is partitioned into two mutually exclusive and exhaustive subsets: an independent set S and its complement vertex cover $C = V \setminus S$. Thus:

$$|S| + |C| = |V|.$$

2. **Direction Inversion (Max vs. Min):**

- **Independent Set** is a maximization objective evaluated against a lower-bound threshold: $|S| \geq k$.
- **Vertex Cover** is a minimization objective evaluated against an upper-bound threshold: $|C| \leq k'$.

3. Because selecting more vertices for S directly reduces the number of vertices remaining for C , satisfying a lower bound of k on $|S|$ is mathematically identical to satisfying an upper bound of $|V| - k$ on $|C|$:

$$|S| \geq k \iff |V| - |S| \leq |V| - k \iff |C| \leq |V| - k.$$

Q.6 (b) Failure Function Construction and KMP Trace for $P = \text{"ABABCABAB"}$ [8 Marks]

(Ref. [1], Goodrich et al., Ch. 12.3.3)

1. Failure Function Table $f[0 \dots 8]$ for $P = \text{”ABABCABAB”}$

Pattern length: $m = 9$.

Characters: $P[0] = \text{'A'}$, $P[1] = \text{'B'}$, $P[2] = \text{'A'}$, $P[3] = \text{'B'}$, $P[4] = \text{'C'}$, $P[5] = \text{'A'}$, $P[6] = \text{'B'}$, $P[7] = \text{'A'}$, $P[8] = \text{'B'}$.

i	Substring $P[0 \dots i]$	Proper Prefixes	Proper Suffixes	Longest Common Prefix-Suffix	$f(i)$
0	"A"	$\emptyset$	$\emptyset$	None	0
1	"AB"	{ "A" }	{ "B" }	None	0
2	"ABA"	{ "A", "AB" }	{ "A", "BA" }	"A"	1
3	"ABAB"	{ "A", "AB", "ABA" }	{ "B", "AB", "BAB" }	"AB"	2
4	"ABABC"	{ "A", ..., "ABAB" }	{ "C", "BC", "ABC", "BABC" }	None	0
5	"ABABCA"	{ "A", ..., "ABABC" }	{ "A", "CA", "BCA", ... }	"A"	1
6	"ABABCAB"	{ "A", "AB", ... }	{ "B", "AB", "CAB", ... }	"AB"	2
7	"ABABCABA"	{ "A", "AB", "ABA", ... }	{ "A", "BA", "ABA", ... }	"ABA"	3
8	"ABABCABAB"	{ "A", "AB", "ABA", "ABAB", ... }	{ "B", "AB", "BAB", "ABAB", ... }	"ABAB"	4

$f = [0, 0, 1, 2, 0, 1, 2, 3, 4]$

2. KMP Execution Trace on $T = \text{”ABABABABCABAB”}$

- Text T ($n = 13$):

Index:	0	1	2	3	4	5	6	7	8	9	10	11	12
Char:	A	B	A	B	A	B	A	B	C	A	B	A	B

- Pattern P ($m = 9$): "ABABCABAB"
- Failure Table f : [0, 0, 1, 2, 0, 1, 2, 3, 4]

Complete Step-by-Step Execution Table:

Step	Text Index (i)	$T[i]$	Pattern Index (j)	$P[j]$	Comparison	Action / Fallback Rule Applied
1	0	'A'	0	'A'	Match	$i \leftarrow 1, j \leftarrow 1$
2	1	'B'	1	'B'	Match	$i \leftarrow 2, j \leftarrow 2$
3	2	'A'	2	'A'	Match	$i \leftarrow 3, j \leftarrow 3$
4	3	'B'	3	'B'	Match	$i \leftarrow 4, j \leftarrow 4$
5	4	'A'	4	'C'	MISMATCH	$j > 0 \implies j \leftarrow f[4 - 1] = f[3] = 2$; (i stays 4)
6	4	'A'	2	'A'	Match	$i \leftarrow 5, j \leftarrow 3$
7	5	'B'	3	'B'	Match	$i \leftarrow 6, j \leftarrow 4$
8	6	'A'	4	'C'	MISMATCH	$j > 0 \implies j \leftarrow f[4 - 1] = f[3] = 2$; (i stays 6)
9	6	'A'	2	'A'	Match	$i \leftarrow 7, j \leftarrow 3$
10	7	'B'	3	'B'	Match	$i \leftarrow 8, j \leftarrow 4$
11	8	'C'	4	'C'	Match	$i \leftarrow 9, j \leftarrow 5$
12	9	'A'	5	'A'	Match	$i \leftarrow 10, j \leftarrow 6$
13	10	'B'	6	'B'	Match	$i \leftarrow 11, j \leftarrow 7$
14	11	'A'	7	'A'	Match	$i \leftarrow 12, j \leftarrow 8$

Step	Text Index (i)	$T[i]$	Pattern Index (j)	$P[j]$	Comparison	Action / Fallback Rule Applied
15	12	'B'	8	'B'	MATCH	Full Match Detected! ($j = m - 1 = 8$)

3. Summary of Transitions and Final Match Location

- Mismatch Fallback Transitions ($j = f[j - 1]$):

- At Step 5 ($i = 4$):** $T[4] = \text{'A'} \neq P[4] = \text{'C'}$.
Fallback: $j \leftarrow f[4 - 1] = f[3] = 2$.
Pointer i remains stationary at 4.
- At Step 8 ($i = 6$):** $T[6] = \text{'A'} \neq P[4] = \text{'C'}$.
Fallback: $j \leftarrow f[4 - 1] = f[3] = 2$.
Pointer i remains stationary at 6.

- Final Match Location Calculation:

$$\text{Starting Index} = i - m + 1 = 12 - 9 + 1 = 4.$$

- Verification:

$$T[4 \dots 12] = \text{"ABABCABAB"} = P.$$

The KMP algorithm successfully terminates and outputs that the pattern occurs at **starting index 4**.